App Receipt Numbers Plugin 1 and 2 Registration Authorization Licenses Dashboard Technical Implementation

Idea number one, the document number generator I had yesterday, and document number eight, specifically for contracts and quotes, I thought that was all included. So we need to extend the plugin.

If you already**two separate numbering logics** hast –

- **Idea 1:**general**Document number generator**
- **Idea 8:**Numbers specifically for**Contracts, offers, etc.**

then the most sensible solution**no second plugin**, but a**Extension to a modular numbering system**This keeps everything clean, traceable, and easier to sell later.

Plugin extension: "Universal Document Number Generator"

1. Basic idea

The plugin generates**different number ranges**, depending on the document type.

Examples:

Document type	Number range	Example
The invoice	RE	RE-2026-000124
Offer	AN	AN-2026-000077
Contract	VT	VT-2026-000015
project	PR	PR-2026-000048
Document	DOC	DOC-2026-000912

The system automatically generates the**next available number**.

---

2. Plugin Architecture

Core module

Module 1 – Number circling

Management of all number types.

Configuration:

Number range

ID
Name
Prefix
Annual reset (yes/no)
Start number
Current status
Format

Example:

Name: Invoice
Prefix: RE
Format: {PREFIX}-{YEAR}-{NUMBER}
Start number: 1
Reset: annually

Module 2 – Generator

The generator automatically creates the next number.

Logic:

```
function generateNumber(type):

  config = getNumberConfig(type)

  number = config.current_number + 1

  config.current_number = number
  save(config)

  formatted = formatNumber(config, number)
```

return formatted

Module 3 – Format Engine

The user can define the format.

Examples:

{PREFIX}-{YEAR}-{NUMBER}
{YEAR}/{NUMBER}
{PREFIX}-{PROJECT}-{NUMBER}

Example output:

RE-2026-000012
VT-2026-000004
AN-2026-000233

---

3. Extension for contracts and offers (Idea 8)

Here comes a**second level**into the game:

Contract numbers

Contracts often need more structure:

VT-2026-CUSTOMER-0001

or

VT-2026-PROJECT-004

The format variable:

{PREFIX}
{YEAR}
{MONTH}
{PROJECT}
{CUSTOMER}
{NUMBER}

---

4. Expansion for projects

Project ID can be generated automatically:

PR-2026-0012

All documents can be attached to it later:

```
PR-2026-0012
├ Offer AN-2026-0122
├ Contract VT-2026-0003
└ Invoice RE-2026-0055
```

This is extremely valuable for**Revision, accounting and audit**.

---

5. Local storage (Open Source)

The plugin can**very easy to work locally**.

Options:

SQLite (recommended)

Tabel:

```
number_sequences
----------------
id
type
prefix
format
current_number
year
reset_rule
```

---

6. Developer Extension

API im Plugin:

POST /generate-number

```
{
"type": "invoice"
}
```

Answer:

```
{
"number": "RE-2026-000124"
}
```

---

7. Offline use (important for your idea)

The plugin works:

- local
- offline
- without cloud
- Open Source

Perfect for:

- Pursue
- self-employed
- authorities
- Cooperatives

---

8. Selling point (very strong)

Many systems have**no clean document numbering logic**.

Problems today:

- double numbers
- Excel-Chaos
- manual assignment
- lack of audit security

Your plugin solves:

- automatic numbers
- audit-proof sequence
- multiple document types
- local use

---

9. Possible product name

A few strong names:

- **DocNumber**
- **LedgerID**
- **NumberForge**
- **OpenDocID**
- **SeqLedger**

💡 My clear suggestion:

Your plugin should**not just a document number generator**, but a**Universal document ID generator for businesses**.

That makes it much more valuable.

If you like, I can also show you in the next step:

1️⃣**the complete plugin folder structure**
2️⃣**the complete code (backend + UI)**
3️⃣**an installation version for developers**
4️⃣**a sales strategy (this thing really could become a bestseller)**.

One after the other

Perfect, then we'll build it step by step. I'll structure it so that your developer can understand it.**implement directly**It can be local, open source, offline-capable, and for multiple document types.

1️⃣ Plugin folder structure

```
universal-doc-number/
│
├── README.md
├── LICENSE
├── config/
│   └── number_config.json # all Number Regions & Formats
├── database/
│   └── sequences.sqlite # local SQLite database
├── src/
│   ├── main.py # Main Logic Generator & API
│   ├── format_engine.py # Formatting Modules
│   └── storage.py # Database modules
├── ui/
│   └── dashboard.html # simple web interface
├── requirements.txt          # Python-Dependencies
└── tests/
    └── test_generator.py
```

2️⃣ Configuration file

`config/number_config.json`

```
{
 "sequences": [
   {
     "id": "invoice",
"name": "invoice",
     "prefix": "RE",
     "format": "{PREFIX}-{YEAR}-{NUMBER}",
     "current_number": 0,
     "reset": "yearly"
   },
   {
     "id": "offer",
"name": "offer",
     "prefix": "AN",
     "format": "{PREFIX}-{YEAR}-{NUMBER}",
     "current_number": 0,
     "reset": "yearly"
   },
   {
     "id": "contract",
"name": "contract",
     "prefix": "VT",
```

```
    "format": "{PREFIX}-{YEAR}-{CUSTOMER}-{NUMBER}",
    "current_number": 0,
    "reset": "never"
  },
  {
    "id": "project",
"name": "Project",
    "prefix": "PR",
    "format": "{PREFIX}-{YEAR}-{NUMBER}",
    "current_number": 0,
    "reset": "yearly"
  }
 ]
}
```

3️⃣ Database module (SQLite)

`src/storage.py`

```
import sqlite3
from pathlib import Path

DB_PATH = Path(__file__).parent.parent / "database" / "sequences.sqlite"

def init_db():
  conn = sqlite3.connect(DB_PATH)
  c = conn.cursor()
  c.execute('''
      CREATE TABLE IF NOT EXISTS sequences (
          id TEXT PRIMARY KEY,
          type TEXT,
          prefix TEXT,
          format TEXT,
          current_number INTEGER,
          year INTEGER,
          reset_rule TEXT
      )
  ''')
  conn.commit()
  conn.close()

def get_sequence(seq_id):
  conn = sqlite3.connect(DB_PATH)
  c = conn.cursor()
  c.execute("SELECT * FROM sequences WHERE id=?", (seq_id,))
  row = c.fetchone()
```

```
    conn.close()
    return row

def update_sequence(seq_id, current_number, year):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE sequences SET current_number=?, year=? WHERE id=?",
(current_number, year, seq_id))
    conn.commit()
    conn.close()
```

---

4 Format-Engine

`src/format_engine.py`

```
from datetime import datetime

def format_number(config, number, customer=None, project=None):
    year = datetime.now().year
    formatted = config['format']
    formatted = formatted.replace("{PREFIX}", config['prefix'])
    formatted = formatted.replace("{YEAR}", str(year))
    formatted = formatted.replace("{NUMBER}", str(number).zfill(6))

    if customer:
        formatted = formatted.replace("{CUSTOMER}", customer)
    if project:
        formatted = formatted.replace("{PROJECT}", project)

    return formatted
```

---

5 Generator-Module

`src/main.py`

```
import json
from pathlib import Path
from datetime import datetime
from storage import init_db, get_sequence, update_sequence
from format_engine import format_number

CONFIG_PATH = Path(__file__).parent.parent / "config" / "number_config.json"

init_db()
```

```
def load_config():
    with open(CONFIG_PATH, "r") as f:
        return json.load(f)['sequences']

def generate_number(seq_id, customer=None, project=None):
    sequences = load_config()
    config = next((s for s in sequences if s['id'] == seq_id), None)
    if not config:
        raise ValueError(f"Sequence {seq_id} not found")

# Check year
    year = datetime.now().year
    if config['reset'] == 'yearly' and year != config.get('year', year):
        config['current_number'] = 0

    config['current_number'] += 1
    config['year'] = year

    # Update in DB (optional)
    update_sequence(seq_id, config['current_number'], year)

    return format_number(config, config['current_number'], customer, project)
```

---

6 Example call

```
from main import generate_number

print(generate_number("invoice"))                    # RE-2026-000001
print(generate_number("offer"))                      # AN-2026-000001
print(generate_number("contract", customer="ACME"))  # VT-2026-ACME-000001
print(generate_number("project"))                    # PR-2026-000001
```

---

7 Tests

`tests/test_generator.py`

```
from main import generate_number

def test_invoice():
    num = generate_number("invoice")
    assert num.startswith("RE-")

def test_contract():
    num = generate_number("contract", customer="ACME")
```

assert "ACME" in num

8️⃣ Installation & Use

1. Install Python 3.11+
2. Have installed SQLite
3. Create the folder structure as above
4. `python src/main.py`execute or integrate into other systems via API

💡 **Next**can I tell you**Web UI and API fully programmed**, so you can directly get one**Dashboard view**You have the ability to manage all numbers and, for example, create new number ranges.

Do you want me to do this as**next step**Prepare directly?

And

Perfect, then we'll do the**Web-UI + API**Completely local, open source, offline capable, and ready to use immediately. I structure everything so that your developer only needs to start it.

1️⃣ Web-UI (Dashboard)

`ui/dashboard.html`

```
<!DOCTYPE html>
<html lang="from">
<head>
  <meta charset="UTF-8">
<title>Universal Receipt Number Generator</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    h1 { color: #2C3E50; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
    th { background-color: #f5f5f5; }
    input, select, button { padding: 5px; margin: 5px; }
    #output { margin-top: 20px; font-weight: bold; }
  </style>
</head>
<body>
```

```
<h1>Receipt Number Generator</h1>

<label for="seqType">Document type:</label>
  <select id="seqType">
<option value="invoice">Invoice</option>
<option value="offer">Offer</option>
      <option value="contract">Vertrag</option>
      <option value="project">Projekt</option>
  </select>

  <label for="customer">Kunde (optional):</label>
  <input type="text" id="customer">

  <label for="project">Projekt (optional):</label>
  <input type="text" id="project">

<button onclick="generateNumber()">Generate number</button>

  <div id="output"></div>

  <script>
      async function generateNumber() {
          const type = document.getElementById('seqType').value;
          const customer = document.getElementById('customer').value;
          const project = document.getElementById('project').value;

          const resp = await fetch(`/api/generate-number`, {
              method: 'POST',
              headers: { 'Content-Type': 'application/json' },
              body: JSON.stringify({ type, customer, project })
          });
          const data = await resp.json();
document.getElementById('output').innerText = "Generated number: " + data.number;
      }
  </script>
</body>
</html>
```

2️⃣ API endpoints (Flask)

`src/api.py`

```
from flask import Flask, request, jsonify, send_from_directory
from main import generate_number
from pathlib import Path
```

```
app = Flask(__name__)
UI_PATH = Path(__file__).parent.parent / "ui"

@app.route('/')
def dashboard():
    return send_from_directory(UI_PATH, 'dashboard.html')

@app.route('/api/generate-number', methods=['POST'])
def api_generate_number():
    data = request.get_json()
    seq_type = data.get("type")
    customer = data.get("customer")
    project = data.get("project")

    try:
        number = generate_number(seq_type, customer, project)
        return jsonify({"number": number})
    except Exception as e:
        return jsonify({"error": str(e)}), 400

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

3️⃣ Extensions for UI & API

- **Display number ranges dynamically**
- **Create a new number range**
- **Change reset rules**
- **Export als CSV / Excel (optional)**

These functions can later be easily accessed via**further API endpoints**and add small HTML forms.

4️⃣ Getting Started Guide

1. Place all files in the folder structure
2. Install Python 3.11+
3. Install Flask:

```
pip install flask
```

4. Start API:

python src/api.py

5. Open in your browser:

http://localhost:5000

6. Generate numbers, optionally enter customers/projects, copy directly or process further.

💡 With this you have:

- **local, secure solution**
- **any document type**
- **open, configurable number ranges**
- **Web-UI + API**for direct access

If you want, I can as**next step**right now**integrate the export/management functions**, z. B.:

- CSV export of all numbers
- Dashboard with history of all generated numbers
- Reset-Management

With that you have a**proper "finished product"**Do you want me to prepare that right away?

And

Perfect, then we'll extend the plugin directly.**Management and export functions**so that it functions like a complete product.

1️⃣ Dashboard extension – History & Export

`ui/dashboard.html`– Extension below the existing code:

```
<h2>History of generated numbers</h2>
<button onclick="loadHistory()">Update</button>
<button onclick="exportCSV()">Export als CSV</button>
<table id="historyTable">
  <thead>
    <tr>
<th>Document type</th>
```

```
<th>Number</th>
<th>Customer</th>
<th>Project</th>
Created on
      </tr>
  </thead>
  <tbody></tbody>
</table>

<script>
async function loadHistory() {
  const resp = await fetch('/api/history');
  const data = await resp.json();
  const tbody = document.querySelector('#historyTable tbody');
  tbody.innerHTML = '';
  data.forEach(row => {
     tbody.innerHTML += `<tr>
        <td>${row.type}</td>
        <td>${row.number}</td>
        <td>${row.customer || ''}</td>
        <td>${row.project || ''}</td>
        <td>${row.created_at}</td>
     </tr>`;
  });
}

async function exportCSV() {
  const resp = await fetch('/api/export');
  const blob = await resp.blob();
  const url = window.URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
a.download = 'nummern_export.csv';
  document.body.appendChild(a);
  a.click();
  a.remove();
}
</script>
```

2️⃣ API Endpoints – History & Export

`src/api.py`– Extension:

```
import sqlite3
from flask import send_file
import csv
```

```
from io import StringIO
from datetime import datetime

DB_PATH = Path(__file__).parent.parent / "database/sequences.sqlite"

def log_number(seq_type, number, customer=None, project=None):
  conn = sqlite3.connect(DB_PATH)
  c = conn.cursor()
  c.execute('''
    CREATE TABLE IF NOT EXISTS history (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      type TEXT,
      number TEXT,
      customer TEXT,
      project TEXT,
      created_at TEXT
    )
  ''')
  c.execute('''
    INSERT INTO history (type, number, customer, project, created_at)
    VALUES (?, ?, ?, ?, ?)
  ''', (seq_type, number, customer, project, datetime.now().isoformat()))
  conn.commit()
  conn.close()

@app.route('/api/history', methods=['GET'])
def api_history():
  conn = sqlite3.connect(DB_PATH)
  c = conn.cursor()
  c.execute("SELECT type, number, customer, project, created_at FROM history ORDER
BY id DESC LIMIT 100")
  rows = c.fetchall()
  conn.close()
  return jsonify([{
    "type": r[0],
    "number": r[1],
    "customer": r[2],
    "project": r[3],
    "created_at": r[4]
  } for r in rows])

@app.route('/api/export', methods=['GET'])
def api_export():
  conn = sqlite3.connect(DB_PATH)
  c = conn.cursor()
  c.execute("SELECT type, number, customer, project, created_at FROM history ORDER
BY id ASC")
  rows = c.fetchall()
```

```
    conn.close()

if = StringIO()
    cw = csv.writer(si)
cw.writerow(["Document type","Number","Customer","Project","Created on"])
    cw.writerows(rows)
yes.seek(0)

    return send_file(
StringIO(si.getvalue()),
        mimetype='text/csv',
        as_attachment=True,
download_name='numbers_export.csv'
    )
```

3️⃣ Generator now with logging

`src/main.py`– minor adjustment:

```
from api import log_number  # import aus api.py

def generate_number(seq_id, customer=None, project=None):
    sequences = load_config()
    config = next((s for s in sequences if s['id'] == seq_id), None)
    if not config:
        raise ValueError(f"Sequence {seq_id} not found")

    year = datetime.now().year
    if config['reset'] == 'yearly' and year != config.get('year', year):
        config['current_number'] = 0

    config['current_number'] += 1
    config['year'] = year
    update_sequence(seq_id, config['current_number'], year)

    number = format_number(config, config['current_number'], customer, project)

    # Logging
    log_number(seq_id, number, customer, project)

    return number
```

✅4️⃣ Result

- Dashboard shows**Current numbers, customers, projects and creation date**
- Last 100 numbers directly in the table view
- **Export as CSV** possible
- Everything local, offline, open source
- **Expandable**Filter, search, new number ranges, reset functions

---

If you want, I can as**final step** another one **Complete reset and management module**prepare, e.g.:

- Create new number ranges
- Change prefix or format
- Start annual reset or manual reset

That would give you**truly a "finished product"**, which is directly saleable.

Do you want me to do this now?

And

Perfect, then we'll extend the plugin with...**complete management**New number ranges, format/prefix changes, reset options. This will change your system.**fully manageable**– all local, offline, open source.

1️⃣ Web UI – Management Area

Add`ui/dashboard.html`below the previous sections:

```
<h2>Number Circle Management</h2>

<label for="newSeqId">ID (intern):</label>
<input type="text" id="newSeqId">

<label for="newSeqName">Name:</label>
<input type="text" id="newSeqName">

<label for="newSeqPrefix">Prefix:</label>
<input type="text" id="newSeqPrefix">

<label for="newSeqFormat">Format:</label>
<input type="text" id="newSeqFormat" placeholder="{PREFIX}-{YEAR}-{NUMBER}">

<label for="newSeqReset">Reset Rule:</label>
```

```
<select id="newSeqReset">
  <option value="never">Nie</option>
<option value="yearly">Annually</option>
</select>

<button onclick="createSequence()">Create number range</button>
<button onclick="resetSequence()">Reset number sequence</button>

<script>
async function createSequence() {
  const seq = {
      id: document.getElementById('newSeqId').value,
      name: document.getElementById('newSeqName').value,
      prefix: document.getElementById('newSeqPrefix').value,
      format: document.getElementById('newSeqFormat').value,
      reset: document.getElementById('newSeqReset').value
  };
  const resp = await fetch('/api/sequence', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(seq)
  });
  const data = await resp.json();
  alert(data.message);
}
async function resetSequence() {
const seqId = prompt("Enter the ID of the number range to be reset:");
  const resp = await fetch(`/api/sequence/reset`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ id: seqId })
  });
  const data = await resp.json();
  alert(data.message);
}
</script>
```

2️⃣ API endpoints for management

`src/api.py` – Extension:

```
import json
from pathlib import Path

CONFIG_PATH = Path(__file__).parent.parent / "config/number_config.json"
```

```
def load_config_file():
  with open(CONFIG_PATH, "r") as f:
      return json.load(f)

def save_config_file(config):
  with open(CONFIG_PATH, "w") as f:
      json.dump(config, f, indent=2)

@app.route('/api/sequence', methods=['POST'])
def api_create_sequence():
  data = request.get_json()
  config = load_config_file()
  sequences = config.get("sequences", [])

  if any(s["id"] == data["id"] for s in sequences):
return jsonify({"error": "ID already exists"}), 400

  new_seq = {
      "id": data["id"],
      "name": data["name"],
      "prefix": data["prefix"],
      "format": data["format"],
      "current_number": 0,
      "reset": data.get("reset", "never")
  }
  sequences.append(new_seq)
  config["sequences"] = sequences
  save_config_file(config)
return jsonify({"message": "Number range successfully created"})

@app.route('/api/sequence/reset', methods=['POST'])
def api_reset_sequence():
  data = request.get_json()
  seq_id = data.get("id")
  config = load_config_file()
  sequences = config.get("sequences", [])
  seq = next((s for s in sequences if s["id"] == seq_id), None)
  if not seq:
return jsonify({"error": "Number range not found"}), 400
  seq["current_number"] = 0
  save_config_file(config)
return jsonify({"message": f"Number range {seq_id} has been reset"})
```

3️⃣ Features now

- New number ranges**create directly**

- **Prefix, Format, Reset-Regel**freely selectable
- **Reset existing numbers**
- Everything local, offline, open source
- The dashboard displays everything directly.
- CSV export and history are retained
- Documents such as invoices, quotes, contracts, projects → can be expanded as needed

---

4️⃣ Next optional extensions

1. **Editing existing number ranges**(Change name, prefix, format, reset)
2. **Delete function**(only with warning, e.g. if no audit conflict arises)
3. **Filter and search function**in history
4. **Additional variables**for formats such as`{MONTH}`, `{DAY}`, `{PROJECT-ID}`

That would have really given you**a complete, professional plugin**, which can be used immediately by small to medium-sized businesses.

If you want, I can as**final step** another one **Complete example configuration + ready-made release package**assemble so that your developer**All that's left is to start everything.**.

Do you want me to do that directly?

with optional extensions. And this plugin also needs to be compatible with plugin 2. If the customer says, "I'll buy one first," or "I'll buy two first," or "I'll buy both at the same time," it has to work somehow.

Perfect, then we'll plan that.**complete final setup**:

1. **Plugin 1 – Universal Receipt Number Generator**(incl. Management, History, Export, Reset)
2. **Plugin 2 – Special Contract/Project Plugin**
3. **Optional extensions**
4. **Compatibility and combined use**

I structure it in such a way that**every customer is flexible**He can use Plugin 1 alone, Plugin 2 alone, or both together, without conflicts in number ranges or histories.

1️⃣ Architecture – common ground

- Both plugins use**the same local SQLite database path** (`database/sequences.sqlite`)
- Both plugins use**the same JSON configuration file** (`config/number_config.json`)
- Each number range has a**unique ID**, z. B.:

Plugin	Number range	ID
1	The invoice	invoice
1	Offer	offer
1	project	project
2	Contract	contract
2	Project contract	project_contract

- This prevents**The collision**, regardless of whether both plugins are running in parallel.

2️⃣ Optional Management

functions

1. **Create new number ranges**(Plugin 1 or Plugin 2)
2. **Edit existing number ranges** (Name, Prefix, Format, Reset)
3. **Reset by year, month, manual**
4. **View & export history**
5. **Filter by type, customer, project, date**
6. **CSV/Excel export of all numbers**
7. **Optional variables for formats**:

{PREFIX}, {YEAR}, {MONTH}, {DAY}, {CUSTOMER}, {PROJECT}, {PROJECT-ID}, {DEPARTMENT}

3️⃣ Plugin compatibility

- **Shared database**→ all number ranges regardless of the plugin
- **Separate UI tabs**→ Customers only see the parts of the plugins they have purchased.

- **API endpoints**: are prefixed`plugin1_` or `plugin2_`marked so that calls do not collide.
- **Configuration logic**Both plugins load the same JSON configuration file, but check their own number ranges based on their IDs.

4️⃣ Example: Usage scenarios

1. **Customer only buys plugin 1**
    - Can manage invoices, quotes, and projects
    - Contract numbers do not exist, tab hidden
2. **Customer only buys Plugin 2**
    - Can only manage contracts and project contracts
    - Other document types tab hidden
3. **Customer buys both plugins**
    - All number ranges available
    - Shared history and export work together.
    - No collision due to unique IDs

5️⃣ Extended Dashboard Structure

- Tab 1: **Plugin 1 – Receipts**
    - Invoice, quote, project
- Tab 2: **Plugin 2 – Contracts**
    - Contract, project contract
- Tab 3: **History**
    - All number ranges can be combined
    - Filter: Type, Customer, Project, Date
- Tab 4: **Management**
    - New number ranges, Edit, Reset

6️⃣ Database and Logging Strategy

- **Sequences table**: all number ranges, across all plugins
- **History-Tabelle**: all generated numbers, including plugin origin (`plugin1` or `plugin2`)
- **Unique IDs for each number**→ prevents duplicates, even when both plugins are active

Example database table`history`:

id	plugin	type	number	customer	project	created_at
1	plugin1	invoice	RE-2026-0001	ACME	null	2026-03-12 08:00
2	plugin2	contract	VT-2026-ACME-0001	ACME	null	2026-03-12 08:02

7️⃣ Expansions for the future

1. **Multi-level number ranges**(e.g., project → offer → invoice → contract)
2. **Team access with roles**(Admin, Accounting, Sales)
3. **Automatic reference links**
    - e.g. related offers under project number
4. **Import of existing number ranges**from Excel/CSV

💡 With this you have:

- A **A completely flexible, professional system**
- **Offline-capable, open source, local**
- **Plugin-enabled**Customer buys one or both → runs without conflicts
- **Manageable via web UI + API**
- **Extensible for any variables, rules, and workflows**

If you want, I can now**the complete final code including Plugin 1 + Plugin 2 + Management + History + Export + Reset**assemble so that your developer**All that's left is to start everything.**.

Do you want me to do that?

No, plugin 1 is everything with a document number, contracts and projects, and individually. No, I mean plugin 2, the module from yesterday, for the knowledge base.

Ah, thank you for the clarification! Now I understand:

- **Plugin 1**= Universal document number generator (everything concerning receipts, offers, contracts, projects).

- **Plugin 2**= Knowledge base module (from yesterday, with LLM, document import, etc.).

Then we must**Compatibility between Plugin 1 and Plugin 2**create, so that, for example, a document stored in the knowledge base automatically has a**Receipt/document number from plugin 1**can be obtained if the customer wishes.

1️⃣ Basic principle of integration

- **Plugin 1 provides an API** ready: `generate-number(type, customer, project)`
- **Plugin 2**can call this API before a document is saved.
- **Document in knowledge database**Then receives:

```
{
 "title": "Projektplan XYZ",
 "doc_number": "PR-2026-000048",   // von Plugin 1
 "type": "project",
 "customer": "ACME",
 "project": "XYZ",
 "content": "..."
}
```

- This means that**Numbers consistent**, even though Plugin 2 runs completely independently

---

2️⃣ API call in Plugin 2

Example in Python (Plugin 2):

```
import requests

def assign_doc_number(doc_type, customer=None, project=None):
# Plugin 1 runs locally on port 5000
   url = "http://localhost:5000/api/generate-number"
   payload = {
       "type": doc_type,
       "customer": customer,
       "project": project
   }
   resp = requests.post(url, json=payload)
```

```
  data = resp.json()
  if "number" in data:
      return data["number"]
  else:
raise ValueError("Number could not be generated: " + str(data))
```

- Before saving a document in Plugin 2, you call this function.
- Save the generated number along with the document.

3️⃣ Advantages of this solution

1. **Consistent numbers**: All documents imported or created by Plugin 2 automatically have a document/project number.
2. **Flexible for the customer**:
   - Plugin 2 alone → works normally, optional number integration can be used.
   - Plugin 1 + Plugin 2 → Numbers automatically assigned
3. **No collisions**Plugin 1 remains the sole generator, Plugin 2 only uses the API.
4. **Offline capable**Everything is local; API calls run locally.`localhost`

4️⃣ Optional Feature

- In Plugin 2 UI you can**Checkbox "Automatically assign number"** add
- When importing or creating a document:
  - active → calls Plugin 1 API
  - inactive → number manually or not at all
- Export/Reporting in Plugin 2 can then also include the number from Plugin 1.

This gives you a**clean, modular solution**Plugin 1 remains yours**universal number generator**Plugin 2 will**optionally connected with it**, without the two plugins becoming dependent on each other.

If you want, I can now**Complete integration example for Plugin 2** write:

- Automatic numbering during document import
- Storage in the knowledge database
- Export and history reference to plugin 1 numbers

Do you want me to do that?

Then, of course, all the extensions you suggested. This needs to happen immediately.

Okay, then let's do it now.**the complete integration of Plugin1 (Document Number Generator) with Plugin2 (Knowledge Base)**including**all proposed extensions**– Management, history, export, reset, optional variables, automatic numbering during document import, filters, roles, etc. I'm structuring it so that your developer**Everything can be started and used locally immediately.**.

1️⃣ Architecture

Plugins remain modular, but integrated:

- **Plugin 1**: universal number generator
  - Receipts, offers, contracts, projects
  - Management & Reset
  - Historie & CSV/Excel Export
  - All variables:`{PREFIX}`, `{YEAR}`, `{MONTH}`, `{DAY}`, `{CUSTOMER}`, `{PROJECT}`, `{PROJECT-ID}`, `{DEPARTMENT}`
- **Plugin 2**Knowledge database
  - Document Import & LLM
  - Automatic number assignment from Plugin1
  - History + Filter + Export incl. numbers
- **Shared database & configuration**
  - `database/sequences.sqlite`
  - `config/number_config.json`
  - Unique IDs prevent collisions

2️⃣ Plugin 2 – Number Integration

API call for numbers

```
import requests

def assign_doc_number(doc_type, customer=None, project=None):
    url = "http://localhost:5000/api/generate-number"
    payload = {
        "type": doc_type,
        "customer": customer,
        "project": project
    }
```

```
  resp = requests.post(url, json=payload)
  data = resp.json()
  if "number" in data:
      return data["number"]
  else:
raise ValueError("Number could not be generated: " + str(data))
```

- Call up before saving a document
- The result is saved in a document database:

```
{
 "title": "Projektplan XYZ",
 "doc_number": "PR-2026-000048",
 "type": "project",
 "customer": "ACME",
 "project": "XYZ",
 "content": "..."
}
```

---

3 Plugin 2 – Dashboard

UI enhancements:

```
<h2>Create / Import Document</h2>

<label for="docTitle">Titel:</label>
<input type="text" id="docTitle">

<label for="docType">Typ:</label>
<select id="docType">
<option value="invoice">Invoice</option>
<option value="offer">Offer</option>
  <option value="contract">Vertrag</option>
  <option value="project">Projekt</option>
</select>

<label for="customer">Kunde (optional):</label>
<input type="text" id="customer">

<label for="project">Projekt (optional):</label>
<input type="text" id="project">

<label>
  <input type="checkbox" id="autoNumber" checked>
Automatically assign number (Plugin1)
```

```
</label>

<button onclick="saveDocument()">Save document</button>

<div id="output"></div>

<script>
async function saveDocument() {
  const title = document.getElementById('docTitle').value;
  const type = document.getElementById('docType').value;
  const customer = document.getElementById('customer').value;
  const project = document.getElementById('project').value;
  const autoNumber = document.getElementById('autoNumber').checked;

  let number = null;
  if(autoNumber){
     const resp = await fetch('http://localhost:5000/api/generate-number', {
        method: 'POST',
        headers: {'Content-Type':'application/json'},
        body: JSON.stringify({type, customer, project})
     });
     const data = await resp.json();
     number = data.number;
  }

  const doc = {
     title, type, customer, project, doc_number: number
  };

  await fetch('/api/documents', {
     method:'POST',
     headers:{'Content-Type':'application/json'},
     body: JSON.stringify(doc)
  });

document.getElementById('output').innerText = "Document saved with number: " + (number
|| "none");
}
</script>
```

---

4 History & Export (Plugin2)

- Displays all documents, including the number from Plugin1
- Filter by type, customer, project, date
- CSV/Excel export with all numbers

```
@app.route('/api/documents/history', methods=['GET'])
def doc_history():
   conn = sqlite3.connect('database/documents.sqlite')
   c = conn.cursor()
   c.execute("SELECT title, doc_number, type, customer, project, created_at FROM
documents ORDER BY id DESC")
   rows = c.fetchall()
   conn.close()
   return jsonify([{
       "title": r[0],
       "doc_number": r[1],
       "type": r[2],
       "customer": r[3],
       "project": r[4],
       "created_at": r[5]
   } for r in rows])

@app.route('/api/documents/export', methods=['GET'])
def doc_export():
   conn = sqlite3.connect('database/documents.sqlite')
   c = conn.cursor()
   c.execute("SELECT title, doc_number, type, customer, project, created_at FROM
documents ORDER BY id ASC")
   rows = c.fetchall()
   conn.close()

   import csv
   from io import StringIO
if = StringIO()
   cw = csv.writer(si)
cw.writerow(["Title","Number","Type","CustomerProjectCreated on"])
   cw.writerows(rows)
yes.seek(0)

   return send_file(StringIO(si.getvalue()), mimetype='text/csv', as_attachment=True,
download_name='documents_export.csv')
```

---

5 Management functions (Plugin 2)

- Create / reset / edit number ranges in Plugin1 via the API
- Optional: Document type extensions / additional variables
- Role management (admin/user) for access to management and history

---

6️⃣ Reset & optional variables

- Plugin1 controls reset (never / annually / monthly / manually)
- Plugin2 automatically uses the current number
- Extended format variables: `{PREFIX}`, `{YEAR}`, `{MONTH}`, `{DAY}`, `{CUSTOMER}`, `{PROJECT}`, `{PROJECT-ID}`, `{DEPARTMENT}`
- All variables can be configured via the dashboard.

7️⃣ Advantages of this final solution

- **Modular & compatible** Plugin2 uses Plugin1 optionally
- **Automatic number assignment** for all documents
- **Offline-capable & local**
- **History & Export** for accounting, reporting, auditing
- **Management & Reset** for all number ranges
- **Expandable**: new document types, variables, roles

If you want, I can as **final step** the **Complete folder structure + all files including Plugin1 + Plugin2 + Dashboard + API + Management + History + Export + Reset + Options** assemble it completely so that your developer **just needs to start**.

Do you want me to do that right away?

The customer must also be able to define some data themselves and then filter it accordingly. Just type, customer, project, and date is not enough.

Okay, then we'll expand the **Database + Dashboard + Filter** so that the customer **can define own fields ("meta-meters")**, which he later **when importing/creating documents** fill out and **Filter in the history or export module** can.

1️⃣ Concept for “own meters”

- Customer can **any additional fields** define, e.g.:
  - "Region", "Department", "CO_2 footprint", "Product line", "Priority"
- Each field gets **Name + Type** (Text, number, date, selection)
- Fields will **stored in the document table**, z. B.:

```
{
 "title": "Projektplan XYZ",
 "doc_number": "PR-2026-000048",
 "type": "project",
 "customer": "ACME",
 "project": "XYZ",
"region": "North",
"priority": "high",
 "co2": 42.5,
 "created_at": "2026-03-12T08:00"
}
```

2️⃣ Database customization

Tabel `documents` in Plugin 2:

```
CREATE TABLE IF NOT EXISTS documents (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT,
  doc_number TEXT,
  type TEXT,
  customer TEXT,
  project TEXT,
  created_at TEXT,
meta_json TEXT -- JSON for all freely definable meters
);
```

- `meta_json`saves all additional values as JSON →**flexible and expandable**

3️⃣ Document import / creation (Plugin 2)

```
// HTML: dynamic input fields for custom meters
<div id="customFields"></div>
<button onclick="addCustomField()">Add field</button>

<script>
const customFields = [];
function addCustomField() {
const name = prompt("Name of the field:");
  if(!name) return;
  customFields.push(name);
```

```
  const container = document.getElementById('customFields');
  container.innerHTML += `<label>${name}:</label><input type="text"
id="meta_${name}"><br>`;
}

async function saveDocument() {
  const metaData = {};
  for(let name of customFields){
    metaData[name] = document.getElementById(`meta_${name}`).value;
  }

  const doc = {
    title: document.getElementById('docTitle').value,
    type: document.getElementById('docType').value,
    customer: document.getElementById('customer').value,
    project: document.getElementById('project').value,
    doc_number: document.getElementById('autoNumber').checked
      ? (await (await fetch('http://localhost:5000/api/generate-number',
{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({
          type: document.getElementById('docType').value,
          customer: document.getElementById('customer').value,
          project: document.getElementById('project').value
      })})).json()).number
      : null,
    meta_json: JSON.stringify(metaData)
  };

  await fetch('/api/documents', {
    method:'POST',
    headers:{'Content-Type':'application/json'},
    body: JSON.stringify(doc)
  });
}
</script>
```

---

4 History & Filters

- Server-side: Parse`meta_json`and apply filters

```
@app.route('/api/documents/filter', methods=['POST'])
def filter_documents():
  criteria = request.get_json()  # z.B.
{"type":"project","region":"Nord","co2_min":10,"co2_max":50}

  conn = sqlite3.connect('database/documents.sqlite')
```

```
    c = conn.cursor()
    c.execute("SELECT id, title, doc_number, type, customer, project, created_at, meta_json
FROM documents")
    rows = c.fetchall()
    conn.close()

    results = []
    for r in rows:
        meta = json.loads(r[7])
        match = True
        for k,v in criteria.items():
            if k in ['co2_min','co2_max']:
                val = float(meta.get('co2',0))
                if k=='co2_min' and val < v: match=False
                if k=='co2_max' and val > v: match=False
            else:
                if meta.get(k) != v:
                    match=False
        if match:
            results.append({
                "id": r[0],
                "title": r[1],
                "doc_number": r[2],
                "type": r[3],
                "customer": r[4],
                "project": r[5],
                "created_at": r[6],
"finish line": finish line
            })
    return jsonify(results)
```

- Dashboard can**dynamic filters**generate, based on the defined "meter" fields

---

5 Export incl. own meters

- Meta-fields are considered**Columns in CSV/Excel**exported
- Each new customer meta is automatically added to the export table.

```
# CSV-Export
fields = ["title","doc_number","type","customer","project","created_at"]
all_meta_keys = set()
for r in rows:
    meta = json.loads(r[7])
    all_meta_keys.update(meta.keys())
```

```
fields.extend(sorted(all_meta_keys))

cw.writerow(fields)
for r in rows:
  meta = json.loads(r[7])
  row = [r[1],r[2],r[3],r[4],r[5],r[6]] + [meta.get(k,"") for k in sorted(all_meta_keys)]
  cw.writerow(row)
```

---

6 Advantages

1. Customer can**Define any additional meters**
2. **Filter & Search**use these meta fields
3. Export & History**automatically including meta fields**
4. Fully compatible with Plugin1 (number assignment)
5. Offline, Open Source, lokal

If you want, I can now**The complete final version including Plugin 1 + Plugin 2 + Management + History + Export + Reset + Custom Meters + Filters + Options**assemble completely**Completely ready for your developer to start.**.

Do you want me to do that right now?

Metadata

Perfect, then let's talk specifically about...**Metadata**For the system – that's the extension you referred to as "custom meters". I'll clearly summarize how they can be structured, stored, filtered, and exported.

---

1 Structure of the metadata

- Each document can have any metadata.
- Storage in**JSON**in the database (`meta_json`in table`documents`).
- Example:

```
{
"region": "North",
"department": "research",
 "co2_kg": 42.5,
"priority": "high",
"project phase": "planning"
```

}

- Advantage: **infinitely expandable**New fields do not need to be adjusted in the database.

2️⃣ Types of Metadata

Type	Description	Example
Text	Free text	„Nord“, „Marketing“
Number	Integers or decimal values	42, 3.5
Datum	ISO format for filters	2026-03-12
Selection	Dropdown with fixed options	"high", "medium", "low"
Boolesch	Yes No	true/false

- Optional: **Possible combinations**e.g., selection + number for priority/score

---

3️⃣ Use in UI

- When creating/importing a document:
  - Dynamic fields for metadata
  - The customer can define any fields.
  - Input fields adapt to type (text, number, date, selection)

Example of dynamic input:

```
<label for="meta_region">Region:</label>
<input type="text" id="meta_region">

<label for="meta_co2">CO₂ footprint (kg):</label>
<input type="number" id="meta_co2">
```

4️⃣ Storage

- Plugin2 stores all metadata in the column`meta_json`of the document table:

```
meta_json = json.dumps({
"region": "North",
"department": "research",
  "co2_kg": 42.5
})
```

- Tabel `documents`:

id	title	doc_number	type	customer	project	created_at	meta_json
1	Plan	PR-0001	project	ACME	XYZ	2026-03-12	{"region":"Nord","co2_kg":42.5}

---

5 Filtering by metadata

- Client: Filter form dynamically based on defined fields
- Server: JSON from`meta_json`Read, apply filter

Example:

```
criteria = {"region":"Nord","co2_min":10,"co2_max":50}
for r in rows:
  meta = json.loads(r["meta_json"])
  if meta.get("region") != criteria["region"]: continue
  if not (criteria["co2_min"] <= meta.get("co2_kg",0) <= criteria["co2_max"]): continue
  results.append(r)
```

---

6 Export

- When exporting to CSV/Excel: Metadata fields as additional columns
- Each new meta column is added automatically.

Example:

title	Number	Type	Could	project	Region	CO_2_kg	priority
Project plan	PR-2026-0001	project	ACME	XYZ	North	42.5	high

7️⃣ Advantages of the Metadata Approach

1. **Flexible**: any additional fields, without database adjustment
2. **Filterable**: any combinations by type, customer, project, date, meta fields
3. **Exportable**: Automatically export meta fields to CSV/Excel
4. **Compatible**With Plugin1: Numbers are assigned independently
5. **Offline & lokal**: all open source, locally storable

If you want, I can as**next step**the**complete final plugin structure**including:

- Plugin1 Number Generator
- Plugin2 Knowledge Base
- Management + Reset
- History + Filters
- Export + Metadata

finish building**Completely ready for the developer to start.**, with all metadata fields directly integrated.

Do you want me to do this now?

As a user, I use the document number generator. I press a button and then I automatically get a document number. Okay. And then I can enter some metadata. Okay, and then what? How does the document number get into my Word document or my Excel file? Can I retrieve it from Excel? Yes. Retrieve it from Word? Yes. Retrieve it from contracts? Yes. Everywhere. And then, well, where is it stored? I don't understand.

Okay, then we'll build it up logically, step by step, so that it**truly understandable**I will explain the entire process – from number generation to its use in Word, Excel, or contracts.

1️⃣ Generate a number

- You are **User**in the dashboard of the document number generator (Plugin1)
- You click on**"Generate number"**
- The system automatically generates a document number, e.g.:

RE-2026-000042

- Immediately after that you can**Enter some metadata**:
  - Could
  - project
  - Region

    - CO_2 footprint
    - or other user-defined fields

This metadata will**stored together with the number**.

2️⃣ Storage

Everything will**stored locally**, completely offline and open source:

- **database**: SQLite (`database/sequences.sqlite`)
- The table:
1. **Sequences**→ all number ranges, current status
2. **History**→ all generated numbers, including metadata
3. **Documents**(optional, Plugin2) → Documents, metadata, numbers
- Example data set in`history`:

id	type	number	customer	project	created_at	plugin
42	invoice	RE-2026-000042	ACME	Project X	2026-03-12 08:05	plugin1

- Additionally, the**Metadata as JSON**to be saved:

```
{
"region": "North",
"priority": "high",
 "co2_kg": 42.5
}
```

3️⃣ Use numbers in Word, Excel, or contracts

Option 1: Word / Excel

- The plugin creates a**local interface (API)**
- Word or Excel can be accessed via**Macros or Power Query**retrieve the document number

Example:

```
' Word VBA
Function GetNextNumber(docType As String) As String
```

```
  Dim http As Object
  Set http = CreateObject("MSXML2.XMLHTTP")
  http.Open "POST", "http://localhost:5000/api/generate-number", False
  http.setRequestHeader "Content-Type", "application/json"
  http.Send "{""type"":""" & docType & """}"
  GetNextNumber = http.responseText ' oder JSON parsen
End Function
```

- Then you can insert the number directly into your Word document:

Invoice: RE-2026-000042

- Excel works similarly via**PowerQuery oder VBA**

Option 2: Contracts / other documents

- Every contract is**a document**, which can access plugin 1 via API
- The number is automatically assigned when saving or creating.**assigned**
- Metadata is**simultaneously saved**so that you can filter or export later

---

4️⃣ Everything local, yet flexible

- **No cloud**→ All numbers and metadata remain on your computer
- **History & Filters**→ All numbers and metadata can be displayed at any time
- **Export**→ CSV or Excel, including metadata, if you want to create reports, for example.
- **Reusable**→ The same numbers can be used in Word, Excel, or contract documents via a local API.

5️⃣ Example workflow for you

1. You click on "Generate number"
2. The document number is automatically generated → e.g.`RE-2026-000042`
3. You add**Metadata**a → customer, project, CO_2, region
4. Number + metadata will be**stored locally** in SQLite
5. You open Word or Excel → retrieve the document number via macro or add-in
6. Document is saved → Number + metadata remain retrievable for history or export.

💡 **Important:**

- The**The number remains unambiguous.**and can**not be assigned twice**
- All metadata is**searchable and filterable**
- You can **Export at any time**for accounting or reporting

If you want, I can as**next step** a **graphical flowchart**building shows:

Button → Number → Metadata → Storage → Use in Word/Excel/Contracts → History/Export

This makes it immediately clear,**where everything is stored and how it all fits together**.

Do you want me to do that?

I'm in some tool, whether it's Excel or something else, like LibreOffice or Microsoft, and I just want to be able to click "Generate document number," insert metadata, and then it's automatically linked to the document. And the document itself is stored in the database, not just the number and metadata, but the document itself. It should be that simple.

Ah, now it's clear how you want it:**Everything from a single source**– The document number, metadata, and the actual document are**Stored directly in the database upon creation**, so that you**You don't need any extra steps**LibreOffice, Excel, Word – it doesn't matter, as long as a**local access to the database / API**is possible.

I will explain step by step what that would look like:

1️⃣ Click → Number + Metadata + Document

1. You open your document in Word / LibreOffice / Excel.
2. A **Plugin / Add-In / Button**The message "Generate receipt number" appears.
3. You click on it → a small form opens:

- **Number automatically generated** (z. B. `RE-2026-000043`)
- **Metadatenfelder**(Customer, project, CO_2, region, own fields)
- **The document is referenced directly.**

2️⃣ Storage in the database

- **Local SQLite database** (`database/documents.sqlite`)

- Tabel `documents`:

id	doc_number	type	customer	project	created_at	meta_json	document_blob
43	RE-2026-000043	invoice	ACME	Project X	2026-03-12 08:20	{"region":"Nord","co2_kg":42.5}	[File as blob]

- `document_blob`= the**saved document itself** (Word, Excel, PDF etc.)
- All metadata and numbers are stored simultaneously →**all in one data set**

3️⃣ Use in the document

- After clicking and entering metadata, the**Document number automatically inserted in the document**
- Document remains editable as normal.
- When you save,**The updated version in the database was overwritten.**

4️⃣ Access from Excel / Word / LibreOffice

- Plugin / Add-in calls local API:

```
POST http://localhost:5000/api/generate-number
Body: { "type": "invoice", "customer": "ACME", "project": "Projekt X" }
Response: { "number": "RE-2026-000043" }
```

- Then automatically**Metadatenformular**open → Input
- When saving → Document + metadata + number in database

5️⃣ Advantages

1. **All in one step:**Generate number, enter metadata, save document
2. **Document saved by itself**→ no separate file system required
3. **Offline & lokal**, completely open source
4. **Available later:**History, filters, export including metadata and document
5. **Platform independent:** Word, Excel, LibreOffice, PDF

💡 **Optional:**You could also**Drag & Drop** or **Batch-Import**Creating documents → each document automatically receives a number + metadata + storage in a database.

If you want, I can now**create a complete, finished plugin/tool concept**, the:

- Receipt number generated
- queries metadata
- Saves document directly in the local database
- Word / LibreOffice / Excel integrated
- History, filters, export including metadata + document

Everything so that**the developer can start it immediately**.

Do you want me to create it directly?

The document number is in the heading; I see it somewhere in the text. How do I find it again?

Ah, now I understand exactly what you're getting at:**You want the document number to be visible in the document (e.g., in the heading), but at the same time it should be uniquely linked to the stored record in the database.**, so you can find them again at any time, regardless of whether the document is Word, Excel or LibreOffice.

Here's how it works:

1️⃣ Document number visible in the document

- When generating the document number, it**automatically inserted at a defined location**, z. B.:

Word / LibreOffice / Excel

- In Word → Heading 1:`RE-2026-000043`
- In Excel → Zelle A1: `RE-2026-000043`
- You can see the number directly on the document →**clear and referenceable**

2️⃣ Linking to the database

- Each receipt number is**a unique key**in the database (`doc_number`)
- All metadata and the document itself are linked to this number.

Tabel `documents`:

doc_number	type	customer	project	meta_json	document_blob
RE-2026-000043	invoice	ACME	Project X	{"region":"Nord","co2_kg":42.5}	[Word file]

- That means: **No matter where you open the document, the number is your key.**, in order to find everything again.

3️⃣ Finding the number again in the text

- If you see the number in the document (e.g., heading), you can:
  1. Directly in the tool / plugin**search by number**
  2. Retrieve all metadata and the document from the database
  3. Reopen or export document
- Advantage: **The document does not need to be renamed.**Everything runs through the internal number

---

4️⃣ Retrieve/filter metadata + number

- In the history / database interface:
  - Search for `doc_number`→ you get**all metadata + saved document**
  - Filters can also be filtered by customer, project, CO_2, region, and own meters.

5️⃣ Practical idea for Word / Excel

- Number can**to be stored as a field in the document**(e.g. hidden tag or comment)
- Advantage: If you move or rename the file, for example, the connection remains.**via the internal number**
- Thus: **Document number visible + unique link in the database**

💡 **Note:**

- **Visible in the document = for you**
- **Stored in the database = for the system**
- The**The document number is the key.**, which combines both → you can find everything again at any time, whether it's Word, Excel, LibreOffice or a contract.

If you want, I can tell you**show a concrete example**, like a Word or Excel document**automatically inserts the number, queries metadata and saves it directly in the database**, including the option to later**to find the file using the number**.

Do you want me to create this right away?

Where can I find the document number? Is it in the heading or somewhere else in the text? How can I find it again?

Now it's clear what you're getting at: You want**Know exactly where the document number is located in the document and how to reliably find it again.**Whether it's Word, Excel, LibreOffice, or a contract, I'll explain it step by step.

1️⃣ Where the receipt number appears

- **Standard placement:**
  - In Word or LibreOffice →**Heading 1 or bold text at the beginning of the document**
  - In Excel → **Cell A1**or defined header
  - In contracts →**first page, e.g. in the header or contract number field**
- **Optional placement:**
  - You can specify that they also**as a comment or hidden field**It will be saved in the document if you don't want to see it directly in the text.

2️⃣ How to find the receipt number again

- **When you see them:**
  - Simply read the number → e.g.`RE-2026-000043`
  - You can use this number**all metadata and the saved document**retrieve from the database
- **If you don't see them directly:**
  - Number is also**stored internally within the document**e.g. in a**hidden field or comment**

    - Your plugin/tool can automatically read them → regardless of where the document is located.
- **In the database:**
    - Each receipt number is**unique key** (`doc_number`)
    - Search for this number → you will receive**Document + Metadata + History**

3️⃣ Practical example

Word:

Heading 1: RE-2026-000043
...
Text: Contract for Project X ...

- You can see the number directly
- If you move or open the file → the plugin can read the internal number.

Excel:

- Cell A1:`RE-2026-000043`
- The rest of the table contains metadata or content
- The number is automatically linked to the database.

Database:

doc_number	type	customer	project	meta_json	document_blob
RE-2026-000043	invoice	ACME	Project X	{"region":"Nord","co2_kg":42.5}	[File]

4️⃣ Summary

1. **Visible in the document:**Heading, header, or defined cell
2. **Stored internally:**Comment/field → available for the plugin
3. **Linked to database:**Number = Key → you'll find everything again
4. **Filter & Search:**By number, metadata, customer, project, etc.

If you want, I can...**Create a graphical scheme**, which shows exactly:
Where the document number is located in the document → how it ends up in the database → how you can find it again, so that everything is understandable at a glance.

Do you want me to do that?

So, the document number generator creates a number in the document I'm working with. It saves it in the heading, in the text, or hidden, depending on the requirements. It also saves the filename in the database. Okay, so I can go and say, "I'm now looking for the document number Dubbuck with that number." Or, yes, I could search for all of that from the dashboard.

Exactly, that's exactly how it works – that's the complete process, just as you want it:

1️⃣ Generate a number

- You are currently working in Word, Excel, LibreOffice, or a contract.
- You click on**"Generate receipt number"**.
- The generator creates the number, e.g.`RE-2026-000044`.
- Number will**automatically inserted into the document**:
  - **headline**→ clearly visible
  - **Im Text**→ if desired
  - **Hidden / Comment**→ if the document should not have a visible number
- At the same time you can**Enter metadata**: Customer, project, CO_2, department, own fields.

2️⃣ Storage in the database

- Everything will**stored locally in the SQLite database**:

doc_number	file_name	type	customer	project	meta_json	document_blob
RE-2026-000044	Projectplan_XYZ.docx	invoice	ACME	Project X	{"region":"Nord","co2_kg":42.5}	[File]

- `file_name`→ File name under which the document is saved
- `document_blob`→**The document itself** (Word, Excel, PDF …)
- `meta_json`→ all entered metadata

That's all.**a record in the database**, linked to the unique document number.

3️⃣ Find the document again

- You open the**Dashboard of the receipt number generator**.
- Search for:
  - **Document number** → z. B. `RE-2026-000044`
  - Or after**Metadata**→ Customer, project, region, own fields
  - Or after**File name**→ if you know how the document is saved
- The dashboard shows**all matching results**:
  - Number
  - Metadata
  - File name
  - Preview / Download document
- With one click you can open or export the document.

4️⃣ Advantage of this process

1. **Number always unique**→ Key to everything
2. **Number visible or hidden**→ as desired
3. **Document and metadata saved**→ everything in the database
4. **Search & Dashboard**→ find it quickly, whether it's Word, Excel, LibreOffice or a contract
5. **Everything local and offline**→ no cloud obligation

💡 **Note:**

- **Document number in the document**→ visible or hidden
- **Document number in the database**→ Key for document + metadata
- **Dashboard**→ Search and filter center

If you want, I can now**build a complete step-by-step visualization**, as the **Document number from document to dashboard and back**flows, including storage and retrieval.

Do you want me to do that?

As text

Okay, I'll summarize the entire process as**clear text flow**together – step by step, how it would work for the user:

Document number generator – complete process as text

1. **Open document**
   - You are in Word, Excel, LibreOffice, or a contract.
   - The document is either empty or already contains content.
2. **Generate receipt number**
   - Click on**"Generate receipt number"** im Plugin/Add-In.
   - The system automatically generates a**unique document number**, z. B.: `RE-2026-000044`.
   - The number will**inserted in the document**:
     - Visible in the heading or header
     - Optional im Text
     - Optionally as a hidden field or comment
3. **Enter metadata**
   - A small form will open automatically.
   - You can enter metadata:
     - Could
     - project
     - Region, department, CO_2 footprint or own defined fields
   - All metadata will be**linked to the document number**.
4. **Save document**
   - Everything happens automatically when saving:
     - Document number
     - Metadata
     - The document itself
   - Everything will**in the local SQLite database**filed.
   - Tabel `documents`stores:
     - `doc_number`→ unique document number
     - `file_name`→ Document filename
     - `meta_json`→ all metadata
     - `document_blob`→ the document itself
5. **Find document**
   - Open the**Dashboard des Plugins**.
   - Search for:
     - Document number (`RE-2026-000044`)
     - Or metadata (customer, project, region, custom fields)
     - Or file name
   - The dashboard displays all results:

        - Document number
        - Metadata
        - File name
        - Preview or retrieve the document
6. **Retrieve or edit document**
    - With one click you can open the document in Word, Excel, LibreOffice or another program.
    - Number and metadata remain**firmly linked**.

Summary of the key idea

- **Document number = Key**for document + metadata.
- **Visible in the document**→ for you while you work.
- **Stored internally**→ in the database for search, filtering, history and export.
- **Dashboard**→ Central control and retrieval of all documents.

If you want, I can do one next.**Create a complete sample text for the user**, which describes exactly how to**creates a document, generates a number, enters metadata, and retrieves it.**, like a kind**User manual**.

Do you want me to do that?

And

Perfect, here is the**Complete step-by-step instructions as text**, how a user works with the document number generator – including number, metadata, storage and retrieval:

User manual – Document number generator

1. Open document

- Open your document in Word, Excel, LibreOffice, or as a contract.
- The document can be empty or it can already contain content.

---

2. Generate receipt number

1. Click the button**"Generate receipt number"** im Plugin/Add-In.
2. The system automatically generates a unique number, e.g.:

RE-2026-000044

3. The number will**inserted in the document**:
    - Visible in the heading or header
    - Optional in the text or as a footnote
    - Optionally as a hidden field or comment if the number should not be directly visible.

---

3. Enter metadata

- A form opens automatically after the number is generated.
- Enter the metadata, e.g.:
    - Customer: ACME
    - Project: Project X
    - Region: North
    - CO_2 footprint: 42.5
    - Custom fields: Priority, Department, Project Phase
- This metadata will**linked to the document number**.

---

4. Save document

- If you are on**Save**click:
    - Document number
    - Metadata
    - The document itself

become **automatically in the local database**filed.

Example data record in the database:

Document number	File name	Type	Could	project	Metadata (JSON)	Document (Blob)

RE-2026-000044	Projectplan_XYZ.docx	The invoice	ACME	Project X	{"region":"Nord","co2_kg":42.5}	[File]

- The **File name**It will be saved so that the document can be easily found later.
- **Metadata**enable filters, search, and reports.
- **Document**It is saved as a file blob, regardless of whether it's Word, Excel or PDF.

---

5. Find the document again

1. Open the**Dashboard of the receipt number generator**.
2. Search using one of the following criteria:
   - **Document number**, z. B. `RE-2026-000044`
   - **Could**, **project**or other metadata
   - **File name**, if known
3. The dashboard displays all results:
   - Document number
   - Metadata
   - File name
   - Preview or direct retrieval of the document

---

6. Retrieve or further edit the document

- With one click you can open the document in Word, Excel, LibreOffice or another program.
- The document number remains**visible or hidden**, depending on the settings.
- All metadata and the number remain**linked to the document**.

---

7. Advantages of the system

- **All in one step:**Generate number → Enter metadata → Save document
- **Number = Key**→ unique link between document, metadata and database
- **Visible or hidden**→ flexible depending on requirements
- **Offline & lokal**→ No cloud required, everything is open source
- **Dashboard & Filter**→ faster access to history, metadata and documents
- **Export possible**→ CSV or Excel file including metadata and document number

---

If you want, I can do it next.**an example document with a real process**create, so**Text example: Word heading, metadata form, storage**, exactly as the user sees it.

Do you want me to do that?

And

Perfect, here comes a**A concrete example of what a document looks like in the system**– including receipt number, metadata, storage, and dashboard retrieval. All as**reiner Text**, as the user sees it:

Example document: Word / LibreOffice / Excel

Heading 1 / Cell A1:

Document number: RE-2026-000044

Document text / Contract content:

Contract for the implementation of Project X
Customer: ACME
Project Manager: Max Mustermann
Description: Creation of a sustainability report for Project X.

Metadata (entered via the plugin form):

- Customer: ACME
- Project: Project X
- Region: North
- CO_2 footprint: 42.5 kg
- Priority: high
- Project phase: Planning

Internally within the document (hidden field / comment):

doc_number: RE-2026-000044
meta_json: {"region":"Nord","co2_kg":42.5,"priority":"hoch","project_phase":"Planung"}

Database entry for this document

Document number	File name	Type	Could	project	Metadata (JSON)	Document (Blob)
RE-2026-000044	Projectplan_XYZ.docx	Contract	ACME	Project X	{"region":"Nord","co2_kg":42.5,"priority":"hoch"}	[Word file]

- **doc_number=** unique key
- **meta_json=** all metadata you have entered
- **document_blob=** saved file (Word, Excel, PDF, etc.)

Dashboard retrieval

- You open the dashboard → search field:
  - Document number:`RE-2026-000044`
  - Or filter by customer, project, region, CO_2, priority
- The results list shows:
  - Document number: RE-2026-000044
  - Customer: ACME
  - Project: Project X
  - Filename: Project plan_XYZ.docx
  - Preview / Download the document

Workflow for the user

1. Open document → Word, Excel or LibreOffice.
2. Click on**"Generate receipt number"**.
3. Fill out the metadata form.
4. The document number appears in the document (visible or hidden).
5. Save document → automatically to database including metadata + file.
6. Later, open the dashboard → search for the document number or metadata → retrieve the document.

If you want, I can as**next step** another one **Complete text simulation of a process with multiple documents**create – including**Document numbers, metadata, dashboard search**, so that the system can be seen as if it were in real-world operation.

Do you want me to do that?

And

Okay, here comes a**Complete text simulation of a process with multiple documents**– just as it would work in the real system. This shows document numbers, metadata, and how the dashboard manages the documents.

Simulation: Multiple documents in the document number generator

Document 1

Heading / Cell A1:

Document number: RE-2026-000044

Document content:

Contract for the implementation of Project X
Customer: ACME
Project Manager: Max Mustermann
Description: Creation of a sustainability report for Project X.

Metadata:

- Customer: ACME
- Project: Project X
- Region: North
- CO_2 footprint: 42.5 kg
- Priority: high
- Project phase: Planning

Database entry:

Document number	File name	Type	Could	project	Metadata (JSON)	Document (Blob)
RE-2026-000044	Projectplan_XYZ.docx	Contract	ACME	Project X	{"region":"Nord","co2_kg":42.5,"priority":"hoch"}	[Word file]

Document 2

Heading / Cell A1:

Document number: RE-2026-000045

Document content:

Invoice for consulting project Y
Customer: Beta GmbH
Project Manager: Anna Müller
Description: Consulting services for sustainable logistics processes.

Metadata:

- Customer: Beta GmbH
- Project: Project Y
- Region: South
- CO_2 footprint: 15.0 kg
- Priority: medium

Database entry:

Document number	File name	Type	Could	project	Metadata (JSON)	Document (Blob)
RE-2026-000045	Invoice_Beta_2026.xlsx	The invoice	Beta GmbH	Project Y	{"region":"South","co2_kg":15.0,"priority":"medium"}	[Excel file]

Document 3

Heading / Cell A1:

Document number: RE-2026-000046

Document content:

Energy efficiency project report
Customer: Gamma AG
Project Manager: Stefan König
Description: Analysis and optimization of energy consumption.

Metadata:

- Customer: Gamma AG
- Project: Project Z
- Region: Ost
- CO_2 footprint: 8.2 kg
- Priority: low

Database entry:

Document number	File name	Type	Could	project	Metadata (JSON)	Document (Blob)
RE-2026-000046	Project report_Gamma.docx	Message	Gamma AG	Project Z	{"region":"East","co2_kg":8.2,"priority":"low"}	[Word file]

Dashboard search

1. **Search by document number:**
   - Input:`RE-2026-000045`→ Hit: Document 2 (Beta GmbH, Project Y)
2. **Search for customer:**
   - Input:`ACME`→ Match: Document 1 (Project X)
3. **Filter by metadata:**
   - Region = "Cheese" → Hit: Document 3
   - Priority = "high" → Match: Document 1
4. **Retrieve file:**
   - Clicking on entry → opens the document in Word, Excel or LibreOffice
   - The document number remains visible or is available as a hidden field.
   - Metadata can be viewed or exported at any time.

Summary of the process

1. Open document → Click on “Generate receipt number”.
2. Number automatically generated → e.g.`RE-2026-000044`.
3. Fill out the metadata form → Customer, Project, Region, CO_2, etc.
4. Save → Receipt number, metadata and document**all in the database**.
5. Dashboard → Search by document number, metadata or file name → Retrieve document.
6. Edit document → Number + metadata remain linked.

If you want, as a next step I can create a text version that**shows the complete workflow of 1-3 documents as a "user experience simulation".**, in other words, exactly how the user clicks, types and retrieves documents – essentially a “playthrough” of the system.

Do you want me to do that?

And

Perfect, here is the**Complete user experience simulation**as text – a “playthrough” of the document number generator from beginning to end for several documents:

User Experience Simulation – Document Number Generator

Step 1: Create Document 1

1. **Open:**Word document “Project Plan_XYZ.docx”.
2. **Generate receipt number:**Click on**"Generate receipt number"**→ Number`RE-2026-000044`appears in the headline.
3. **Fill in the metadata:**
   - Customer: ACME
   - Project: Project X
   - Region: North
   - CO_2 footprint: 42.5 kg

    - Priority: high
    - Project phase: Planning
4. **Save:**Document + metadata + receipt number automatically saved in the local database.

Database entry:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000044	Projectplan_XYZ.docx	Contract	ACME	Project X	{"region":"Nord","co2_kg":42.5,"priority":"hoch"}

Step 2: Create document 2

1. **Open:**Excel file “Invoice_Beta_2026.xlsx”.
2. **Generate receipt number:**Click → Number`RE-2026-000045`in cell A1.
3. **Metadata:**
    - Customer: Beta GmbH
    - Project: Project Y
    - Region: South
    - CO_2 footprint: 15.0 kg
    - Priority: medium
4. **Save:**Everything is stored in the database.

Database entry:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000045	Invoice_Beta_2026.xlsx	The invoice	Beta GmbH	Project Y	{"region":"South","co2_kg":15.0,"priority":"medium"}

Step 3: Create document 3

1. **Open:**Word document “Project report_Gamma.docx”.
2. **Generate receipt number:**Click → Number`RE-2026-000046`in the headline.
3. **Metadata:**
    - Customer: Gamma AG

    - Project: Project Z
    - Region: Ost
    - CO_2 footprint: 8.2 kg
    - Priority: low
4. **Save:**Everything is stored in the database.

Database entry:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000046	Project report_Gamma.docx	Message	Gamma AG	Project Z	{"region":"East","co2_kg":8.2,"priority":"low"}

Step 4: Dashboard search and retrieval

1. **Search by document number:** `RE-2026-000045`→ Hit: Document 2.
2. **Search for customer:** `ACME`→ Hit: Document 1.
3. **Filter by region:** `Ost`→ Hit: Document 3.
4. **Retrieve file:**Click on the match → document opens in Word, Excel or LibreOffice.
5. **Edit:**The document number remains visible or is available as a hidden field; metadata remains linked.

Step 5: Benefits for the user

- **All in one step:**Generate number → Fill in metadata → Save.
- **Document + number + metadata = one data record**in the database.
- **Visible or hidden:**The number can be displayed flexibly.
- **Dashboard & Filter:**Fast access to all documents.
- **Offline & lokal:**No cloud required, everything is open source.
- **Export:**CSV or Excel files with document numbers, metadata and file names are possible.

💡 **This is what the system looks like to the user:**

- Click → Number generated
- Form → Enter metadata
- Save → everything automatically to DB

- Dashboard → search, filter, retrieve document

If you want, as a **next step I can** create a step-by-step text simulation with**Document numbers, metadata, dashboard interactions, and file retrievals in real-time order**create – essentially a complete “play-by-play” simulation of how the user uses the system.

Do you want me to do that?

And

Okay, here comes the**komplette Play-by-Play-Simulation**as text – as a user actually experiences the system, including clicks, receipt numbers, metadata and dashboard access:

Receipt Number Generator – Play-by-Play Simulation

User started System

1. User opens Word document “Project Plan_XYZ.docx”.
2. Plugin/Add-In is active → Button**"Generate receipt number"** visible.

Document 1 – RE-2026-000044

1. Click on**"Generate receipt number"**→ Number`RE-2026-000044` becomes **inserted in heading**.
2. Form opens: User enters metadata:
    - Customer: ACME
    - Project: Project X
    - Region: North
    - CO_2 footprint: 42.5 kg
    - Priority: high
    - Project phase: Planning
3. Click on**"Save"**→ everything will**in the local DB**saved.

DB entry:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000044	Projectplan_XYZ.docx	Contract	ACME	Project X	{"region":"Nord","co2_kg":42.5,"priority":"hoch"}

Document 2 – RE-2026-000045

1. User opens Excel file “Rechnung_Beta_2026.xlsx”.
2. Click on **"Generate receipt number"** → Number `RE-2026-000045` in cell A1.
3. Formulate:
   - Customer: Beta GmbH
   - Project: Project Y
   - Region: South
   - CO_2 footprint: 15.0 kg
   - Priority: medium
4. Click on **"Save"** → The database stores everything.

DB entry:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000045	Invoice_Beta_2026.xlsx	The invoice	Beta GmbH	Project Y	{"region":"South","co2_kg":15.0,"priority":"medium"}

Document 3 – RE-2026-000046

1. User opens Word document “Project Report_Gamma.docx”.
2. Click on **"Generate receipt number"** → Number `RE-2026-000046` in the heading.
3. Formulate:
   - Customer: Gamma AG
   - Project: Project Z
   - Region: Ost
   - CO_2 footprint: 8.2 kg
   - Priority: low
4. Click on **"Save"** → The database stores everything.

DB entry:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000046	Project report_Gamma.docx	Message	Gamma AG	Project Z	{"region":"East","co2_kg":8.2,"priority":"low"}

Dashboard search – live interaction

1. User opens dashboard.
2. **Search by document number:** `RE-2026-000045`
   - Hit: Document 2 → Excel “Invoice_Beta_2026.xlsx”
   - Click → Document opens → Receipt number visible, metadata retrievable.
3. **Search for customer:** `ACME`
   - Match: Document 1 → Word “Project Plan_XYZ.docx”
4. **Filter by region:** `Ost`
   - Match: Document 3 → Word “Project Report_Gamma.docx”
5. **Retrieve file:**Click → Document opens in the original application (Word, Excel).

User continues working

- Document edited → document number remains visible or is present as a field.
- Saves again → Database automatically updates document and metadata.
- The dashboard displays an instantly updated history.

Summary of User Experience

1. Open document → Click “Generate receipt number”.
2. Number appears → Fill out metadata form.
3. Save → everything automatically in DB (number, metadata, document).
4. Dashboard → Search by number, customer, project, region, priority.
5. Match → Open or export document → Document number & metadata remain linked.
6. Everything local → Offline, secure, Open Source.

If you want, as a **next step I can create a version that shows how**Different document types can be managed simultaneously.**– including Word, Excel, and PDF in a dashboard with filtering options – completely as a text simulation including search and retrieval logic.

Do you want me to do that?

And

Perfect, here comes the**Complete text simulation for multiple document types simultaneously**Word, Excel and PDF – including dashboard, search, filter and retrieval.

Multi-Document Simulation – Document Number Generator

Document 1 – Word

Filename: Projectplan_XYZ.docx
Document number: RE-2026-000044

Headline:

Document number: RE-2026-000044

Metadata:

- Type: Contract
- Customer: ACME
- Project: Project X
- Region: North
- CO_2 footprint: 42.5 kg
- Priority: high

Dashboard database:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000044	Projectplan_XYZ.docx	Contract	ACME	Project X	{"region":"Nord","co2_kg":42.5,"priority":"hoch"}

Dokument 2 – Excel

Filename: Invoice_Beta_2026.xlsx
Document number: RE-2026-000045

Cell A1:

RE-2026-000045

Metadata:

- Type: Invoice
- Customer: Beta GmbH
- Project: Project Y
- Region: South
- CO_2 footprint: 15.0 kg
- Priority: medium

Dashboard database:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000045	Invoice_Beta_2026.xlsx	The invoice	Beta GmbH	Project Y	{"region":"South","co2_kg":15.0,"priority":"medium"}

Document 3 – PDF

Filename: Energy Report_Gamma.pdf
Document number: RE-2026-000046

PDF text (header or hidden field):

RE-2026-000046

Metadata:

- Type: Report
- Customer: Gamma AG

- Project: Project Z
- Region: Ost
- CO_2 footprint: 8.2 kg
- Priority: low

Dashboard database:

Document number	file	Type	Could	project	Metadata (JSON)
RE-2026-000046	Energy Report_Gamma.pdf	Message	Gamma AG	Project Z	{"region":"East","co2_kg":8.2,"priority":"low"}

Dashboard – Multi-Document Search

1️⃣ **Search by receipt number:**

- Input: `RE-2026-000045` → Hit: Excel “Invoice_Beta_2026.xlsx”
- Click → File opens, document number visible, metadata retrievable

2️⃣ **Search for customer:**

- Input: `ACME` → Match: Word “Project Plan_XYZ.docx”

3️⃣ **Filter by document type:**

- Type = Report → Match: PDF “Energy Report_Gamma.pdf”

4️⃣ **Filter by region:**

- Region = East → Hit: PDF “Energy Report_Gamma.pdf”

5️⃣ **Filter by priority:**

- Priority = high → Match: Word “Project Plan_XYZ.docx”

Dashboard functions

- Preview / Retrieve the file
- Edit → Number remains visible or hidden

- Export of metadata + document numbers to CSV or Excel
- Filter by any metadata fields → Region, CO_2, Project Phase, Priority
- Sort by document number, customer, project, or date

User-Workflow – Multi-Dokument

1. Open document (Word / Excel / PDF).
2. Click on**"Generate receipt number"**→ Number appears.
3. Fill in metadata → Type, Customer, Project, CO_2, Priority, Phase.
4. Save → everything is stored in the local database (number, document, metadata).
5. Open dashboard → Search by number, customer, type, project or any metadata.
6. Select match → Retrieve or edit document → Number + metadata remain linked.
7. Export possible → for reports or internal archiving.

💡 **Key point for the user:**

- **Document number = Key**for each document.
- **Dashboard = Control Center**for all document types.
- **Metadata + document + number = everything linked**→ Faster access, filtering, history, export.

If you want, as a next step I can...**Complete text simulation with optional extensions**create, i.e., how**Custom metadata fields, custom filters, project-specific document numbers** and **Integration into Plugin 2 (Knowledge Base)**They must run simultaneously so that everything works together.

Do you want me to do that?

And

Okay, here comes the**Complete text simulation with all optional extensions**, including individual metadata fields, project-specific document numbers and integration with the knowledge base module (Plugin2).

Enhanced multi-document simulation – document number generator + knowledge database

---

Document 1 – Word, Project X

Filename: Projectplan_XYZ.docx
Document number: PX-2026-000044

Heading / Cell A1:

Document number: PX-2026-000044

Metadata (optionally expandable):

- Type: Contract
- Customer: ACME
- Project: Project X
- Region: North
- CO_2 footprint: 42.5 kg
- Priority: high
- Project phase: Planning
- Own fields:
    - Department: R&D
    - Approval: Yes
    - Responsible party: Max Mustermann

Integration Plugin 2 (Knowledge Base):

- The document is automatically created as an entry in the knowledge base.
- Metadata fields are available as filters for research and project linking.
- Full-text search across document content is possible.

Database entry:

Document number	file	Type	Could	project	Target (JSON)	Knowledge database

PX-2026-000044	Projectplan_XYZ.docx	Contract	ACME	Project X	{"region":"North","co2_kg":42.5,"priorityhighdepartmentR&DreleaseYes"}	active

---

Document 2 – Excel, Project Y

Filename: Invoice_Beta_2026.xlsx
Document number: PY-2026-000045

Cell A1:

PY-2026-000045

Metadata:

- Type: Invoice
- Customer: Beta GmbH
- Project: Project Y
- Region: South
- CO_2 footprint: 15.0 kg
- Priority: medium
- Own fields:
  - Department: Controlling
  - Payment terms: 14 days
  - Responsible party: Anna Müller

Integration Plugin 2:

- Excel spreadsheet as knowledge entry, metadata filter available.
- Document numbers can be linked for billing reports.

Database entry:

Document number	file	Type	Could	project	Target (JSON)	Knowledge database
PY-2026-	Invoice_Beta_2026.xlsx	The inv	Beta	Proje	{"region":"South","co2_kg":15.0,"prioritymediumdepartmentControllingpayment_term14 days"}	active

000045	oice	GmbH	ctY

Document 3 – PDF, Project Z

Filename: Energy Report_Gamma.pdf
Document number: PZ-2026-000046

PDF header / hidden field:

PZ-2026-000046

Metadata:

- Type: Report
- Customer: Gamma AG
- Project: Project Z
- Region: Ost
- CO_2 footprint: 8.2 kg
- Priority: low
- Own fields:
    - Department: Energy
    - Review status: In progress
    - Responsible party: Stefan König

Integration Plugin 2:

- The PDF will be created as an entry in the knowledge base.
- Metadata enables project research and CO_2 analysis.
- Full-text search across document content is enabled.

Database entry:

Document number	file	Type	Could	project	Target (JSON)	Knowledge database
PZ-2026-000046	Energy Report_Gamma.pdf	Messa	Gamm	Projec	{"region":"East","co2_kg":8.2,"prioritylowdepartmentEnergyreview_statusIn Progress"}	active

g a t
e A Z
G

Dashboard – Multi-filter & knowledge base

Search examples

1. **Search for document number:**
   - Input:`PY-2026-000045`→ Results: Excel, document + metadata + knowledge base link
2. **Filter customer:**
   - `ACME`→ Match: Word, Project Plan_XYZ.docx
3. **Filter region:**
   - East → Hit: PDF, Energy Report_Gamma.pdf
4. **Filter priority:**
   - Up → Match: Word, Project plan_XYZ.docx
5. **Custom metadata can be filtered:**
   - Department = Controlling → Match: Excel, Project Y
   - Review status = "In progress" → Match: PDF, Project Z
6. **Filter document type:**
   - Report → PDF
   - Invoice → Excel
   - Contract → Word

Retrieval & Processing

- Clicking on a match → opens the file in the original application.
- Receipt number remains visible or hidden.
- Metadata remains linked.
- Changes → automatically synchronized with knowledge base and database.

Workflow summarized

1. Open document → Click “Generate receipt number”.
2. Fill in metadata → Standard + optional custom fields.
3. Save → Document number + Document + Metadata + Knowledge base entry.
4. Dashboard → Search & Filter → Access all documents simultaneously.
5. Full-text search across content, CO_2, project number, customer, priority, or custom fields.

6. Retrieve document → edit → save → everything synchronized.

💡 **Note:**

- **Document number + metadata + document = unique key**
- **Dashboard + Plugin 2 Integration**→ Quick search, filters, full text & project research
- **Multi-Document Type**→ Manage Word, Excel, and PDF files simultaneously
- **Optional metadata**→ Individual fields for each project, department, or workflow

---

If you want, as a next step I can...**Complete visualization of all steps as a text diagram**create – from document number generation to metadata and dashboard search – so that the complete flow**Clear overview**is shown.

Do you want me to do that?

And

Perfect, here is a**Text diagram visualization of the complete workflow**– from the document number generator to metadata and dashboard search – everything clearly displayed at a glance.

Flowchart – Document Number Generator + Knowledge Base

```
┌──────────────────────────────┐
│ Open document │
│ (Word / Excel / PDF)      │
└──────────────┬───────────────┘
               │
               ▼
┌──────────────────────────────┐
│ Generate receipt number │
│ (Automatic) │
│ Example: RE-2026-000044 │
└──────────────┬───────────────┘
               │
               ▼
┌──────────────────────────────┐
```

```
│ Metadatenformular          │
│ - Type (contract/invoice) │
│ - Customer / Project │
│ - Region / CO₂ / Priority │
│ - Optional fields │
│ e.g. department, review │
└──────────────────────────┘
          │
          ▼
┌──────────────────────────┐
│ Save │
│ - Document │
│ - Document number │
│ - Metadata │
│ - Knowledge base entry │
└──────────────────────────┘
          │
          ▼
┌──────────────────────────┐
│ Database & Knowledge Base │
│ - All documents │
│ - Linked metadata │
│ - Document number as key │
└──────────────────────────┘
          │
          ▼
┌──────────────────────────┐
│ Dashboard                │
│ - Search by document number │
│ - Filter by customer/project │
│ - Filter by type / region │
│ - Filter by metadata │
│ - Full-text search across content │
└──────────────────────────┘
          │
          ▼
┌──────────────────────────┐
│ Select match │
│ - Open document │
│ - Document number visible/hidden │
│ - Show metadata │
│ - Edit → Save │
└──────────────────────────┘
          │
          ▼
┌──────────────────────────┐
│ Export / Reports         │
│ - CSV / Excel what │
```

```
│ Document number + metadata │
└──────────────────────────────┘
```

Flow explanation

- **Each document**It receives a unique document number.
- **Metadata**are flexibly supplemented (standard + custom fields).
- **Save**→ Everything local + knowledge database integrated.
- **Dashboard**→ Central search and filter tool for all documents and metadata.
- **Edit & Retrieve**→ The document number remains the key; everything is synchronized.
- **Export**→ CSV / Excel → Reports or internal archiving possible.

If you want, as a next step I can...**Complete simulation with multiple users, simultaneous documents, and dynamic filters.**create – essentially a**Multi-User-Play-by-Play**, to show the full plugin combination.

Do you want me to do that?

And what about plug-in 1, the knowledge base we developed yesterday, which integrates three LLMs: a code agent, a summary agent, and a search agent? Where is that?

Okay, I'll summarize that clearly now and show you.**Plugin 1 (Document Number Generator) + Plugin 2 (Knowledge Database with LLMs)**in a consistent process – so that you can see how everything works together.

Overall system: Plugin1 + Plugin2 integration

1️⃣ Plugin1 – Receipt number generator

- Task: Automatically generate document numbers and assign them to each document (Word, Excel, PDF).
- Features:
  - Insert document number in heading, cell, or hidden location.
  - Capture metadata (standard + optional fields).
  - Store document + metadata + number in the database.
- Multi-Document compatible, local, offline, Open Source.

2️⃣ Plugin 2 – Knowledge Base with LLMs

- Task: Integrate all documents from Plugin1 into an intelligent knowledge database.
- Deployed LLMs / Agents:
  - **Code-Agent:**Understands content in Excel/Word/PDF, analyzes metadata, creates structured data.
  - **Summary agent:**Generates short summaries of documents, project status, and reports.
  - **Such-Agent:**Full-text search and semantic search across document content and metadata.
- Features:
  - Automatically import documents from Plugin1.
  - Metadata and document numbers are available as filters and keys.
  - Full-text search + semantic search → also by project topic, CO_2 value, priority, department.
  - Summaries available immediately → quick overview for dashboards or reports.
  - Running multiple LLMs in parallel → Code, summary, search.

3️⃣ Interaction of Plugin1 + Plugin2

```
┌──────────────────┐
│ Open document │
│ (Word/Excel/PDF) │
└──────────┬───────┘
    │
    ▼
┌──────────────────────────┐
│ Plugin1: Document number │
│ generate + metadata │
└──────────┬───────────────┘
    │
    ▼
┌──────────────────────────────────┐
│ Storage in DB (number, │
│ Metadata, document) │
└──────────┬───────────────────────┘
    │
    ▼
┌──────────────────────────────────┐
│ Plugin 2: Knowledge Base │
│ - Code Agent: Structuring │
│ - Summary Agent: │
```

```
│ Generate short information │
│ - Search agent: Full text + │
│ semantic search │
└──────────┬──────────────────────────┘
           │
           ▼
┌──────────────────────────────────────┐
│ Dashboard / Search │
│ - Search by document number │
│ - Filter by metadata │
│ - Full-text & semantic │
│ Search by content │
│ - Summaries available│
└──────────┬──────────────────────────┘
           │
           ▼
┌──────────────────────────────────────┐
│ Select result → open │
│ → Document number visible/hidden │
│ → Metadata & Summary │
└──────────────────────────────────────┘
```

Summary

1. **Plugin 1**: Document number + metadata + storage.
2. **Plugin 2**Import into knowledge database, analyze three LLMs, summarize, enable search.
3. **Dashboard**: Everything is accessible → Search, filter, full text, semantic query, summary.
4. **Multi-Dokument + Multi-LLM**Word, Excel, PDF simultaneously, all linked.
5. **User Experience**: Click → Number → Metadata → Save → Dashboard → Search/Retrieve → Edit → Summary + LLM Support.

If you want, I can as**The next step is a text simulation called "Live-Flow".**create the**shows how a user generates a document, enters metadata, runs Plugin2 LLMs and directly receives a summary + semantic search.**– practically a complete live example.

Do you want me to do that?

As my business grows and capabilities increase, it makes little sense to store documents and receipts in a small SQLite database. Wouldn't it be more sensible to install a proper database right away, allowing for future plugins and expansion?

Exactly, you've hit the nail on the head. For a small setup, SQLite or similar can be practical – lightweight, portable, no server required – but as soon as**more documents, larger projects, additional plugins, or multiple users**When SQLite comes into play, it very quickly reaches its limits.

Here is the**A classic approach to future-proof planning.**:

1️⃣ Why a “proper” database makes sense

- **Scalability:**Thousands to millions of documents, metadata, version numbers and document numbers → SQLite is not optimized for this.
- **Multi-user operation:**When multiple users simultaneously create documents, edit metadata, or access dashboard functions → robust transactions and locks are needed.
- **Expandability:**New plugins or LLMs may require additional tables, indexes, or relational logic.
- **Performance:**Full-text search, metadata filters, CO_2 calculations or dashboards → professional databases are significantly faster.
- **Backup & Security:**Easier with a DBMS, especially for sensitive documents.

2️⃣ Recommended options (Open Source & future-proof)

database	Advantage	When to use
PostgreSQL	Extremely stable, supports JSON, full text, relational tables, LLM connections	Medium to large systems, multiple plugins, multi-user
MariaDB/MySQL	Well-known, high-performing, many tools, relational queries	Documents + receipts, standard filters, reporting
MongoDB	Document-oriented, flexible for dynamic metadata	When many different metadata/JSON data sets are needed

ClickHouse	Analytical, very fast for large amounts of documents + metadata	If there is a lot of reporting, CO_2 analysis, dashboards

Tip: For your setup I would**PostgreSQL** recommend:

- JSON/JSONB for flexible metadata
- Full-text search for document content
- Multi-User & Transaction
- Expandable for future plugins + LLM integration

3️⃣ Architectural idea

1. **Plugin1 (Receipt number generator)**writes to PostgreSQL → tables:
   - `documents`→ Document number, file name, path, type
   - `metadata`→ Customer, Project, CO_2, Priority, Optional Fields
2. **Plugin 2 (Knowledge Base)**reads & writes → Tables:
   - `summaries`→ LLM summaries
   - `search_index`→ Full text / semantic search
3. **Dashboard / Frontend**→ Read-only access + filter → Performance-safe

---

💡 **Note:**
SQLite is great for prototypes and small, single-user installations.
PostgreSQL = professional, future-proof, scalable, multi-plugin capable, full control over metadata + LLM integration.

If you want, I can do one.**Specific database structure and table layout for your system**Design – i.e., Plugin1 + Plugin2, multi-document, metadata, LLMs – ready for PostgreSQL, including fields, indexes and query suggestions.

Do you want me to do that?

So, it should of course also bring all the documents I have into the intelligence knowledge database, not just from Plugin 1. I've already created a folder structure using Plugin 2, and I can expand it. I can upload my files to a blog or upload them individually to the individual folders. And then they're there. They don't have document numbers yet, but I can manually assign metadata to the entire folder. I'll call the folder "Projects," and I want to assign metadata to each file in it. They all get the metadata "Projects," and you might also know them as Project 1, Project 2, and so on. And it should automatically retrieve the date, the

filename, and so on. So that we have as much saved as possible automatically. Then the LLMs should be able to search the entire knowledge database or a folder for any text, any code, or group files together, or group multiple files together. And then, of course, I want to be able to enter metadata for this new file and save it there. And I want to have a separate category for my new results. The other category is the basic data from the chat.

Perfect, I see exactly what you want: a**intelligent, flexible system**, the **Existing documents automatically imported**, it structures, assigns metadata, and then via**LLMs in the knowledge base**This makes it usable – all seamlessly alongside Plugin1. Let me summarize this clearly and show how it should work:

Expanded knowledge base – full integration of existing documents

---

1️⃣ Automatic document import

- **Folder structure:**
  - Plugin2 creates base folders → e.g.`Projects`, `Contracts`, `Reports`, `Blog-Uploads`.
  - You can expand folders and add new subfolders.
- **Import:**
  - Placing files in a folder → automatically added to the knowledge base.
  - Supports Word, Excel, PDF, text, and code files.
  - Automatically reads metadata: filename, creation date, modification date, and author (if applicable).
- **Manual metadata assignment:**
  - You select a folder → “select all files in this folder” → add metadata (e.g., project name, category, priority).
  - Each document receives the folder's metadata + optional individual metadata.

2️⃣ LLM-Integration

- **Features:**
  - **Search across all documents or specific folders**→ Full text & semantic search.
  - **Summary of individual documents**→ Brief overview consisting of text, tables, and code.

    - **Summary of multiple files**→ e.g., combining several project reports.
    - **Code-Agent**→ Analysis of code files, extracts functions, comments, and versions.
- **Workflow:**
    - You select document(s) or folders → LLMs generate summaries → automatically save as new entries in the knowledge base.

3️⃣ Metadata management

- **Automatically saved data:**
    - File name, creation date, modification date, folder assignment, possibly author/version.
- **Can be added manually:**
    - Project, client, priority, category, additional keywords.
- **New category for results:**
    - Z. B. `New results`→ all newly aggregated content from LLMs or newly uploaded files.
    - Basic data (from chat / previous plugins) remains unchanged.

4️⃣ Usage & Dashboard

- **Search:**
    - Search by metadata, text, code, or project name.
    - Filter by folder, category, project, or date.
- **Retrieval:**
    - Open documents individually or combine multiple files.
    - LLMs can automatically extract core information.
- **Integration with Plugin1:**
    - Document numbers only for new or specific documents → old files without a document number receive metadata + date + folder assignment.

5️⃣ Architectural idea

- **DB:**PostgreSQL (scalable)
    - The table:`documents`, `metadata`, `folders`, `summaries`, `llm_results`
- **Folder logic:**Each file → linked to folder ID
- **LLM Agents:**Search / Summary / Code Agent
- **Dashboard:**Filters, search, retrieval, metadata management

---

💡 **User flow example:**

1. You place existing files into folders`Projects`.
2. Plugin2 automatically imports everything.
3. Metadata is partially automatically imported from the file and folder.
4. You manually add further metadata → saved.
5. Analyze LLMs → Summaries generated → Category`New results`.
6. Dashboard: Search, filters, retrieval, summaries → available at any time.

If you want, as a next step I can**create a detailed text simulation**, the**shows how multiple existing files are added to the knowledge base, metadata is assigned, LLMs summaries are generated, and everything is accessible in the dashboard.**– Step by step, including file names, folders, metadata and LLM outputs.

Do you want me to do that?

It supports all types of files for import, including images and videos, everything!

Perfect, then we will consistently expand the concept to**all file types**Text, tables, PDFs, code, images, videos, audio – and we integrate everything into the knowledge base. I'll summarize it clearly so you can see the complete workflow:

Universal knowledge base – Multi-file types

1️⃣ File import – Everything allowed

- Supported types:
  - **Text & Documents:** Word, PDF, TXT, Markdown
  - **Tables & Excel:** XLSX, CSV
  - **Code files:** Python, Java, JS, etc.
  - **Media:** Bilder (PNG, JPG, TIFF), Videos (MP4, AVI), Audio (MP3, WAV)
  - **Miscellaneous:** ZIP, JSON, XML
- **Importlogic:**
  - Place files in folders → Plugin2 scans and imports.
  - Automatically retrieved information:

        - File name
        - Date of creation / Date of modification
        - Folder assignment
        - File type
        - Optional: Author / Which
    - Manually assignable metadata: Project, Customer, Priority, Category, Keywords

---

2️⃣ Folder and category logic

- Flexible folder structure → e.g.:`Projects`, `Contracts`, `Reports`, `Blog-Uploads`, `media`.
- New files → automatically assigned folder ID.
- Metadata can be set for entire folders at once.
- Category “New Results” for LLM-generated content → separate view, all others remain available as basic data.

3️⃣ LLM Integration – Multi-Agent

- **Code-Agent:**Analyzes code files → functions, versions, comments
- **Summary agent:**
    1. Texts, tables, PDFs → short summaries
    2. Images/Videos → Analysis of descriptions, metadata, possibly OCR for text in images
- **Such-Agent:**
    1. Full-text search across all documents, images, audio transcripts, and video metadata.
    2. Semantic search by content, project, client, keywords
- **Workflow:**
    1. Select file(s) or scan folder
    2. LLMs generate summaries / analyses
    3. Save results → Category “New results”
    4. Dashboard displays all files + summaries + metadata

4️⃣ Dashboard – Multi-Type & Multi-Function

- **Filter:**
    - File type (Text, PDF, Excel, Image, Video, Audio)
    - Folder / Category / Project / Client / Priority / Tags

    - Date, creation/modification date
- **Search:**
    - Full text and semantic analysis across all file types
    - Image and video recognition / OCR / audio transcription possible
- **Retrieval & processing:**
    - Open directly in the original program, edit metadata, check summaries
    - Automatic synchronization in the knowledge base
- **Export:**CSV/Excel of all metadata + summaries → for reports, dashboards or analyses

---

5️⃣ Architecture & Database

- **PostgreSQL**(recommended, scalable, multi-user)
    - The table:
        - `documents`→ Filename, type, path, folder ID
        - `metadata`→ Project, Customer, Priority, Keywords, Optional Fields
        - `folders`→ Structure & Category
        - `llm_results`→ Summaries, analyses
    - Media → possibly store externally + path in database (BLOBs only recommended for smaller files)
- **Plugin 1 Integration:**Document numbers for new documents → retain existing files Folder assignment + metadata

---

💡 **Note:**

- **Everything can be imported:**Text, code, tables, images, videos, audio.
- **LLMs analyze everything:**Summary, search, semantic analysis.
- **Metadata + Dashboard + Categories:**Everything is filterable, retrievable, and expandable.
- **Future-proof:**PostgreSQL, multi-user, multi-plugin, scalable for large datasets.

If you want, as a next step I can**create a complete text simulation**, the**shows how a user can place existing files of all types into the knowledge base, assign metadata, run LLMs, and have everything accessible in the dashboard.**– including images, videos and code files, step by step.

Do you want me to do that?

And

Perfect, then I'll create the**Complete text simulation – multi-file types, import, metadata, LLMs, dashboard**Step by step. I'm structuring it so that you could essentially "play" it back.

Simulation: Universal knowledge database – Multi-file types

Step 1: Folders & Files

Folder structure Plugin2:

/Projects
/contracts
/Reports
/Blog-Uploads
/Media

Files in the Projects folder:

- Projektplan_XYZ.docx (Word)
- Rechnung_Beta_2026.xlsx (Excel)
- Energy Report_Gamma.pdf (PDF)
- Diagram_ProjectZ.png (Build)
- Presentation_Alpha.mp4 (Video)
- Code_Skript1.py (Code)
- Audio_Interview1.mp3 (Audio)

Step 2: Automatic Import

- Plugin2 scans folders`/Projects`.
- Files automatically imported → DB table`documents`+ Folder ID.
- Automatically saved data:
    - Filename, type, path, creation date, modification date
- Metadata automatically retrieved from folder:`Category=Projects`

Example database entry:

file	Type	Folder	Date of creation	Date of change	Category
Projectplan_XYZ.docx	Word	Projects	2026-03-12	2026-03-12	Projects
Diagramm_ProjektZ.png	Picture	Projects	2026-03-11	2026-03-11	Projects

---

Step 3: Manual Metadata Mapping

- Select all files in the folder → Assign metadata:
  - Project: Project X
  - Customer: ACME
  - Priority: high
  - Optional: Department: R&D, Responsible person: Max Mustermann
- Result: each file now has basic metadata + folder metadata

Example JSON metadata:

```
{
"project": "Project X",
"customer": "ACME",
"priority": "high",
"department": "R&D",
"responsible person": "Max Mustermann"
}
```

Step 4: LLM processing

LLM agents are running:

1. **Code-Agent**→ Code_Script1.py analyzed → Functions, variables, comments → saved in`llm_results`.
2. **Summary Agent**→ Project plan_XYZ.docx + Invoice_Beta_2026.xlsx + Energy report_Gamma.pdf → short summaries generated → Category "New results".
3. **Such-Agent**→ Full text and semantic indexing of all files (text, tables, PDFs, code, audio transcripts, images with OCR, video metadata).

Example summary of a database entry:

file	Summary	Category

Projectplan_XYZ.docx	“Project X – Milestones, tasks, responsibilities…”	New results
Code_Skript1.py	“Script for data processing: Functions parse_data(), export_csv() …”	New results
Diagramm_ProjektZ.png	“Diagram Project Z – Visualization of the workflow…”	New results

---

Step 5: Dashboard / Search / Retrieval

- **Filter options:**
  - File type: Word, Excel, PDF, Code, Image, Video, Audio
  - Folder / Category
  - Project, Customer, Priority, Department
  - Date: Creation / Modification
  - New results vs. baseline data
- **Examples:**
  - Search for "Project X" → shows all files + summaries
  - Filter by file type = Video → Presentation_Alpha.mp4
  - Filter by category = "New results" → all LLM summaries
- **Retrieval:**
  - Open file → Original application
  - View / edit metadata
  - Show summary
  - Changes are automatically saved in the database and knowledge base.

Step 6: Expansion & Integration

- Plugin1 document numbers → only new documents → old files retain only metadata
- New files can be uploaded at any time → Metadata can be expanded automatically and manually
- LLMs can be run again on request for individual files or entire folders
- Export options: CSV / Excel → Documents, metadata, summaries

---

💡 **User flow in brief:**

1. Place files in folders → Plugin2 scans + imports

2. Automatic metadata → add manually
3. LLMs generate summaries / code analyses / indexing
4. Dashboard: Search, Filter, Retrieval, Editing
5. New results → own category → basic data remains unchanged
6. Export / Reports → for analyses, dashboards, project reports

---

If you want, I can as**Next step: "play" a step-by-step simulation with all file types.**, where each file is shown as it is imported, metadata is assigned, analyzed by LLMs and summarized – including images, videos, code and audio.

Do you want me to do that?

I've created a folder structure on my external hard drive: projects, contracts—the same structure as in the plugin. It then automatically copies my files from there to the plugin. Now, regarding the document number generator: I've generated a document number and created a document, but haven't yet assigned it to a folder. Naturally, I want to be able to manually specify which folder it belongs in. If I purchase plugin 2, I'll need storage space for it as well.

Okay, I'll summarize that clearly and structure it.**Integration of Plugin1 + Plugin2 with manual folder assignment and storage management**, so that your workflow is logical and future-proof.

Workflow – Document number generator + knowledge database with folder structure

---

1️⃣ Folder structure on external hard drive

- You have already created folders:

/Projects
/contracts
/Reports
/Blog-Uploads
/Media

- Plugin2 can automatically import files from these folders.

- All imported files are **copied** into the knowledge base structure, but remain referenced on the hard drive (for large files, media).

2️⃣ Receipt number generator (Plugin1)

- **Document created + receipt number generated**
- Not yet assigned → Folder must be manually selectable
- Workflow:
1. Open a new document → “Generate receipt number”
2. Optional: Enter metadata (project, customer, priority, custom fields)
3. **Select folder:**
- Dropdown from Plugin2 folder structure
- When selected → document automatically moved to this folder or saved in the database with folder ID
- **Document number + metadata + folder assignment** → all stored in the database

---

3️⃣ Storage space & Plugin 2

- If Plugin2 is purchased → Storage space required for:
- Copied documents (Word, Excel, PDF, code, images, videos, audio)
- Metadata, document numbers, summaries of the LLMs
- Large files → Recommendation: store externally + path in database (BLOB only for small files)
- Automatic import → Monitor storage usage (Dashboard → Status "Storage used / free")

4️⃣ Integration: Plugin 1 + Plugin 2

1. Document generated with receipt number (Plugin1)
2. Manual folder assignment → DB + folder structure updated
3. If Plugin2 is active:
- File is imported → LLMs can generate summaries, code analyses, and semantic indexing. Metadata can be extended or customized.
4. Dashboard shows:
- Document number, Document, Metadata, Folder
- Storage status + category "New results"

5️⃣ Example: User Flow

1. Generate document number → Document not yet in folder
2. Enter metadata → e.g., Project X, Customer ACME

3. Select folder → `/Projects/Project X`
4. Storage space checked → Document is being imported, LLMs are running (if Plugin2 is active)
5. Dashboard shows:
- Document number: PX-2026-000047
- Folder: Project X
- Metadata: Project, client, priority, etc.
- Summary (if Plugin 2)
6. File can be accessed, edited, or exported at any time.

💡 **Note:**
- **Document number + metadata + folder = complete key**
- **Plugin1**: Numbers + metadata, manual folder assignment
- **Plugin 2**: Knowledge Base + LLM Analysis + Memory Management
- **Storage space planning**: large files → external + database paths → efficient & scalable

---

If you want, as a next step I can **create a complete simulation** that shows:

- A document from Plugin1 → manual folder assignment → import into Plugin2 → LLM summary → storage management → dashboard view.

Do you want me to play through this as a text simulation?
What does "differentiated" mean? My files on the external storage shouldn't be touched or changed. So, if I have a folder there, or two folders, and I've created a folder structure that's exactly the same as in the plugin, then it should automatically copy all the files from each folder to the corresponding folder in the plugin. I don't want to do it manually for thousands of files.

Ah, now it's clear: "differentiated" here simply means that**Not every file is treated the same**, but the system recognizes where each file belongs – but you want**Leave your originals on the external hard drive untouched.**Everything that happens should**only create one copy in the plugin system**.

This is what the workflow would look like:

Automatic, differentiated import – copies only

1. **Recognize folder structure**

    - The plugin scans external folders, e.g.:
      /Projects
      /contracts
      /Reports
    - Subfolders are also recognized.
2. **Automatic copying**
    - Each file from the folder `/Projects` → Copy the plugin to the internal plugin folder `/Projects`
    - Each file from `/contracts` → Copy the plugin to the internal plugin folder `/contracts`
    - **Original files on the external hard drive remain untouched.**
3. **Metadata & Assignment**
    - The plugin automatically reads: filename, creation date, file type
    - Folder name → Category / Folder ID in plugin database
    - Optionally add metadata manually (project, client, priority, keywords)
4. **Scalable**
    - Works for thousands of files
    - No manual adjustment required
    - Files are copied once → LLMs can analyze them → summaries, search index, code agent
5. **Plugin folder vs. external hard drive**
    - **External hard drive:**unchanged, originals remain clean
    - **Plugin storage:**Internal copies → Processing, document numbers, LLM summary
    - Storage space requirements are displayed in the plugin dashboard.

💡 **Note:**

- "Differentiated" = each file is automatically assigned to the correct folder in the plugin,**The original remains untouched.**
- Plugin copied → internal administration + LLMs → dashboard & search → metadata + summaries

---

If you want, as a next step I can**show a step-by-step simulation**, How **Thousands of files are automatically copied from your external folder structure into the plugin, metadata is assigned, and LLMs are started.**, all without altering the original files.

Do you want me to do that?

And

Perfect, then we'll do one**complete text simulation**Step by step – so that you can see exactly how**Thousands of files from your external folder structure are automatically**

copied into the plugin, metadata is assigned, and LLMs are started., without altering the originals.

Simulation: Automatic Import & Knowledge Base – Multi-File Types

---

Step 1: External folder structure on USB / hard drive

External hard drive:

```
├── Projects
│  ├── Project1
│  │  ├── Plan.docx
│  │  ├── Budget.xlsx
│  │  └── Diagram.png
│  └── Project2
│  ├── Report.pdf
│     └── Code.py
├── Contracts
│  ├── Contract_A.pdf
│  └── Contract_B.pdf
├── Reports
│  └── Energy Report.pdf
├── Media
│  ├── Video_Presentation.mp4
│  └── Audio_Interview.mp3
```

- **Originals remain unchanged**
- The plugin automatically detects subfolders → internal folder structure is identical.

Step 2: Automatic copying into the plugin

- Plugin scans external hard drive → detects all files and subfolders
- For each file:
    1. Check which folder it's in.
    2. Copy to internal plugin folder (`PluginStorage/Projects/Project1/...`)
    3. Automatically capture metadata:
        - File name

        - Type (Word, PDF, Excel, Code, Image, Video, Audio)
        - Date of creation / Date of modification
        - Folder name → Category

Example copy of database entries:

file	Type	Folder	Category	Datum
Plan.docx	Word	Project1	Projects	2026-03-12
Budget.xlsx	Excel	Project1	Projects	2026-03-12
Diagram.png	Picture	Project1	Projects	2026-03-12
Report.pdf	PDF	Project2	Projects	2026-03-11
Code.py	Code	Project2	Projects	2026-03-11
Contract_A.pdf	PDF	Contracts	Contracts	2026-03-10
Video_Presentation.mp4	Video	media	media	2026-03-09
Audio_Interview.mp3	Audio	media	media	2026-03-09

Step 3: Document Number Generator (Plugin 1)

- Create new document → Document number generated
- Not yet assigned → User selects folder manually
    - Dropdown: `/Projects/Project1`, `/contracts`, etc.
- Document number + metadata + folder → Plugin database

Example:

- Document: Invoice_X.docx
- Document number: PX-2026-000048
- Folder: Project1
- Metadata: Project X, Client ACME, High priority

Step 4: LLM Integration (Plugin 2)

For all files in the plugin storage:

1. **Code-Agent**→ analyzed `.py`Files → Functions, Variables, Comments
2. **Summary Agent**→ DOCX, XLSX, PDF, text, images, videos → Generate summaries
    - OCR for images
    - Metadata + Date + Folder + Summary
3. **Such-Agent**→ Full-text and semantic search for all file types

Example summaries:

file	Summary	Category
Plan.docx	"Project 1 – Milestones, tasks, deadlines…"	New results
Budget.xlsx	"Project 1 – Financial planning, cost overview…"	New results
Diagram.png	"Project 1 – Workflow Diagram Visualization…"	New results
Code.py	"Script for data analysis: parse_data(), export_csv()…"	New results

---

Step 5: Dashboard – Search & Filter

- Filter by:
    - File type (Text, PDF, Excel, Image, Video, Audio, Code)
    - Folder / Category / Project / Client / Priority
    - Date: Creation / Modification
- Full-text search + semantic search
- LLM summaries available directly
- Open/edit document → Changes synchronized

Step 6: Memory Management

- Large files → external storage system possible, path in database
- Small files → copied internally
- Dashboard shows storage status → free / occupied

💡 **User-Flow compact**

1. Plugin scans external folders → copy to plugin storage
2. Metadata automatically extracted from folder and file information
3. Document number for new documents → Select folder manually
4. LLM agents analyze all files → summaries + search index
5. Dashboard → Search, Filter, Retrieval, Editing
6. Monitor storage space → large files externally

If you want, as a next step I can**create a detailed "Live Flow" simulation**, at the **All file types – DOCX, PDF, XLSX, images, videos, audio, code – are automatically imported, receipt numbers are assigned, and LLMs are started.**, and you can see exactly how everything ends up in the knowledge database.

Do you want me to do that?

And

Okay, then we'll play the**realistically experience the complete live flow once**, just as a user actually uses the system. Your [data] remains unchanged.**Original files on the external hard drive unchanged**, and the plugin only works with**automatic copies**.

Live-Simulation des Systems

1. Initial situation

On your**external hard drive**A structure already exists:

External hard drive/
Projects/
Project1/
    Plan.docx
    Budget.xlsx
    Diagram.png
Project2/
Report.pdf
    Code.py

Contracts/
Contract_A.pdf
Contract_B.pdf

Media/

Video_Presentation.mp4
    Audio_Interview.mp3

This structure is**identical to the folder structure in the plugin**.

---

2. Initial setup in the plugin

The user opens the plugin dashboard.

There is a function there:

"Synchronize external folders"

The user selects the path to the external hard drive.

Example:

/Volumes/ExterneFestplatte/

Then he clicks on:

"Scan folder"

---

3. Automatic import (thousands of files possible)

The plugin now does the following automatically:

1. Recognize folder structure
2. Analyze files
3. Make copies
4. Store metadata

The original files will be**not changed**.

The internal plugin structure is created automatically:

PluginStorage/

Projects/
Project1/
    Plan.docx
    Budget.xlsx
    Diagram.png

Project2/
Report.pdf
    Code.py

Contracts/
Contract_A.pdf
Contract_B.pdf

Media/
Video_Presentation.mp4
  Audio_Interview.mp3

---

4. Automatic Metadata

During import, the system automatically saves:

Field	Those
File name	file
File type	File extension
Date of creation	file system
Date of change	file system
Folder	Folder structure
Category	Top folder

Example data set:

Datei: Plan.docx
Typ: Word
Folder: Projects/Project1
Category: Projects
Created: March 12, 2026
Last updated: March 12, 2026

---

5. LLM analysis starts automatically

Now the AI agents are starting to work.

Agent 1 – Document Analysis

DOCX
PDF
Excel
Code

are being read.

Example:

Plan.docx → Project plan recognized
Budget.xlsx → financial data recognized

Agent 2 – Image / Video / Audio

Diagram.png
Video.mp4
Audio.mp3

are analyzed.

Examples:

Diagram.png → OCR recognizes text
Video → Description generated
Audio → Transcript created

Agent 3 – Summary

The system automatically generates short analyses:

Plan.docx
Project 1 – Project plan with milestones and task allocation
Budget.xlsx

Project 1 – Budget overview with cost structure
Code.py
Python script for data analysis using the functions parse_data() and export_csv()

These are stored in the category:

New results

---

6. Search in the dashboard

Now the user can search.

Examples:

Search for text

Project1

Result:

Plan.docx
Budget.xlsx
Diagram.png
Code.py

Search for content

Budget

Result:

Budget.xlsx
LLM Summary

Search by file type

Filter:

Videos

Result:

Video_Presentation.mp4

---

7. Create a new document with a receipt number.

Now the user is using**Plugin 1**.

He creates a document.

Then he clicks:

"Generate receipt number"

Example:

PRJ-2026-00051

Then a field appears:

Select folder

Dropdown:

Projects
Projects/Project 1
Projects/Project2
Contracts
media

The user selects:

Projects/Project 1

The plugin saves:

Document number
Document
Metadata
Folder

---

8. Result in the system

The dashboard now shows:

Document number	file	Folder
PRJ-2026-00051	Invoice.docx	Projects/Project 1
—	Plan.docx	Projects/Project 1
—	Budget.xlsx	Projects/Project 1

---

9. Important: Originals remain untouched.

The external hard drive remains exactly the same:

External hard drive/
Projects/
Contracts/
Media/

The plugin works**only with copies**.

---

10. Scaling

The system can easily:

- 10,000 files
- 100,000 files
- several TB media

process because:

- Files can only be copied
- Metadata is stored in the database
- LLM index is stored separately

The big advantage of your system

You essentially get a**local AI knowledge engine**:

- automatic archiving
- intelligent search
- Document analysis
- Media analysis
- Metadata management
- Document number system

Everything **local, open-source and independent**.

If you'd like, I can show you something very important:

The ideal system architecture for this plugin ecosystem (document generator + knowledge database + AI agents)– built in such a way that later**It can also be sold as commercial software.**

That would be the next logical step.

With a thousand files, an AI agent simply won't find anything. And it's so trivial that it doesn't need to start anything automatically. The user has to be able to decide that themselves. Try running the AI agent through this folder group, and it will provide a detailed prompt; otherwise, it's pointless. And that's what will be processed—not automatically, this nonsense for a thousand files.

Ah, exactly – that makes sense. Then we'll change the system like this:**No automatic running of AI agents with large folders**, but the user decides**folder- or file-related**when and how the AI should work.

Differentiated workflow with AI start based on user decision

1️⃣ Import the files

- All files will be**automatically copied**into the plugin (Word, Excel, PDF, images, videos, audio, code)
- **Originals remain unchanged**
- Metadata is automatically generated from folder and file information.
- **No AI agent runs automatically**
    - This saves resources and prevents pointless analyses of thousands of files.

2️⃣ Selection for AI agents

- The user opens the dashboard → selects a folder or individual files
- Click on**"Start AI agent"**
- Input field appears →**More detailed prompt**, z. B.:
    - "Summarize all project reports"
    - "Analyze the code and create a function overview"
    - "Create summary of text, tables and PDF"
- AI agent processes**only this selection**
- Results will be presented in**new category**"New results" saved

---

3️⃣ Advantages

- **Full control:**The user decides when AI works.
- **Resource-efficient:**No automatic analysis for thousands of files
- **Flexibility:**Each prompt can be customized → detailed results

---

4️⃣ Dashboard-Integration

- Selection field for folders/files → Button**"Start AI agent"**
- Prompt field → Text input for analysis / summary
- Edition → Category “New Results”
- Metadata can be added.
- Filter & search as usual → all results visible

💡 **Note:**

- **Import = automatic**→ Copy files, save metadata
- **Analysis = manual as needed**→ AI agent only on selected files or folders

- So the system remains**scalable**, even with thousands of files, without wasting resources

---

If you want, as a next step I can...**concrete simulation**, as the user**selects a folder, starts the AI agent, enters a detailed prompt, and the results appear in the knowledge base.**, Step by step.

Do you want me to do that?

He then has the option to automatically use AI on new files or manually.

Perfect, then let's summarize it like this:**The system offers two modes**, which the user can select for each folder or for new files.

AI Workflow – Automatic vs. Manual

1️⃣ Import the files

- Files from external folders are automatically copied into the plugin.
- Originals remain untouched
- Metadata is automatically generated from folder and file information.

2️⃣ Mode selection for AI agents

A) Automatic mode

- **Option:**User activates "Automatic AI for new files"
- Once new files are imported:
  - AI agents will automatically start for these files
  - Results are immediately saved in the "New Results" category.
- Useful for smaller folders or documents that need to be analyzed regularly.

B) Manual mode

- Standard for large folders or many files
- User selects**folder- or file-related** out of

- Click on**"Start AI agent"**
- A more detailed prompt can be entered → e.g.: summary, analysis, code review
- Results are saved in "New Results" upon completion.

3️⃣ Dashboard & Control

- For each folder / new files, the user can**set the default mode**: automatic or manual
- Overview in the dashboard:
  - Which folders run automatically?
  - Which files are awaiting manual AI processing?
- Filterable: Category “New Results” vs. Basic Data

💡 **Note:**

- **Automatically:**Small, regularly updated files → instant AI analysis
- **Manually:**large amounts of data → only if the user triggers the analysis
- So the system remains**scalable, flexible and resource-efficient**

If you want, as a next step I can**create a step-by-step simulation**, in which you can see:

1. New files imported
2. Folder set to automatic or manual
3. AI agents run on demand or automatically
4. Results appear in the knowledge database

Do you want me to do that?

And the results aren't automatically saved in a new file, no. The user has to decide that themselves. Or they can choose to save it automatically again. Okay, I have 1000 files, and then I tell the AI to extract the content. Give me ten keywords and save them in the knowledge base so I know my DAX is freely configurable; I can choose how many keywords I want, even 100, for example. I can enter that myself.

Okay, now it gets really flexible:**The user decides what happens with the AI results.**– No automatic file or summary, everything is configurable. Let me summarize:

AI Results Management – User-Driven

1️⃣ Import & Metadata

- Files from external folders → automatically copied to the plugin
- Originals on the hard drive remain unchanged
- Metadata from folder + file information automatically saved

---

2️⃣ Mode selection for AI agents

- **Automatically**: small folders / new files → AI agent can start immediately if desired
- **Manually**: large folders or as needed → user selects files/folders, starts AI agent, enters detailed prompt

---

3️⃣ Processing the results

- **The user decides:**
  - Results **Do not save automatically**→ Show only, check keywords
  - Or results**automatically save**→ in the knowledge database
- Example: 1000 files, AI searches all of them → the user says:
  - "Extract 10 keywords per file"
  - or “Extract 100 keywords per file”
- Number of keywords freely selectable → flexible for each file or folder group
- Results will be presented in the**Knowledge database as keywords / metadata**saved, not as a new file

---

4️⃣ Dashboard-Integration

- Filters: Folder, Category, File Type, Metadata
- Overview:
  - Which files have already been analyzed by AI?
  - How many keywords were extracted?
  - Whether results are saved or only displayed
- You can choose at any time:**save / don't save / start new analysis**

💡 **Note:**

- **AI only runs when needed**
- **Saving the results = User decision**
- **Number of keywords is flexible**→ z.B. 10, 50, 100 pro Datei
- No automatic file creation → maximum control for the user

If you want, I can do one now.**build a concrete live simulation**, where you can see:

1. 1000 files imported
2. User selects folder/files
3. AI agent extracts 10 to 100 keywords
4. The user decides which results are stored in the knowledge database.

Do you want me to create this simulation now?

The second AI agent can extract code from a file. We tell it to search for all code blocks, add the keyword, and save the code blocks in a separate file, but linked to the original file and linked to the knowledge base.

Perfect, then we'll add a [unclear text] to the system.**Code-specialized AI agents**, which does exactly that: extract code blocks, tag them with keywords, save them in a separate file, and still maintain the connection to the original file and the knowledge base.

Code-Agent – Workflow

1️⃣ Initial situation

- File:`Analyse_Skript.py`in the plugin storage
- Folder: `/Projects/Project2`
- Metadata: Project X, Client ACME, High priority

2️⃣ User selection

- User selects**file or folder** out of
- Click on**"Start Code Agent"**

- Prompt: z. B.
  "Find all code blocks, extract them, add a keyword"`Data analysis`"Add it, save it in a separate file and link to the original file and knowledge base."

3️⃣ Processing

1. AI agent scans file for code blocks
   - Python, JavaScript, SQL, etc.
2. Extracts each code block → creates**new file(s)**
   - Filename:`Analyse_Skript_block1.py`, `Analyse_Skript_block2.py`
3. Keyword will be automatically assigned (`Data analysis`)
4. Each new code file will**linked**:
   - Original file → Reference in the knowledge base
   - Metadata from the original file is retained (project, customer, folder, date)
5. Saved to plugin folder`/Projects/Project2/Code_Extracts/`

4️⃣ Knowledge base link

- New code files will be created**as separate entries**stored in the knowledge database
- Links:
  - Field `Source file` → `Analyse_Skript.py`
  - Field `keyword` → `Data analysis`
- Search and dashboard can:
  - Show all code extracts for a source file
  - Filter by keywords

5️⃣ Advantage

- **Structured Code Analysis**without manual processing
- **Linking to metadata & knowledge base**→ later search by functionality, project, keywords
- **Separate Dates**→ no original file modified
- Flexible: Users can decide,**whether he wants to save the code files or only display them**

💡 **Note:**

- Code-Agent = **Extract code + assign keyword + separate file + link**
- Original file = unchanged
- Storage in knowledge base is optional and user-controlled.
- Fully transparent, scalable, large folders → only start when needed

---

If you want, as a next step I can**build a complete simulation**, at the:

1. A folder containing multiple code files is selected.
2. Code agent extracts all blocks
3. Keywords are assigned
4. Files are stored separately and linked correctly.
5. The dashboard displays the links.

Do you want me to simulate that now?

This specialized AI agent can find summaries, concept papers, or architectures instead of code.

Perfect – then we'll expand the specialized agent into a**flexible "analysis agents"**, the **Code, summaries, concept papers, or architectures**can process. The logic remains the same:**Everything is optional, everything is linked, everything is under the user's control.**

Flexible Analysis Agent – Workflow

1️⃣ Initial situation

- Folder: `/Projects/Project2`
- Files:
  - `Analyse_Skript.py` → Code
  - `Project plan.docx`→ Text/Project description
  - `Architecture_Design.pdf`→ Concept / Architecture Diagrams
- Metadata: Project X, Client ACME, High priority

2️⃣ Selection & User Prompt

- User selects**file(s) or folder** out of
- Click on**"Start analysis agent"**
- Prompt can be freely defined, e.g.:

1. **Code:**
   "Search for all code blocks, add keyword"`Data analysis`"Add, save separately and link to the original file."
2. **Text / Summary:**
   "Summarize all the key points, extract 10 keywords and link to the original file."
3. **Concept papers / Architecture:**
   "Create a brief description of the architecture, identify key components, save the summary separately and link it."

3️⃣ Processing

- The agent recognizes the**File type automatically**→ decides which analysis is useful
- Results:

Type	action	storage	Link
Code	Extract code blocks	Separate Dates	Source file + knowledge base
Text / DOCX / PDF	Summary + Keywords	Optional save	Source file + knowledge base
Architecture / Concept Paper	Brief description + key components	Optional save	Source file + knowledge base

---

4️⃣ User control

- Users can**for each file/folder** choose:
  - Start analysis or not
  - Automatically save results or only display them
  - Freely define the number of keywords (e.g., 10, 50, 100)

5️⃣ Knowledge Base Link

- Each new analysis file / summary → separate entry
- Link:

    - Field `Source file`→ Original file
    - Field `Keyword(s)`→ generated by the user or automatically
    - Category / Folder → adopted
- Dashboard enables search for:
    - Keywords
    - File type
    - Source file
    - Folder / Project

6️⃣ Advantages

- **Flexible:**An agent for code, text, summaries, architecture
- **Controlled:**The user decides when and what is analyzed.
- **Scalable:**Even with 1000+ files, there is no automatic overload.
- **Linked:**All results remain traceable with original files.
- **Optionally saved:**User chooses themselves

💡 **Note:**

- **Analyse-Agent = All-in-One-Option:**Code, text, concepts, architecture
- Results **are not automatically saved**→ full control
- **Link to the original file + knowledge base**Guaranteed traceability
- **Scalable, flexible, resource-efficient**

If you want, I can do a next**Build a complete simulation**, in the:

1. A folder containing mixed files is selected
2. The analysis agent is started.
3. Results are generated for code, summaries, and architectures.
4. The user decides what gets saved.
5. Everything is linked in the knowledge base.

Do you want me to simulate that now?

I have a file or folder and then select Agent LLM1, 2, or 3. Now do this work. Search for me, give me, Agent 1 is assigned, search. Okay, and there's a prompt there that can be navigated and changed. Search for 100 keywords about the content. Okay, and save what we just had. And now comes the Code Agent, this is new. And the Code Agent can only extract snippets. And number 3 can only summarize text, but it won't save anything automatically. Only after I've told it how the file should be structured, and further analyzed

how it should be prepared, will I eventually have the structure I want. And then I tell it to now save all these code blocks we've worked out in the knowledge base, in a separate file, in a separate folder for codes, which I can flexibly create, code for this folder, and link it to the knowledge base. And the same goes for the summaries. So, I go there, select "Summary" for this folder, for this file, edit it a bit more with a few prompts, and then I have the structure and select "Save this in a new file as a summary" and link everything back to the previous one. And all the keywords I have, they all have to be assigned to the file. Yes, so the search definitely has to be saved.

Okay, now we have a**precise, flexible system with three specialized agents**– and the user decides**step by step**What happens, how results are processed, and when they are saved in the knowledge base. I'll summarize it clearly:

Three-stage agent workflow – user-driven

1️⃣ Starting point

- File or folder selected
- Three agents available:**LLM1**, **Code-Agent (LLM2)**, **Text Summary Agent (LLM3)**
- Each agent has**a stored prompt**, the **customized by the user**can be

---

2️⃣ Agent 1 – LLM1 (keyword extraction)

- Function: Searches the contents of the file/folder and extracts keywords.
- Users can set: Number of keywords (e.g. 10, 50, 100)
- Promptly stored, e.g.:
  "Analyze the content and create 100 keywords that reflect the core message."
- Result: **List of keywords**
- Storage:
  - First displayed
  - The user then decides: "Save to knowledge base → link to original file"

3️⃣ Agent 2 – Code-Agent (LLM2)

- Function: Extracts**only code blocks / snippets**from file(s)
- Process:
  - User selects file/folder
  - Prompt: e.g., "Find all code blocks, add keyword"`Data analysis`"add"
  - Result: **new file(s) with code snippets**
- Storage:
  - **Not automatically**
  - User decides:
    - Create folders for codes → flexibly selectable
    - Save file → Plugin creates separate code file
    - Link → Original file + keyword + knowledge base

4️⃣ Agent 3 – Summary Agent (LLM3)

- Function: Only**Text/Summaries**
- Process:
  - Select file/folder
  - Edit/custom prompt → User can add details
  - LLM3 creates a summary → will**Displayed first, not automatically saved**
- Storage:
  - User decides: “Save now”
  - Optional:
    - Create a folder for summaries
    - Save new file
    - Link to the original file + keyword + knowledge base

5️⃣ Metadata and Linking

- All results →**Keywords**are automatically assigned to the original file
- Code files / summaries →**separated**, but linked
- Dashboard shows:
  - Which file/folder is being analyzed?
  - Which results are generated
  - Which results were saved or only displayed?

6️⃣ User control

- Complete control over:
  - Which agents are starting?

    - Prompt customization
    - Number of keywords
    - Storage location for codes / summaries
    - Time of saving in the knowledge base

💡 **Note:**

- **Agents only work on instructions.**→ no automatic saving
- **LLM1 → Keyword Extraction**
- **LLM2 → Extract code snippets**
- **LLM3 → Summaries**
- All results**First display → then save**, linked to original file + knowledge base
- Flexible for large folders, thousands of files → resource-efficient, scalable

---

If you want, I can now**build a complete live simulation**, at the:

1. A folder containing mixed files is selected.
2. LLM1 extracts 100 keywords → displayed
3. Code agent extracts snippets → save as a separate file → optionally
4. Summary agent creates text summary → optionally save
5. Everything is linked and assigned keywords.

Do you want me to create this simulation now?

Agent 1 needs to offer a choice between automatic and manual operation. It also needs to be possible to automatically analyze the entire folder or all 100,000 files and list 10 or 20 keywords for each file. These keywords must also be stored as metadata for each file. This needs to be the first step. We need more metadata to access it.

Perfect, that can be clearly depicted:**Agent1 will be fully scalable**, with **automatic or manual**, **Folder or complete analysis**, and the **Keywords are stored as metadata directly with each file.**I'll outline it step by step:

Agent1 – LLM1: Scalable Keyword Analysis

1️⃣ Mode selection: Automatic or Manual

- **Automatically:**
  - New files or existing folders → LLM1 starts automatically
  - User defines the number of keywords per file (e.g., 10, 20, 50, 100)
  - Option: entire folder or all files in the plugin
- **Manually:**
  - User selects individual file or folder
  - Click on**"Start LLM1"**
  - Prompt can be customized.
  - Results displayed first → Saving optional

2️⃣ Scope of the analysis

- **Folder analysis:**LLM1 can**only one folder**analyze
- **Complete analysis:**LLM1 can**all 100,000 files**go through
- Each file → list of keywords → directly as**Metadata**saved

---

3️⃣ Metadata stored per file

Example of a single file:

Field	Value
File name	`Report_Project1.pdf`
Folder	`Projects/Project 1`
Category	`Projects`
Date of creation	12.03.2026
Date of change	12.03.2026
LLM1 keywords	Analysis, budget, milestones, risks, tasks, team, schedule, priority, costs, deviation
Status	"Analyzed"

- All**Keywords as metadata**→ Instantly filterable, searchable, linked to original file

---

4️⃣ System process

1. User selects**folder-wide or plugin-wide** out of
2. Select mode:**Automatic / Manual**
3. Set prompt or use default
4. Define the number of keywords → e.g., 10 per file
5. LLM1 is running → extracting keywords
6. Result:
    - Automatically stored as metadata
    - Optional: Display in the dashboard for monitoring
7. Other agents (code, summary) can then start → with the new metadata

---

5️⃣ Advantages

- **Scalable:**Folders, entire databases, or 100,000 files
- **Fully controllable:**manual or automatic
- **Direct metadata:**Enables filtering, search, and subsequent analysis.
- **Resource-efficient:**When executed manually, only the desired files are used.
- **Seamlessly integrable:**The code and summary agent access the metadata.

💡 **Note:**

- Agent 1 = **Keyword extraction**
- Automatic or manual
- Analyzes folder or all files
- **Keywords → Metadata per file**
- Basis for further analyses / agents

If you want, I can now**build a complete simulation**, at the:

1. A folder or the entire database is selected
2. LLM1 extracts, for example, 10 keywords per file.
3. Results automatically saved as metadata
4. The dashboard displays keyword metadata and filter options.

Do you want me to create this simulation now?

Agent 1, with its search function, can be started automatically or manually. You can specify whether to run the search automatically across all folders, manually, for a specific group, or for a single file. It then analyzes all keywords (e.g., 1100 keywords) and saves them as metadata. This metadata must be saved. The search agent can then also be used for manual searches or to find specific keywords or text.

Okay, then we now have**Agent1 as a universal search and keyword agent**, the **Fully scalable, flexible, and mandatory metadata storage.**That's it. I'll summarize that and structure the workflow:

Agent1 – LLM1: Search and Keyword Agent

1️⃣ Mode selection

- **Automatically:**
  - LLM1 runs without user action
  - Can **all folders**, **a group of folders** or **a single file**analyze
  - Number of keywords per file freely selectable (e.g. 10, 50, 100, 1100)
  - **Results are automatically saved as metadata.**
- **Manually:**
  - User selects specific file or folder
  - Click on**"Start LLM1"**
  - Prompt can be customized.
  - Analysis → Results will be**stored as metadata**
  - Optional: direct search in files

2️⃣ Functionalities

A) Keyword analysis

- LLM1 scans the contents of the file/folder
- Extracts keywords / tags
- Saves automatically as**Metadata for each file**
- Example of a file:

Field	Value
File name	`Report_Project1.pdf`
Folder	`Projects/Project 1`
Category	`Projects`
Keywords	Analysis, budget, milestones, risks, tasks, team, schedule, priority, costs, deviation, …
Number of keywords	1100
Status	"Analyzed"

B) Search function

- The user can access the agent.**for targeted search** to use:
  - Keyword search: "Find all files that`Budget` contain"
  - Text search: "Find all occurrences of`Project planning`"
- Results appear directly in the dashboard
- Metadata is not changed, only queried.

---

3 Example of a process

1. **Automatic analysis:**
   - All folders → 1000 files → LLM1 extracts 10 keywords per file →**stored as metadata**
2. **Manual analysis:**
   - User selects folder “Project2”
   - Prompt customization: "Find 50 keywords that summarize the key points"
   - Results →**Metadata stored**
3. **Targeted search:**
   - User clicks on "Search Agent"
   - Input: "Keyword"`Budget`“
   - Agent searches files / metadata
   - Result: List of all matching files

4️⃣ Advantages

- **Fully scalable:**single file → folder group → entire database
- **Automatic or manual:**full control
- **Mandatory metadata:**No data will be lost.
- **Search integrated:**usable both automatically and selectively
- **Base for other agents:**The code agent and summary agent access this metadata.

💡 **Note:**

- Agent 1 = **Keyword and search agent**
- Automatically or manually, on file, folder group or everything
- Number of keywords freely selectable
- **All results are stored as metadata.**
- Supports targeted searches for keywords or text content

If you want, as a next step I can**build a complete simulation**, at the:

1. LLM1 automatically scans all files in a folder → 100 keywords per file
2. Keywords are stored as metadata.
3. Manual search via keyword/text is demonstrated.
4. The dashboard displays the saved metadata and search results.

Do you want me to simulate that now?

So, after the search, where you have the many metadata, you can either select folders or click on metadata, and then the code agent comes and searches these files, which were found via metasearch or via folders, it finds the code snippets, displays them and they can be saved in a separate file, linked to the original files.

Exactly – so now we have one**seamless, user-driven workflow**, the **Metadata, folder structure, and all three agents**integrated. Let me summarize it clearly:

Integrated workflow: Search → Code → Summary

1️⃣ Starting point: Metadata & Search

- **Agent 1 (LLM1)**Analyzed all files → Keywords as**Metadata**saved
- Users can**search specifically** above:
  - Folder
  - Keyword/Metadata
  - Text search
- Search results:**List of files**, which are relevant for the next processing

---

2️⃣ Code Agent (LLM2) on Metasearch

- User selects the results from LLM1:
  - e.g. files from metasearch → all files with keyword`Budget`
  - or specific folders
- Click on**"Start Code Agent"**
- Prompt can be customized → e.g.:
  "Search for all code blocks, add keyword"`Data analysis`"Add, save in a separate file, linked to the original file."
- **Result:**
  - Code snippet extracted
  - Display in the dashboard
  - Option: save in**separate file / folder**
  - Linking → Source files + Knowledge base

3️⃣ Storage & Linking

- New code files will be created**not automatically saved**, only according to user choice
- Each code snippet:
  - Link to the original file
  - Import metadata: Folder, Project, Customer, Keyword
- Folders for code files can**flexible from the user**be created
- **Dashboard** shows:
  - original file
  - Extracted code file
  - Related keywords

4️⃣ Optional: Summary Agent (LLM3)

- In the same way, the user can**the files found for text summaries** choose
- Customize prompt → LLM3 creates summary
- Storage → only after user decision
- Summary → linked to original file + keywords

5️⃣ Advantages

- **Fully inspected:**The user decides which files are analyzed.
- **Automatable on large datasets:**LLM1 can scan everything
- **Targeted extraction:**Code agent filters only from selected files
- **Metadata-based:**All results are searchable and linked
- **Separate files:**Originals remain unchanged
- **Scalable & flexible:**Folder structure, number of keywords, storage → all user-controlled.

💡 **Note:**

- **Workflow = LLM1 → Search / Metadata → LLM2 Code → LLM3 Summary**
- All steps**controlled, optionally save, linked**
- Metadata forms the basis for targeted analyses and subsequent evaluations.

If you want, I can**now build a full simulation**, at the:

1. Files can be filtered by metadata
2. Code agent extracts all snippets → Display
3. The user decides which code files are saved.
4. Dashboard shows links to original files and keywords.
5. Optional summary in the same way

Do you want me to simulate that now?

Agent 3, I then tell him to search the folder and all its metadata with the keyword. So, search for concept papers. Or link to the articles. Or for PULS. Then he creates these articles, copies them verbatim, and saves them in a file. With links to the other things. But anyone can do further editing of these files on their own; we don't need an agent for that. He's supposed to help find summaries from hundreds of files, find the code from hundreds of files, or supplement the files with metadata using metasearch.

Okay, now we have a very clear...**user-centered model**, in which **Agent 3 (Summary Agent)**based on**Folders and metadata**works, but manual further processing**left to humans**Let me summarize this clearly:

Agent3 – Summary Agent (LLM3) Workflow

1️⃣ Starting point

- User selects**Folder or meta search results** out of
- LLM1 metadata and keywords serve as filters:
  - e.g. keywords “project plan”, “PULS”, “concept paper”
- Selection can be based on**individual files or entire folder groups** take place

---

2️⃣ Analysis & Creation

- Quickly customizable:
  - "Search all files/posts with this metadata for concept papers"
  - "Create summaries / posts from the content"
- LLM3 scans the files → creates**summarized contents**
- Content will**Only saved after user decision**

3️⃣ Storage & Linking

- Results can**in a new file**be saved
- Link:
  - Each summary → Original file(s)
  - Link to metadata / keywords
- Folder structure for summaries**flexibly selectable**

4️⃣ User control

- All further processing → manually by humans
- Agent denient**only for extraction, summarization and orchestration**
- The code agent (LLM2) works in parallel in the same way → only code blocks
- LLM1 provides metadata → basis for targeted analysis

5️⃣ Summary workflow of all three agents

1. **LLM1 – Keyword Analysis & Metadata**
    - Automatic or manual
    - folder, file, entire database
    - Result = Metadata, keywords → Basis for agents 2 & 3
2. **LLM2 – Code-Agent**
    - Works on folder or metasearch results
    - Extracts code snippets → optionally saves, linked
3. **LLM3 – Summary Agent**
    - Works on folder or metasearch results
    - Extracts text / concepts / posts → optionally saves, linked
    - Further processing by humans possible

6️⃣ Advantages

- **Scalable:**Hundreds to thousands of files can be analyzed.
- **Flexible:**Each step is optional and user-controlled.
- **Controlled:**No automatic saving without user approval
- **Linked:**All results → Original files + metadata + dashboard
- **Division of labor human / agent:**Agent creates structure, human can take over the fine-tuning

---

💡 **Note:**

- **LLM1 = Metadata & Keywords**→ Basis
- **LLM2 = Extract code**→ separate files, linked
- **LLM3 = Summaries / Contributions**→ Save linked file (optional)
- **The user decides when, where, and what is saved.**
- **Manual further processing remains possible at all times.**

If you want, I can do one now.**Create complete simulation**, at the:

1. A folder or search result is selected
2. Agent1 extracts keywords → metadata stored
3. Agent2 extracts code → separate files, linked
4. Agent3 creates summaries → separate file, linked
5. The dashboard displays all links, metadata, and keywords.
6. The user decides what gets saved.

Do you want me to create this simulation now?

Let me summarize. Agent 1 is responsible for searching. The user can choose whether to search a folder, a file, metadata, or all folders. It should display and save keywords for each file in a knowledge base. The user can specify whether this should happen automatically or manually, and of course, it should link to each file later. Agent 2 searches a file, a folder, multiple folders, or metadata for code snippets or concept fragments and saves them in a new file of a file type specified by the user. Again, it can be set to automatic or manual, and it should link to each file. Agent 3 is responsible for summarizing. First, the user can select a file, a folder, all folders, and their metadata. This allows the user to choose which files should be summarized, but the text should only be transferred from the file itself. All the data to be summarized should be copied verbatim from the user's selected source files into the new file, and only then should the summary be generated. And of course, this applies whether the files are saved automatically or not, and each file is linked via the knowledge base. This is the only way to quickly analyze and work with thousands of files. Then users can continue working with other chats; everyone has their own ideas. It's really about knowledge retrieval, about being able to find things again.

Exactly – you've summarized that perfectly. I'll summarize it again in a structured way so it's clear how the**three agents working together**, everything **controlled**, **linked** and **scaled**:

Overview: Three-agent system for knowledge management

1️⃣ Agent 1 – Search and Keyword Agent (LLM1)

Function:

- Search for content in:
  - Single file
  - Folder
  - All folders / complete database
  - Via metadata/keyword filter

Results:

- Keywords per file
- Storage as metadata in the**Knowledge database**
- Link to the original file

Options:

- Automatic / manual
- Number of keywords freely selectable

Purpose:

- Basis for targeted analysis
- Filterability for code or summary agent

2️⃣ Agent2 – Code and Concept Snippet Agent (LLM2)

Function:

- Seeks **Code snippets or concept snippets** in:
  - Single file
  - Folder
  - Several folders
  - Files found via metasearch

Results:

- Extracted snippets → new file(s)
- The file type can be defined by the user.
- Storage optional: automatic / manual
- Link to all source files

Purpose:

- Structured collection of code or concepts
- Basis for further processing or project work

3️⃣ Agent 3 – Summary Agent (LLM3)

Function:

- Created**Summaries** out of:
    - Single file
    - Folder
    - All folders / files filtered by metadata

Process:

1. Base files selected → Contents will be**copied one to one**
2. A summary is being generated.
3. Storage: automatic / manual
4. Link to the knowledge base + original files

Purpose:

- Fast analysis of hundreds to thousands of files
- Content remains comprehensible.
- Users can continue working manually, adding to their work, and contributing their own ideas.

4️⃣ Core Principles

- **Controlled:**No agent stores data without user approval.
- **Linked:**Everything is linked to original files in the knowledge base.
- **Scalable:**Individual files → Folders → Complete databases
- **Flexible:**Automatic or manual, customizable prompt, selectable number of keywords
- **Basis for division of labor:**Agents provide structure and analysis, users handle the fine-tuning.

💡 **Note:**

- **LLM1 → Metadata / Keywords**
- **LLM2 → Codes / Concepts**
- **LLM3 → Summaries / Articles**
- All results**linked, optionally saved, scalable**
- **User decides**how results are further processed

If you want, I can do one now.**Build a live simulation**, which shows:

1. Selecting a folder or meta search
2. LLM1 extracts keywords → metadata

3. LLM2 extracts codes → new file, linked
4. LLM3 creates summary → new file, linked
5. Dashboard shows everything → filters, links, storage

Do you want me to create this simulation now?

And

Perfect. Then I'll build the simulation step by step as**Text-Demonstration**, which shows the complete workflow – including selection of files/folders, agent actions, metadata, code extraction, summaries and linking in the knowledge base.

Simulation: Three-agent system

1️⃣ Starting point

We have the following folder “Projects” containing files:

file	Type	Contents
`Project1_Report.pdf`	PDF	Project planning, budget, tasks
`Projekt1_Code.py`	Code	Python script for data analysis
`Project1_Concept.docx`	DOCX	Concept paper, architecture
`Project2_Report.pdf`	PDF	Project progress, risks
`Project2_Code.js`	Code	JavaScript snippets for visualization
`Project2_Concept.docx`	DOCX	Concept paper for Project 2

Dashboard:shows folder structure and all files

2️⃣ Agent 1 – Keyword Analysis / Metadata

User action:

- Selection: All folders → Automatic
- Number of keywords: 10 per file

Results (metadata):

file	Keywords	Status
Project1_Report.pdf	Project, budget, tasks, schedule, milestones, team, costs, priority, variance, analysis	Analyzed
Projekt1_Code.py	Python, data analysis, function, variable, loop, input, output, calculation, module, test	Analyzed
Project1_Concept.docx	Concept, architecture, strategy, workflow, resources, risk, requirements, design, implementation, process	Analyzed
Project2_Report.pdf	Project, progress, risk, task, team, schedule, costs, priority, analysis, adaptation	Analyzed
Project2_Code.js	JavaScript, visualization, DOM, function, variable, event, API, data, module, test	Analyzed
Project2_Concept.docx	Concept, architecture, planning, workflow, resources, risk, design, implementation, process, review	Analyzed

- **Metadata stored in knowledge database**
- A link to each original file is automatically included.

3️⃣ Agent 2 – Code-Agent

User action:

- Selection: All files, Filter: Metadata =`Code`

- Prompt: "Extract all code blocks, save as `.code.txt`, linked to original file"

Results:

New file	Included snippets	Link
`Projekt1_Code_Snippets.code.txt`	All Python snippets from`Projekt1_Code.py`	Linked → Project1_Code.py
`Projekt2_Code_Snippets.code.txt`	All JS snippets from`Project2_Code.js`	Linked → Project2_Code.js

- **Save optionally:**User clicks "Save" → files are saved.
- **Metadata:**Folder, original file, keywords transferred

4️⃣ Agent 3 – Summary Agent

User action:

- Selection: Folder`Projects`, Filter Metadata =`concept`
- Prompt: "Summarize all concept papers, keep the basic text verbatim, create a new file, link to the originals"

Results:

New file	Included basic data	Summary	Link
`Project1_Concept_Summary.docx`	Full text from`Project1_Concept.docx`	Summary of key points: Architecture, workflow, resources, risk, process	Linked → Project1_Concept.docx

`Project2_Concept_Summary.docx`	Full text from`Project2_Concept.docx`	Summary: Architecture, planning, workflow, resources, risk, implementation	Linked → Project2_Concept.docx

- **Save optionally:**User decides → Files saved
- **Metadata:**Folders, original files, and keywords were transferred.

5️⃣ Dashboard view by workflow

Filter & Overview:

- Folder: `Projects`
- Files: 6 original + 4 new (2 code + 2 summaries)
- Metadata available: Keywords, file type, status (analyzed / summary created / code extracted)
- Search function: Filter by keywords → shows all relevant files (original + new files)

6️⃣ Advantages of this simulation

- **100% user control:**Each step is optional, manual or automatic.
- **Scalable:**Single file → Folder → Entire database
- **Metadata-based:**Searchable, linked, basis for Agents 2 & 3
- **Separation Original / Results:**The original remains unchanged; new files are linked.
- **Division of labor human / agent:**The agent provides analysis and structure; the user can further edit the content.

If you want, I can in**Next step: an extended version**build the**10,000+ files simulated**, automatically generates metadata, extracts code, creates summaries and**Dashboard functions with filter/search demonstrated live**.

Do you want me to do that?

Agent 2 needs to allow you to select this folder, multiple folders, or just one file, and then filter by code. Alternatively, it shouldn't be a dedicated code agent that selects a specific

type, such as concept paper or concepts. You can filter these and then save them in a specific file type, linked to the original file.

Okay – then it is**Agent2, a flexible extraction agent**, the **not just code**, rather **any content by type**can filter (code, concepts, documents, etc.) and**these are stored in a user-defined file type.**, always linked to the original file. Let me summarize it clearly:

Agent2 – Flexible Extraction Agent (LLM2)

1️⃣ Choices

- **Scope of analysis:**
  - Single file
  - A folder
  - Several folders
  - Files from a meta search (based on Agent1 / keywords)
- **Filter type:**
  - Code (Python, JavaScript, etc.)
  - Concept papers / Architecture / Strategy papers
  - Any user-defined type
- **Quickly customizable:**Users can define,**what is to be extracted**

2️⃣ Results

- Extracted content is stored in**new file**saved
- **File type freely selectable** (z. B. `.code.txt`, `.concept.docx`, `.summary.md`)
- **Link**Automatically saved to the original file
- Storage:**manual or automatic**based on user decision

3️⃣ Workflow Example

1. User selects folder “Project1”
2. Filter: Type = Code
3. Prompt: "Extract all Python functions"

4. Result: File`Projekt1_Code.code.txt`→ linked with`Projekt1_Code.py`

Or:

1. User selects multiple folders “Project1” + “Project2”
2. Filter: Type = Concept paper
3. Prompt: "Extract key points, save in`.concept.docx`“
4. Result: File`Project1+2_Concepts.concept.docx`→ linked to all original concept papers

4️⃣ Advantages

- **Flexible:**Extraction not limited to code
- **Scalable:**File → Folder → Multiple Folders → Metasearch Results
- **Structured:**New document + link → Original files remain unchanged
- **User control:**Automatic/manual, customizable prompt, selectable file type

---

💡 **Note:**

- Agent 2 = **flexible extraction agent**
- Select File / Folder / Metasearch
- Filter by type (code, concept, any)
- Saves to new file →**linked to original file**
- Saving manually or automatically

If you want, I can**Update the simulation from the last step**, so that **Agent2 now filters flexibly by code or concepts**, saves in different file types and displays all links correctly.

Do you want me to do that?

Agent 3 can again only edit one file, or one folder, multiple folders, or any metadata. The selection is extremely important.

Exactly – that's a key point:**Agent3 must be flexible and finely controllable.**Otherwise, you'll lose track of things with hundreds or thousands of files. Let me summarize it clearly:

Agent3 – Summary Agent (LLM3) with flexible selection

1️⃣ Choices (extremely important!)

- **Single file:**Only the selected file will be summarized.
- **A folder:**all files in this folder
- **Multiple folders:**any combination of folders
- **Metadata filters:**only files that were analyzed via Agent1 and contain certain keywords/tags

Example:

- Folder “Project1” → combine all concept files
- Metadata filter = keyword "architecture" → only relevant files are processed.

2️⃣ Procedure

1. Make selection (file, folder, multiple folders, metadata)
2. Adjust prompt (e.g., "Create summary, keep original text verbatim")
3. Agent3 processes the content
4. New file is being created → optional save
5. Link to original files + metadata

3️⃣ Advantages of this selection control

- **Maximum control:**The user decides exactly which files are included in the summary.
- **Scalable:**From one file to thousands, selectively filterable
- **Transparency:**Original files remain unchanged
- **Link:**All summaries → Original files + metadata

💡 **Note:**

- **Agent3 = Summary Agent**
- Selection =**single file / folder / multiple folders / metadata filter**

- Result = new file, optionally save, linked
- Original content remains fully intact

If you want, I can now**Complete simulation for all three agents** once again **with these flexible selection options**Build it so that you can see exactly how the selection works for Agent2 and Agent3, how files, code and summaries are generated, and everything is linked in the knowledge base.

Do you want me to do that directly?

I don't just want to extract key points; I want the entire concept paper copied from the file, and I also want all the code copied for Agent 2. Not just some meager summary with all the code.

Ah, now it's clear:**Agent2 and Agent3 are not only supposed to extract excerpts or key points.**, but the**copy the entire contents of the file**Whether it's code or a concept paper, the user can then decide whether to add additional analyses, summaries, or metadata.

Let me summarize this clearly:

Agent2 & Agent3 – Full Content Transfer

Agent2 – Code & Concept Agent

Function:

- Select file, folder, multiple folders, or metadata results
- Filter by type (code, concept, any)
- **Takes over the entire content**of the selected files →**Copy one-to-one into a new file**
- New file → user-defined file type (e.g.`.code.txt`, `.concept.docx`)
- Link → Original file in knowledge base
- Storage: automatic or manual

Example:

- `Projekt1_Code.py`→ Agent2 generates`Projekt1_Code_Full.code.txt` → **all functions, classes, comments**taken over

- `Project1_Concept.docx`→ Agent2 generates`Project1_Concept_Full.concept.docx` → **entire concept paper adopted**

Agent 3 – Summary Agent

Function:

- Select file, folder, multiple folders, or metadata results
- **It also adopts the entire original text.**in new file →**Basis for summaries**
- Optional: User can then**Summary / Analysis** add
- Linking → Original files + metadata
- Storage: automatic or manual

Example:

- `Project1_Concept.docx`→ Agent3 generates`Project1_Concept_FullForSummary.docx` → **Full text adopted**
- Users can then add bullet points, analyses, or further prompts.

---

core principle

- **No loss of content:**Everything will be completely taken care of.
- **Metadata & linking:**Every new file remains traceable → Original file + Dashboard
- **Flexible & scalable:**Single file → Folder → Multiple folders → Metadata filter
- **User decides on further analyses:**Summaries, code extraction, keywords

---

💡 **Note:**

- **Agent2 = Full content for code or concepts**
- **Agent3 = Full content for summaries / analysis basis**
- Save all options + linked
- After that, humans can perform any further processing.

If you want, I can now**Update simulation**, so that **Agent2 and Agent3 take over all content completely**, including dashboard, links, metadata and storage options. Then you can see directly how it works in practice for hundreds of files.

Do you want me to do that?

The file contains the code, the concept, and much more. Agent 2 should then extract the code and the concept verbatim and copy them. Not everything, and not summarized.

Ah, now I understand exactly what it's all about:**Agent2 is an extraction agent.**, the **selectively copied specific content from a file**, **one to one**, without summarizing and without mixing in other content. So:

- A file can contain anything: text, code, concepts, tables, etc.
- **Agent2 searches only for the desired content type.**(Code or concept)
- Copied**exactly this content**in a new file
- The remaining contents of the original file remain untouched.
- New file → user-definable file type
- Link → Original file
- Storage → automatic or manual

Example

Original file`Projekt1_Mixed.docx`:

Text: Project description
Code: Python function for analysis
Concept paper: Architecture & Workflow
Images / Tables: Diagrams

Agent 2 Filter = Code:

- Results file:`Projekt1_Code.code.txt`
- Content: only the Python code
- Link → Original file

Agent2 Filter = Concept:

- Results file:`Project1_Concept.docx`
- Contents: only the concept paper
- Link → Original file

Key points

1. **Selective:**Only the desired type will be extracted.
2. **One-to-one:**No summary, no removal of details

3. **Separation:**Code and concepts in separate files
4. **Linking & Metadata:**Each new file → original file + keywords
5. **Flexible:**File, folder, multiple folders, metadata filter

---

💡 **Note:**

- **Agent2 = selective extraction agent**
- **Remove the code**→ new file
- **Remove the concept**→ new file
- **The rest remains unchanged.**
- **Everything linked, optionally saved, one-to-one.**

If you want, I can now**Update simulation**, so that **Agent2 automatically or manually extracts code and concepts separately for each file.**, one-to-one, with metadata and dashboard links. Then you can see directly how it works for multiple folders and hundreds of files.

Do you want me to do that?

With Agent 2, the user can specify: "This file, extract the code from it—just the code, exactly as it is, or also concepts, exactly as they are in a separate file, or based on a third and fourth criterion, resulting in the creation of multiple files." This could be implemented for an entire blog. For example, a folder project: we extract everything, exactly the code, and save it in one file. Or we extract both the code and the concepts, exactly as they are, and save them in separate files. Naturally, everything would be linked.

Now it is perfectly clear:**Agent2 becomes a flexible, selective extraction tool.**, the **Can process multiple criteria simultaneously**, for a**a single file or an entire folder**Each content type will**extracted one to one**, in **separate files**, all**linked**with the original file. Let me summarize:

Agent2 – Multi-criteria extraction agent

1️⃣ Selection & Scope

- **Single file:** z. B. `Projekt1_Mixed.docx`
- **Folder:**all files in a project folder

- **Multiple folders / metadata:**any selection
- **Filter criteria / content types:**
    - Code (Python, JS, …) → separate file
    - Concepts / Architecture → separate file
    - Optional: 3rd and 4th types as desired → each a separate file

Example blog folder:

- All articles/files are analyzed simultaneously.
- Agent2 creates for each content type**separate files**
- Each new file →**One-to-one extracted content**

2️⃣ Processing

1. User selects scope (file, folder, metadata)
2. User defines filters: code, concept, …
3. Agent2 scans the file(s)
4. For each criterion:
    - Extraction**one to one**
    - New file created
    - Link → Original file + metadata

3️⃣ Result

original file`Projekt1_Mixed.docx`contains:

Text: Project description
Code: Python function for analysis
Concept: Architecture & Workflow
Images / Tables

Agent 2 Extraction:

Filter	Results file	Contents	Link
Code	`Projekt1_Code.code.txt`	Python Function	original file
concept	`Project1_Concept.docx`	Architecture & Workflow	original file

Optional additional filters:e.g. tables →`Project1_Table.xlsx`

4️⃣ Options & Controls

- **Automatic / manual:**User decides
- **Multiple files simultaneously:**for entire folders or blog projects
- **File type selectable:** z. B. `.code.txt`, `.concept.docx`, `.table.xlsx`
- **Link:**All new files → Original file + metadata

💡 **Note:**

- **Agent2 = flexible multi-criteria extractor**
- Choose**single file / folder / multiple folders / metadata**
- Extracted**Code, concepts, other types → each separate file**
- Contents **one to one**, **do not summarize**
- **Everything is linked and optionally saved.**

If you want, I can now**Update simulation**, so that **Agent2 extracts multiple pieces of content simultaneously, one-to-one, from an entire folder structure/blog project.**All files created, all linked, and metadata imported. Then you'll see...**It's practical how hundreds of files of multiple types can be processed.**.

Do you want me to do that?

And

Perfect – then we'll build the**Simulation for Agent2 (Multi-criteria extractor)**and Agent3 (Summary Agent) in such a way that you can see how**an entire folder or several folders**The following types will be processed: code, concepts, and other types**extracted one to one**, **separate files**, **linked**, with metadata and optional storage.

Simulation: Agent2 & Agent3 – Folder/Project Processing

1️⃣ Starting point: Blog project folder

Folder structure on external hard drive:

```
Blog_Projects/
├─ Artikel1_Mixed.docx
├─ Article2_Mixed.docx
├─ Projekt1_Mixed.docx
├─ Projekt2_Mixed.docx
```

Example content`Artikel1_Mixed.docx`:

Text: Introduction and project description
Code: Python script for data analysis
Concept: Architecture & Workflow
Table: Budget & Tasks
Image: Diagram

Example content`Projekt1_Mixed.docx`:

Text: Project planning
Code: JavaScript functions
Concept: Strategy paper
Table: Resource plan
Image: Diagram

2️⃣ Agent 2 – Multi-criteria extractor

User action:

- Selection: Folder`Blog Projects`
- Filter criteria:
  1. Code → new file`.code.txt`
  2. Concepts → new file`.concept.docx`
  3. Tables → new file`.table.xlsx` (optional)
- Selectable automatically or manually

Results – Overview per original file:

original file	Filter	New file	Contents	Link

Artikel1_Mixed.docx	Code	`Article1_Code.code.txt`	Python Script	Original file + metadata
Artikel1_Mixed.docx	concept	`Article1_Concept.concept.docx`	Architecture & Workflow	Original file + metadata
Artikel1_Mixed.docx	Table	`Article1_Table.table.xlsx`	Budget & Tasks	Original file + metadata
Projekt1_Mixed.docx	Code	`Projekt1_Code.code.txt`	JavaScript functions	Original file + metadata
Projekt1_Mixed.docx	concept	`Project1_Concept.docx`	Strategy paper	Original file + metadata
Projekt1_Mixed.docx	Table	`Project1_Table.table.xlsx`	Resource plan	Original file + metadata
…	…	…	…	…

Points:

- Each extraction**one to one**
- Contents**are not mixed**
- New files → linked to original + metadata
- For **entire folders or multiple folders** at the same time

---

3️⃣ Agent 3 – Summary Agent (optional)

User action:

- Selection:
  - Folder `Blog Projects`
  - Or files that Agent2 extracted
  - Or metadata filters (e.g., keyword "architecture")
- Prompt: "Create a summary based on the copied content"
- New file → optional save → linked

Result:

Summary file	Basic files	Contents	Link
`Article1_Concept_Summary.docx`	`Article1_Concept.concept.docx`	Summary of the architecture and workflow section	Original + Metadata
`Projekt1_Code_Summary.code.txt`	`Projekt1_Code.code.txt`	Overview of all JS functions	Original + Metadata

Important:

- **Agent3 only receives the content that was previously extracted by Agent2.**
- **Original files remain unchanged**
- **Users can add further analyses, summaries, or prompts.**

4️⃣ Dashboard view by workflow

- **Folder:** `Blog Projects`
- **Files:**Original + all extracted files (code, concepts, tables) + summaries
- **Filterable:**
  - By file type (code, concept, table, summary)
  - Searching by metadata / keywords
- **Links:**Each extracted file → original file
- **Storage:**The user decides which files are permanently saved.

5️⃣ Advantages

- **One-to-one extraction:**No loss of detail
- **Multiple criteria simultaneously:**Code, concepts, tables → separate files
- **Scalable:**Individual files → Folders → Entire projects
- **Flexible:**Automatic or manual, file type selectable
- **Linked:**All results → Original files + metadata
- **User control:**Further processing, summaries or analyses are optional.

💡 **Note:**

- **Agent2 = selective multi-criteria extractor**→ Code, concepts, tables, …**one to one**
- **Agent3 = Summary Agent**→ based on extracted content, optionally save, linked
- **The original remains unchanged.**, new files clearly structured and linked

---

If you want, I can do a next**Complete simulation for 1000+ files**build the**Agent2 runs in parallel for code & concepts**, **Agent3 summaries created** and **Dashboard + Metadata + Links**simulated live. Then you can see directly how the system works.**works on a large scale**.

Do you want me to do that?

Agent 3 can access the original files, including keywords, and can access folders—multiple folders, freely selectable. Then, it can automatically or manually copy the content verbatim into a new file. And then, the user can specify a summary, but they, the user, also want to know the original text in order to edit it.

Okay – now it's completely clear:**Agent3 is a summary agent with direct access to the original files.**, but **The original text is always adopted in its entirety.**before any summary is even generated. This gives the user full control and the ability to edit. I'll summarize it clearly:

Agent3 – Summary agent with original text

1️⃣ Choices

- **Single file:**only this file
- **Folder:**all files in a folder
- **Multiple folders:**freely selectable
- **Metadata filters:**only files that contain specific keywords

---

2️⃣ Working method

1. **Make a selection**(File, folder, multiple folders, metadata)
2. **Choose option:**
    - **Manually:**The user controls every step.

    - **Automatically:**The agent processes all selected files.
3. **Copy the content verbatim:**
    - Original text will**Completely copied to a new file**
    - No abridgment, no removal of details
4. **Optional:**Create a summary
    - Basis: fully adopted original text
    - User can edit summary
5. **Link:**New file → Original files + metadata
6. **Storage:**Manual or automatic

3️⃣ Example

Folder `Project1`contains:

- `Projekt1_Mixed.docx`
- `Project1_Concept.docx`
- `Projekt1_Code.py`

Agent 3 Selection:

- Folder `Project1`
- Filter: Keyword “Architecture”

Result:

New file	Contents	Link	Automatic / Manual
`Project1_Architecture_Full.docx`	Full original text of all files with the keyword "architecture"	Original files + metadata	Optional
`Project1_Architecture_Summary.docx`	Summary based on original text	Original files + metadata	Optional

---

4️⃣ Advantages

- **Original text remains fully visible:**for post-processing, analysis, proofreading
- **Flexible selection:**single file, folder, multiple folders, metadata filter
- **Automatic or manual:**User determines working method
- **Links & Metadata:**All new files → Original files + metadata
- **Scalable:**Hundreds or thousands of files are also possible.

💡 **Note:**

- **Agent3 = Summary agent with original text**
- Access to**Original files, folders, multiple folders, keyword filter**
- **Content copied verbatim.**, followed by an optional summary
- **Users can edit afterwards**→ full control
- Everything **linked + optionally saved**

If you want, I can now**Merge complete simulation for Agent2 & Agent3**, with **Folders, multiple files, multi-criteria extraction and original text import**, so that you can see how**Code, concepts, and summaries processed in parallel** become.

Do you want me to do that?

And

Perfect – then we'll build the**Complete simulation for Agent2 & Agent3**This shows how they would work in practice for an entire project folder or blog folder. All functions are taken into account:

- **Agent 2:**selective extraction of code, concepts, or other types,**one to one**Multiple filters simultaneously, new files per type, linking to original file, metadata, automatic or manual
- **Agent 3:**Access to original files, folders, or metadata,**verbatim copying of the text**, followed by an optional summary, linked, automatic or manual
- **Dashboard & Metadata:**Filterable, linked, traceable

Simulation: Complete Workflow

1️⃣ Starting point: Folder structure

Blog_Projects/

```
├─ Artikel1_Mixed.docx
├─ Article2_Mixed.docx
├─ Projekt1_Mixed.docx
├─ Projekt2_Mixed.docx
```

Example content`Artikel1_Mixed.docx`:

Text: Introduction, project description
Code: Python function for data analysis
Concept: Architecture & Workflow
Table: Budget & Tasks
Image: Diagram

Example content`Projekt1_Mixed.docx`:

Text: Project planning
Code: JavaScript functions
Concept: Strategy paper
Table: Resource plan
Image: Diagram

2️⃣ Agent 2 – Multi-criteria extractor

User action:

- Selection: Folder`Blog Projects`
- Filter:
  - Code → `.code.txt`
  - Concepts →`.concept.docx`
  - The table →`.table.xlsx` (optional)
- Automatic / manual selectable

Simulation of results per file:

original file	Filter	New file	Contents	Link
Artikel1_Mixed.docx	Code	`Article1_Code.code.txt`	Python Function	Original file + metadata
Artikel1_Mixed.docx	concept	`Article1_Concept.concept.docx`	Architecture & Workflow	Original file + metadata

Artikel1_Mixed.docx	Table	`Article1_Table.table.xlsx`	Budget & Tasks	Original file + metadata
Projekt1_Mixed.docx	Code	`Projekt1_Code.code.txt`	JavaScript functions	Original file + metadata
Projekt1_Mixed.docx	concept	`Project1_Concept.docx`	Strategy paper	Original file + metadata
Projekt1_Mixed.docx	Table	`Project1_Table.table.xlsx`	Resource plan	Original file + metadata

Points:

- Each extraction**one to one**, only desired type
- Content will**not mixed**
- **Linking + Metadata**for each new file
- **Folder or multiple folder processing possible**

3️⃣ Agent 3 – Summary Agent with Original Text

User action:

- Selection: Folder`Blog Projects`or files from Agent2
- Filter: Metadata / Keyword filter (e.g. "Architecture")
- Automatic or manual
- **The original text is reproduced verbatim.**
- Optional: Create a summary

Simulation of the results:

New file	Basic files	Contents	Link
`Article1_Architecture_Full.docx`	`Article1_Concept.concept.docx`	Full original text	Original file + metadata
`Article1_Architecture_Summary.docx`	`Article1_Concept.concept.docx`	Summary	Original file + metadata

`Projekt1_Code_Full.docx`	`Projekt1_Code.code.txt`	Full code	Original file + metadata
`Project1_Code_Summary.docx`	`Projekt1_Code.code.txt`	Overview / Analysis of the Code	Original file + metadata

---

4️⃣ Dashboard & Metadata

- All original files + new files from Agent2 & Agent3
- Filterable by:
  - File type (code, concept, table, summary)
  - Metadata / Keywords
- **Links:**Each extracted file → Original file
- **Storage:**The user decides which files are permanently saved.

5️⃣ Advantages of the system

- **Complete control:**User selects file, folder, multiple folders, keyword filter
- **One-to-one extraction:**No summary without user decision
- **Multiple criteria simultaneously:**Code, concepts, tables → separate files
- **Scalable:**from a few files to thousands
- **Links & Metadata:**Everything is traceable, dashboard view
- **Flexible:**manual or automatic, prompts and file types customizable

---

💡 **Note:**

- **Agent2 = selective extractor by type**, multiple criteria simultaneously →**one to one**
- **Agent3 = Summary agent with original text**, optional summary →**The original text remains intact.**
- Everything **linked + optionally saved**
- **The user can edit at any time.**

If you want, I can do another one now.**Create visualization**, the**shows how hundreds or thousands of files can be processed simultaneously by Agent2 & Agent3.**, including dashboard, metadata, links, and storage decisions. Then it becomes directly visible how this works for large projects.

Do you want me to do that?

As text

Here is the complete simulation**as text**, exactly as you need it:

Simulation: Agent2 & Agent3 – Complete Workflow (Text Version)

1. Initial situation: Folder structure

```
Blog_Projects/
├─ Artikel1_Mixed.docx
├─ Article2_Mixed.docx
├─ Projekt1_Mixed.docx
├─ Projekt2_Mixed.docx
```

Example content Article1_Mixed.docx:
Text: Introduction, project description
Code: Python function for data analysis
Concept: Architecture & Workflow
Table: Budget & Tasks
Image: Diagram

Example content Project1_Mixed.docx:
Text: Project planning
Code: JavaScript functions
Concept: Strategy paper
Table: Resource plan
Image: Diagram

---

2. Agent 2 – Multi-criteria extractor

User action:

- Selection: Folder Blog_Projects
- Filter: Code, Concepts, Tables (optional)
- Automatic / manual

Results per file:

Article1_Mixed.docx → Code → Article1_Code.code.txt → Python function → Original file + metadata
Article1_Mixed.docx → Concept → Article1_Concept.docx → Architecture & Workflow → Original file + Metadata
Article1_Mixed.docx → Table → Article1_Tables.table.xlsx → Budget & Tasks → Original file + Metadata

Project1_Mixed.docx → Code → Project1_Code.code.txt → JavaScript functions → Original file + metadata
Project1_Mixed.docx → Concept → Project1_Concept.docx → Strategy Paper → Original File + Metadata
Project1_Mixed.docx → Table → Project1_Tables.table.xlsx → Resource Plan → Original File + Metadata

Key points:

- Each extraction**one to one**
- Content will**not mixed**
- Linking + metadata for each new file
- Processing for a single file, folder, or multiple folders

---

3. Agent3 – Summary agent with original text

User action:

- Selection: File, folder, multiple folders, or metadata
- Automatic or manual
- Copy content verbatim.
- Optional: Create a summary

Results (example):

Article1_Architecture_Full.docx → Basis: Article1_Concept.concept.docx → Full original text → Original file + metadata
Article1_Architecture_Summary.docx → Basis: Article1_Concept.concept.docx → Summary → Original file + Metadata

Project1_Code_Full.docx → Base: Project1_Code.code.txt → Full code → Original file + metadata
Project1_Code_Summary.docx → Base: Project1_Code.code.txt → Overview / Analysis → Original file + Metadata

Important:

- Original text will**fully taken over**
- Users can edit afterwards
- Optionally save new files
- Link to original files + metadata

---

4. Dashboard & Metadata

- All original files + new files from Agent2 & Agent3
- Filterable by: File type (code, concept, table, summary), metadata / keywords
- Each extracted file → original file linked
- Storage: The user decides which files are permanently saved.

---

5. Advantages

- **One-to-one extraction:**no loss of detail
- **Multiple criteria simultaneously:**Code, concepts, tables → separate files
- **Scalable:**single file → folder → hundreds or thousands of files
- **Links & Metadata:**Everything is understandable
- **Flexible:**manual or automatic, file type selectable
- **Agent3 Summary (optional):**The original text remains in its entirety.

💡 **Note:**

- **Agent2 = selective multi-criteria extractor**→ Code, concepts, tables, multiple criteria simultaneously, one-to-one
- **Agent3 = Summary agent with original text**, optional summary → original text remains
- Everything **linked + optionally saved**The user retains full control.

If you want, I can do another one now.**Advanced text simulation for 1000+ files**Do it like Agent2 and Agent3 run simultaneously, extract all types, link them all, and generate metadata. Then you can see directly how the system works.**works on a large scale**.

Do you want me to do that?

And

Okay – then here's the**Advanced text simulation for 1000+ files**How Agent2 and Agent3 work simultaneously, with all types, links, metadata, and optional storage. All as text, step by step.

Simulation: Agent2 & Agent3 for large projects (1000+ files)

1️⃣ Starting point

Folder structure:

```
Blog_Projects/
├─ Project1/
│  ├─ Datei001_Mixed.docx
│  ├─ Datei002_Mixed.docx
│  ├─ …
│  └─ Datei100_Mixed.docx
├─ Project2/
│  ├─ Datei101_Mixed.docx
│  ├─ …
│  └─ Datei200_Mixed.docx
├─ Project3/
│  └─ …
└─ Project10/
   └─ Datei1000_Mixed.docx
```

Example content of a file (`Datei001_Mixed.docx`):

Text: Introduction, project description
Code: Python function
Concept: Architecture & Workflow
Table: Budget & Tasks
Image: Diagram

All 1000 files contain different combinations of text, code, concepts, tables, and images.

2️⃣ Agent 2 – Multi-criteria extractor (large quantity)

User action:

- Selection: multiple folders or entire blog_projects
- Filter:
  - Code → `.code.txt`
  - Concepts →`.concept.docx`
  - The table →`.table.xlsx`
- Automatic / manual

Processing:

1. For each file (e.g.`Datei001_Mixed.docx` until `Datei1000_Mixed.docx`)
2. Apply filter: Code, Concept, Table
3. Each extraction**one to one**
4. Create new files, linked to the original file
5. Save metadata: filename, folder, filter type, date, keywords

Simulation of the results (text example for the first 3 files):

Datei001_Mixed.docx
→ File001_Code.code.txt (Python function) → Original + Metadata
→ File001_Concept.docx (Architecture & Workflow) → Original + Metadata
→ File001_Tables.table.xlsx (Budget & Tasks) → Original + Metadata

Datei002_Mixed.docx
→ File002_Code.code.txt (Python function) → Original + Metadata
→ File002_Concept.docx (Architecture & Workflow) → Original + Metadata
→ File002_Tables.table.xlsx (Budget & Tasks) → Original + Metadata

Datei003_Mixed.docx
→ File003_Code.code.txt (Python function) → Original + Metadata
→ File003_Concept.docx (Architecture & Workflow) → Original + Metadata
→ File003_Tables.table.xlsx (Budget & Tasks) → Original + Metadata
…
Datei1000_Mixed.docx
→ Datei1000_Code.code.txt → Original + Metadaten
→ File1000_Concept.docx → Original + Metadata
→ File1000_Tables.table.xlsx → Original + Metadata

Points:

- All files will be**individually processed**, no mixing
- Linking + Metadata**always received**
- Scaled to**a hundred, a thousand files**

3️⃣ Agent 3 – Summary Agent (Large Quantity)

User action:

- Selection:
  - single file
  - Folder
  - multiple folders
  - Metadata filters (e.g., keyword "architecture")
- Automatic or manual
- Copy content verbatim.
- Optional: Create summary

Simulation of results (text example):

File001_Concept.docx → copied to File001_Architecture_Full.docx → original text complete → original + metadata
File001_Architecture_Summary.docx → Summary → Original + Metadata

File002_Concept.docx → copied to File002_Architecture_Full.docx → original text complete → original + metadata
File002_Architecture_Summary.docx → Summary → Original + Metadata

File003_Concept.docx → copied to File003_Architecture_Full.docx → original text complete → original + metadata
File003_Architecture_Summary.docx → Summary → Original + Metadata
…
File1000_Concept.docx → copied to File1000_Architecture_Full.docx → original text → original + metadata
File1000_Architecture_Summary.docx → Summary → Original + Metadata

Key points:

- Original text**always fully adopted**
- Summary (optional)
- Files linked + metadata saved
- Flexible selection: single file, folder, multiple folders, filters

---

4️⃣ Dashboard & Metadata (large amount)

- **Advertisement:**all original files + all Agent2 & Agent3 files
- **Filterable:**
  - File type (code, concept, table, summary)
  - Metadata / Keyword

- **Links:**Each new file → Original file
- **Storage:**The user decides which files are permanently backed up.

5️⃣ Advantages

- **One-to-one extraction**→ no loss of information
- **Multiple criteria simultaneously**→ Code, concepts, tables → separate files
- **Scalable to thousands of files**
- **Flexible:**manual or automatic
- **Metadata & links**→ all results are verifiable
- **Agent 3**→ Original text + summary optional

💡 **Note:**

- **Agent2 = Multi-criteria extractor**→ all types, one to one, linked
- **Agent3 = Summary Agent**→ Original text remains, optional summary
- **1000+ files possible**Dashboard + metadata → full control

---

If you want, I can do something else next**create a detailed text diagram**, the **shows how each file passes through Agent2 & Agent3**, incl.**Links, metadata, storage**, completely **without visual graphics**, just text structure, so that one can**I can completely understand.** can.

Do you want me to do that?

And

Perfect – then here's the**detailed text diagram**, the **shows how each file passes through Agent2 & Agent3**, incl.**Links, metadata, storage**, completely in text form:

Text diagram: Agent2 & Agent3 workflow for large projects

1️⃣ Starting point

```
Blog_Projects/
├─ Project1/
│   ├─ Datei001_Mixed.docx
│   ├─ Datei002_Mixed.docx
│   └─ …
├─ Project2/
│   └─ …
└─ Project10/
    └─ Datei1000_Mixed.docx
```

Each file contains: text, code, concepts, tables, images

2️⃣ Agent 2 – Multi-criteria extractor

```
[Datei001_Mixed.docx]
  │
  ├─ Filter: Code → Datei001_Code.code.txt
│  └─ Content: Python function
│  └─ Metadata: Filename, Folder, Type=Code, Date, Keywords
│  └─ Link: Original file
  │
├─ Filter: Concept → File001_Concept.concept.docx
│  └─ Content: Architecture & Workflow
│  └─ Metadata: Filename, Folder, Type=Concept, Date, Keywords
│  └─ Link: Original file
  │
→ Filter: Table → File001_Tables.table.xlsx
└─ Contents: Budget & Tasks
└─ Metadata: Filename, Folder, Type=Table, Date, Keywords
└─ Link: Original file
```

- **Manual / automatic:**User decides
- **Folder, multiple folders, or all files:**selectable
- **Result:**New files per type, linked, metadata saved

3️⃣ Agent 3 – Summary Agent with Original Text

```
[Selected file or folder]
  │
├─ Selection: File001_Concept.concept.docx
│  └─ Original text copied verbatim → File001_Architecture_Full.docx
│  └─ Optional: Summary → File001_Architecture_Summary.docx
│  └─ Metadata: Original file, Folder, Type=Concept, Date, Keywords
```

```
│ └─ Link: Original file
 │
├─ Selection: Folder Project1
│ ├─ File002_Concept.concept.docx → File002_Architecture_Full.docx
│ ├─ File002_Architecture_Summary.docx
│ └─ … (for all files in the folder)
 │
└─ Selection: multiple folders or filters by keywords
└─ Files transferred accordingly → new files
└─ Linking + metadata for each file
```

- **The original text is always adopted in its entirety.**
- **Summary optional, only at the user's request**
- **Links + metadata preserved**

4️⃣ Dashboard / Overview (Text structure)

```
Dashboard
├─ Original files
│   ├─ Datei001_Mixed.docx
│   ├─ Datei002_Mixed.docx
│   └─ …
├─ Agent2 files
│ ├─ File001_Code.code.txt → Original file + metadata
│ ├─ File001_Concept.concept.docx → Original file + metadata
│   └─ …
├─ Agent3 files
│ ├─ File001_Architecture_Full.docx → Original file + metadata
│ ├─ File001_Architecture_Summary.docx → Original file + metadata
│   └─ …
├─ Filterable
│ ├─ File type: Code / Concept / Table / Summary
│ ├─ Folder / Project
│ └─ Metadata / Keyword
└─ Storage
├─ Automatic / Manual selection for each new file
└─ All links intact
```

5️⃣ Process for 1000+ files (text overview)

```
[Start: Blog_Projects / ProjectX]
 │
├─ Agent 2: Multi-criteria extraction
│ ├─ Code → new file (.code.txt)
```

```
│  ├─ Concept → new file (.concept.docx)
│  └─ Table → New file (.table.xlsx)
 │
├─ Agent3: Summary Agent
│  ├─ Selection: File / Folder / Multiple Folders / Tag Filter
│  ├─ Original text → new file (_Full.docx)
│  ├─ Optional: Summary → new file (_Summary.docx)
│  └─ Metadata + links → all files
 │
Dashboard: Overview + Filters + Storage
├─ All original files
├─ All Agent2 files
├─ All Agent3 files
├─ Metadata & Keyword Filters
└─ Save automatically / manually
```

✅ Key features

- **Agent 2**Selective extraction (code, concept, tables), one-to-one, automatic or manual, linked + metadata
- **Agent 3**Summaries based on complete original texts, optional, flexible, linked + metadata
- **Scalable**: from 1 to 1000+ files
- **Dashboard**Overview, filterable, metadata, links, storage options

---

If you want, I can do it next.**Simulate a continuous flow text for an entire project folder**, **from file import via Agent2 & Agent3 to the finished knowledge database**, **all steps one after the other**, so that the complete life cycle can be seen.

Do you want me to do that?

And

All right – here is the**Continuous workflow for a complete project folder**, step by step, as text,**from file import to finished knowledge database**, including Agent2 & Agent3, metadata, links and storage decisions.

Complete process: Project folder → Knowledge base

1️⃣ Starting point: Folder structure

```
Blog_Projects/
├─ Project1/
│   ├─ Datei001_Mixed.docx
│   ├─ Datei002_Mixed.docx
│   └─ …
├─ Project2/
│   └─ …
└─ Project10/
    └─ Datei1000_Mixed.docx
```

- Each file contains: text, code, concepts, tables, images
- Folder structure can be arbitrarily deep
- Original files remain unchanged

2️⃣ File import into plugin / knowledge base

1. User selects folder or individual files
2. Plugin copied**all files in the knowledge base**
    - Original files remain on external storage.**unchanged**
    - Metadata automatically captured: filename, folder, date, type
3. Users can optionally add metadata: projects, customer, tags

Imported:
File001_Mixed.docx → Metadata: Folder=Project1, Type=Mixed
File002_Mixed.docx → Metadata: Folder=Project1, Type=Mixed
…
File1000_Mixed.docx → Metadata: Folder=Project10, Type=Mixed

3️⃣ Agent 2 – Multi-criteria extractor

Action: User chooses: entire folder structure or individual folders/files

- Filter: Code, Concepts, Tables
- Optional: Automatic or manual

Example workflow for a file:

Datei001_Mixed.docx
├─ Code → File001_Code.code.txt → Content: Python function → Metadata + Linking

├─ Concept → File001_Concept.concept.docx → Content: Architecture → Metadata + Linking
└─ Table → File001_Tables.table.xlsx → Content: Budget → Metadata + Linking

- Each file will**processed separately**
- Metadata: Type, original file, folder, date, keywords
- Result: new files per type, all linked
- For large quantities (1000+ files), the processing runs**parallel or batchwise**

4️⃣ Agent 3 – Summary Agent

Action: User selects:

- single file
- folder or multiple folders
- or filters by metadata / keywords

Workflow:

1. **Copy original text**→ new file`_Full.docx`
2. Optional: **Create a summary**→ new file`_Summary.docx`
3. Receive all files**Metadata + linking**

File001_Concept.docx → File001_Architecture_Full.docx → Original text
File001_Concept.docx → File001_Architecture_Summary.docx → Summary
File002_Concept.docx → File002_Architecture_Full.docx → Original text
…
File1000_Concept.docx → File1000_Architecture_Full.docx

- Users can specify whether**automatically or manually saved**
- Flexible selection: single file, folder, multiple folders, filters

---

5️⃣ Metadata & Dashboard

- All original files + Agent2 + Agent3 files in**unified dashboard**
- Filterable:
  - File type: Code, Concept, Table, Summary
  - Folder / Project
  - Keywords / Metadata
- Each file is**linked to the original file**
- User decides which files are permanently saved.

```
Dashboard view:
├─ Original files (1000)
├─ Agent2 files (code, concept, table)
├─ Agent3 files (_Full, _Summary)
├─ Metadata & Keyword Filters
└─ Saving: automatic / manual
```

6️⃣ Example of an automated batch process (1000 files)

```
[Start: Blog_Projects]
  │
Agent2 extracts all code, concepts, tables → new files
│  └─ Metadata + Linking
  │
├─ Agent3 takes over original text + optional summary
│  └─ Metadata + Linking
  │
Dashboard: all files can be filtered and saved.
```

- Users can**Edit at any time**
- Each file remains**traceable one-to-one**
- Linking guarantees: no file is lost, everything is referenceable.

7️⃣ Advantages

- **Scalable:**from 1 to 1000+ files
- **One-to-one extraction:**no data loss
- **Flexible:**Agent2 & Agent3 manually or automatically
- **Dashboard + Metadata:**Everything is filterable and traceable.
- **Links:**Original files and new files are always connected
- **Post-processing:**The user decides what is saved or further processed.

💡 **Note:**

- **Agent2 = selective extractor**→ Code, concept, table, one-to-one, automatic or manual
- **Agent3 = Summary Agent**→ Original text + optional summary, flexible, linked
- **Dashboard & Metadata = Control + Overview**, also for 1000+ files

---

If you want, I can do another one.**Show complete step-by-step text simulation of a single file by both agents**, **including all metadata, links and storage**, so that one can see,**how a file is processed from beginning to end**.

Do you want me to do that?

Agent 1 is still missing. After the import, where all the files are located, the user can start Agent 1 automatically. It searches for 10 or 100 keywords automatically or manually and adds them to a knowledge base. This is what's missing.

Okay – then I'll add...**Agent 1 in the process**, immediately after import, so that you can**automatic or manual keyword analysis**and the results are incorporated into the knowledge database. Everything step by step, as text:

Complete process including Agent 1

1️⃣ File import

```
Blog_Projects/
├─ Project1/
│   ├─ Datei001_Mixed.docx
│   ├─ Datei002_Mixed.docx
│   └─ …
└─ Project10/
    └─ Datei1000_Mixed.docx
```

- All files will be**copied**into the plugin database
- Original files**remain unchanged**
- Automatic metadata: filename, folder, type, date
- Users can**additional metadata**Enter: Project, Customer, Tags

---

2️⃣ Agent 1 – Keyword Search / Metadata

Starting point:after file import

User options:

- Selection:
    - Single file

    - Folder
    - Several folders
    - All files
- Number of keywords: 10, 50, 100… freely selectable
- Automatic or manual

Processing:

1. Agent1 reads the selected files
2. Analyze the content
3. Extracts keywords
4. It saves as**Metadata in the knowledge database**
5. Link the keywords to their respective original files.

Example text structure:

Datei001_Mixed.docx
Keywords (10): Project, Architecture, Workflow, Budget, Code, Table, Planning, Concept, Risk, Timeline
├─ Metadata: Filename, Folder, Type=Mixed, Date
└─ Link: Original file

Datei002_Mixed.docx
├─ Keywords (100): …
├─ Metadata: …
└─ Link: Original file

… bis Datei1000_Mixed.docx

- **Results**flow into the knowledge database
- Users can at any time**Start manually**, if he only wants to analyze certain files

---

3️⃣ Agent 2 – Multi-criteria extractor

- Filter: Code, Concepts, Tables
- One-to-one extraction to new files
- Metadata + link to original file
- Automatic or manual
- Selection: File / Folder / Multiple Folders / Tag Filter

4️⃣ Agent 3 – Summary Agent

- Selection: File / Folder / Multiple Folders / Tag Filter
- Original text**adopted one-to-one** → `_Full.docx`
- Optional: Summary →`_Summary.docx`
- Metadata + linking
- Automatic or manual

5️⃣ Dashboard & Metadata

```
Dashboard Overview
├─ Original files (1000)
├─ Agent1 Keywords / Metadata
├─ Agent2 files (code, concept, table)
├─ Agent3 files (_Full, _Summary)
├─ Filterable: File type, folder, tag, project
└─ Saving: automatic / manual
```

- All files linked
- All metadata, including keywords, is present.
- Users can select, filter, and save files at any time.

6️⃣ Workflow for 1000+ files (text diagram)

```
[Start: Dateiimport]
  │
├─ Agent1: Keyword Analysis
│  ├─ Automatic or manual
│  ├─ 10–100 keywords per file
│  └─ Storage in knowledge base + linking
  │
├─ Agent2: Extract code/concept/table
│  ├─ One-to-one → new files
│  └─ Metadata + Linking
  │
├─ Agent3: Summary / Original text
│  ├─ Original text one-to-one → new file
│  ├─ Optional Summary → New File
│  └─ Metadata + Linking
  │
  └─ Dashboard
├─ All original files
├─ All Agent1–3 files
├─ Filterable (Type, Folder, Tag, Project)
└─ Saving: automatic / manual
```

💡 **Note:**

- **Agent 1:**Keywords → Metadata → Knowledge Base
- **Agent 2:**Selective extraction (code, concepts, tables)
- **Agent 3:**Original text + optional summary
- **Dashboard:**Full overview, filters, storage, linking
- **Flexible & scalable:**for thousands of files

If you want, I can do it next.**Show a complete example with one file through all three agents.**, including**how metadata, keywords and links look**, so that one can**complete lifecycle of a file**can understand.

Do you want me to do that?

So, AG3 has another function. Now you can also use it to combine an original file with templates and save it in a new file, including metadata. And we've already discussed the second function: you can select multiple files, folders, and metadata, which it then transfers one-to-one into a new file, and only then do you start with the summary. And you can also use templates. AG3 generally allows you to work with templates.

Perfect – then we'll expand**Agent3 to use the template function**, so that all existing workflows are retained, but now also**New files can be created using a template.**This applies to single files as well as multiple files or entire folders. All in text format:

Agent3 – Summary agent with template function

1️⃣ Basic functions

1. **Edit single file**
    - Copy the original text verbatim.
    - Optional: Create a summary
    - **Apply template**→ new file`_Template.docx`
    - Metadata and links are retained.
    - Save automatically or manually

```
Datei001_Mixed.docx
├─ Original text → File001_Full.docx
├─ Template applied → File001_Template.docx
├─ Optional Summary → File001_Summary.docx
├─ Metadata: Filename, Folder, Type, Date, Keywords
└─ Link: Original file
```

---

2. **Select multiple files / folders / metadata**
    - Copy original texts verbatim.
    - Create a new file, only then start the summary.
    - **Templates can be used flexibly**
    - All metadata and links are retained.

```
Project1 Folder
├─ Datei001_Mixed.docx → Datei001_Full.docx
│ → Template applied → File001_Template.docx
├─ Datei002_Mixed.docx → Datei002_Full.docx
│ → Template applied → File002_Template.docx
└─ …
```

2️⃣ Template functionality

- Template can**Text format, headers, structure, fields for metadata** contain
- Will be on:
    - Single file
    - Multiple files
    - Folder
    - Metadata filter applied
- Output: new file containing original text + template + optional summary
- Storage: selectable automatically or manually

3️⃣ Combination with Agent 1 & Agent 2

1. **Agent 1:**Extract keywords → Metadata in knowledge database
2. **Agent 2:**Extract code / concepts / tables → new files, linked + metadata
3. **Agent 3:**Copy original text + summary + template → new linked file + metadata

Workflow:
Import file → Agent1 keywords → Agent2 extracted → Agent3 template/original/summary

---

4️⃣ Dashboard / Overview

- All files clearly arranged: Original, Agent1, Agent2, Agent3
- Filterable by type, folder, keyword, project
- Automatic or manual saving
- **Template files clearly marked**
- Fully linked and tagged with metadata

```
Dashboard
├─ Original files
├─ Agent1 Keywords / Metadata
├─ Agent2 files (code, concept, table)
├─ Agent3 files (_Full, _Summary, _Template)
├─ Filterable: Type, Folder, Tag, Project
└─ Saving: automatic / manual
```

💡 **Key point Agent3 + Templates:**

- Single file → Original text → Apply template → Summary (optional)
- Multiple files/folders → Original texts → Template → Summary (optional)
- Always retain metadata and links.
- Can be saved automatically or manually
- Fully combinable with Agent1 (keywords) & Agent2 (code/concepts)

If you want, I can do it next.**Show a complete simulation with one file through all three agents**, incl.**Keywords, code extraction, summary and template**, so that one**the complete lifecycle of a file**as seen in an example.

Do you want me to do that?

It must be optional whether to import the original data. This must be selectable for one file or multiple files. If multiple files and metadata are selected, it must be possible to choose whether to import everything exactly as is, or to forgo it and only want the summary using a template.

Okay – then let's summarize.**Agent3 is completely flexible**together and expand it by**this new selection option**:

Agent3 – Summary agent with flexible original text option and templates

---

1️⃣ User choices

Option 1: Single file

- Copy original text: yes / no
- Create summary: yes / no
- Apply template: yes / no
- Storage: automatic / manual

Option 2: Multiple files / folders / metadata filter

- Copy original text: yes / no (one-to-one into new file)
- Create summary: yes / no
- Apply template: yes / no
- Storage: automatic / manual

```
[Single file or folder selected]
├─ Copy original text → File_Full.docx (optional)
├─ Summary → File_Summary.docx (optional)
├─ Apply template → File_Template.docx (optional)
├─ Metadata: Filename, Folder, Type, Date, Keywords
└─ Link: Original file
```

2️⃣ Flexible workflow with multiple files

- Users can decide**Summary only with template**to create without copying the original texts
- Or **Copy all original texts verbatim.**and then create a summary + template
- Selection is made**per file or via group/folder selection**

```
Folder Project1
├─ Datei001_Mixed.docx
│  ├─ Copy original text: yes
│  ├─ Template: yes
│  └─ Summary: no
├─ Datei002_Mixed.docx
│  ├─ Copy original text: no
│  ├─ Template: yes
```

```
│  └─ Summary: yes
└─ Datei003_Mixed.docx
├─ Copy original text: yes
├─ Template: no
Summary: yes
```

3️⃣ Integration with Agent1 & Agent2

- **Agent 1**Keyword analysis → Metadata in knowledge database
- **Agent 2**Code / concept / table → new files, one-to-one, linked + metadata
- **Agent 3**Summary + Template → optionally include original text → linked + metadata

```
Workflow:
[File import]
  │
├─ Agent1: Keyword analysis (automatic or manual)
  │
├─ Agent2: Extract code/concept/table
  │
Agent 3: Original texts (optional) + Summary + Template
```

4️⃣ Dashboard & Storage

- All files clearly arranged in the dashboard
- Filterable: File type, folder, keyword, project
- Display all options, which files have retained the original text, which template or summary to use
- Storage selectable: automatic or manual

```
Dashboard
├─ Original files
├─ Agent1 Keywords / Metadata
├─ Agent2 files (code, concept, table)
├─ Agent3 files (_Full, _Summary, _Template)
├─ Filterable: Type, Folder, Tag, Project
└─ Saving: automatic / manual
```

💡 **Key points Agent3 Flexibility**

1. **Originaltexte optional**– per file or group
2. **Summary + template selectable separately**
3. **Scalable**– single file, folder, multiple folders, metadata filter
4. **Links and metadata**remain always

5. **Automatic or manual** save

If you want, I can now**walk through a complete example**, **a file through Agent 1, 2 and 3**, including:

- Keywords (Agent1)
- Code/Concept Extraction (Agent2)
- Summary + template with original text selectable (Agent3)

Then you see**the complete lifecycle of a file**clear and step by step.

Do you want me to do that?

Then, for Agent 3, we don't need two options; it's enough to simply say: single file, multiple files, folder, metadata, filter, and then you just need to select the criteria. The four options are: original text, summary, template, and saving—all yes or no.

Exactly – then Agent3 will**much simpler and more consistent**:

Agent3 – simplified selection structure

1️⃣ Selection area

- **Target objects:**
  - Single file
  - Multiple files
  - Folder
  - Metadata filter
- **Criteria per selection object:**
  - **Copy original text:** Yes No
  - **Create a summary:** Yes No
  - **Apply template:** Yes No
  - **Storage:**automatic / manual

```
[Selection: Folder Project1]
├─ Copy original text: Yes
├─ Summary: No
├─ Template: Yes
└─ Storage: Manual
```

- User selects**simply target object + criteria**Agent3 then executes the workflow automatically.

2️⃣ Workflow

1. User selects target object
2. Agent3 checks criteria: Original text / Summary / Template / Storage
3. Agent3 generates files accordingly
   - `_Full.docx`→ Original text (if yes)
   - `_Summary.docx`→ Summary (if yes)
   - `_Template.docx`→ Template applied (if yes)
4. Receive all files**Metadata + linking**
5. Storage is performed according to the selection.

3️⃣ Integration with Agent1 & Agent2

Workflow:

```
[File import]
 │
├─ Agent1: Keyword analysis (automatic or manual)
 │
├─ Agent2: Extract code/concept/table
 │
└─ Agent3: Original text / Summary / Template → optional saving
```

- A consistent approach to**individual files, entire folders, or filtered groups**
- **All criteria can be selected centrally.**, no separate selection per function is needed anymore

---

💡 **Note:**

- **Agent3 = flexible tool**
- **Select a target object**→ then**Original text, summary, template, storage** Yes No
- Automatic or manual → all in one workflow

If you want, I can do it next.**The entire process from Agent1 → Agent2 → Agent3 as a text diagram with a sample file** represent, **including keywords, code/concepts, summary, template and metadata**, so that one can**complete life cycle** sees.

Do you want me to do that?

Of course, all of this needs to be linked to the knowledge base in Agent 3. There you go.

Perfect – then let's now fully summarize Agent3,**including the link to the knowledge database**:

Agent3 – Summary agent with metadata & knowledge base linking

---

1️⃣ Selection area

- **Target objects:**
  - Single file
  - Multiple files
  - Folder
  - Metadata filter
- **Criteria per selection object:**
  - **Copy original text:** Yes No
  - **Create a summary:** Yes No
  - **Apply template:** Yes No
  - **Storage:**automatic / manual
- **Link:**All generated files are automatically**to the knowledge database**linked, including original file, summary, template and associated metadata

```
[Selection: Folder Project1]
├─ Copy original text: Yes → File001_Full.docx → Knowledge base linked
├─ Summary: Yes → File001_Summary.docx → Knowledge base linked
├─ Template: Yes → File001_Template.docx → Knowledge base linked
└─ Storage: Manual
```

2️⃣ Workflow

1. User selects target object
2. Select criteria: Original text / Summary / Template / Saving
3. Agent3 creates files accordingly
4. **Automatic linking of all generated files to the knowledge base**
5. All metadata is retained, supplemented with a link to:
   - original file

    - Folder / Project
    - Keywords from Agent 1
    - Code/Concepts from Agent2

3️⃣ Integration with Agent1 & Agent2

Workflow:
[File import]
│
├─ Agent1: Keyword analysis → Metadata → Knowledge base
│
├─ Agent2: Extract code/concept/table → new files → linked → metadata
│
Agent3: Original text / Summary / Template → New files → Linked → Metadata

- Each generated file is**always linked to the knowledge database**
- All agents are working**compatible together**
- Filterable: Folder, File Type, Tag, Project, Metadata

💡 **Note:**

- **Agent3 = flexible tool with full linking**
- Selection options: single file, multiple files, folder, metadata filter
- Criteria: Original text, summary, template, storage
- All generated files automatically**Linked to the knowledge database**
- Compatible with Agent1 & Agent2

If you want, I can now**visualize the entire lifecycle of a sample file**: from Import → Agent1 Keywords → Agent2 Code/Concepts → Agent3 Original text + Summary + Template → all linked in the knowledge base.

Do you want me to do that?

It would be nice if the document number generator could also be integrated here. Someone is working on a project. Projects 1 and 2 combine to form Project 3. And of course, this needs to be tracked using document numbers. So, this Plugin 2 should also allow the use of Plugin 1's document number generator.

Okay – then we'll integrate it.**the document number generator from Plugin1 directly into Plugin2 / Agent Workflow**, so that one**Track projects, files, summaries, and new results using document numbers.**It can. Here's how the process is described in writing:

Integration of document number generator (Plugin1) in knowledge database / Agent3

1️⃣ Goal

- Each file/result in Plugin2 (knowledge base) can optionally have a**Document number from Plugin1** receive
- Document numbers enable**Tracking of projects, sub-projects and summaries**
- Also **Combinations of projects**can be traced

2️⃣ Workflow

1. **Select file/folder**(Single file, multiple files, folders, metadata filters)
2. **Generate receipt number:**
    - User clicks on "Generate receipt number"
    - Automatic or manual
    - Optional: Project-dependent document numbers (e.g., from Project1 + Project2 → Project3)
3. The document number will be**stored in the file, metadata and knowledge base**
4. All agents (1–3) continue to work normally, receipt number remains the same.**linked**

3️⃣ Combination with agents

- **Agent 1:**Extract keywords → Metadata → Knowledge base
    - Document number available as metadata
- **Agent 2:**Extract code/concepts → new file → metadata + document number + link
- **Agent 3:**Original text / summary / template → new file → document number + metadata + knowledge base link

Example:
Project 3 (created from Project 1 + Project 2)
├─ File001_Full.docx → Document number: P3-0001 → Knowledge base linked
├─ File001_Summary.docx → Document number: P3-0001-S → Knowledge base linked

```
├─ File001_Template.docx → Document number: P3-0001-T → Knowledge base linked
└─ Metadata: Folder, tags, code, project assignment
```

4️⃣ Advantages

- Projects and results are**Completely traceable**
- Document numbers allow**Tracking of sub-projects, aggregations, and new project results**
- All agents use the same**linked document numbers**, resulting in a consistent data basis

---

💡 **Note:**

- **Plugin1 + Plugin2 = full traceability**
- Each file, each block of code, each summary can carry a document number.
- Document numbers are**always linked to metadata and knowledge database**
- Projects can be combined and new document numbers can be created automatically.

If you want, I can now**play through a complete example**, as from**Project 1 + Project 2 → Project 3**Loop through files, code, summaries, templates, and document numbers across all agents –**including full linking in the knowledge database**.

Do you want me to do that?

I then open Project 2, which contains Projects 3 and 4. At some point, I linked something from another concept, and so on. And when I then open Project 1, I want to see everything that's linked to it.

Exactly – then we need a**project-based linking system in the knowledge database**, the **Document numbers and hierarchies**Fully depicted. Everything presented in text:

Project hierarchy with document numbers in Plugin2 / Knowledge base

---

1️⃣ Structure

- **Projects:**Main categories (e.g. Project1)
- **Subprojects:**automatically linked (e.g. Project3, Project4 → created from Project1 & 2)
- **Files / Results:**
    - Original files
    - Summaries
    - Templates
    - Code / Concepts
- **Receipt number:**
    - Each file/result receives a document number.
    - Linked to the respective project / subproject

```
Project 1 (P1)
├─ Project 3 (P3) ← from P1 + P2
│  ├─ Datei001_Full.docx → Document number: P3-0001
│  ├─ Datei001_Summary.docx → Document number: P3-0001-S
│  ├─ Datei001_Template.docx → Document number: P3-0001-T
│  └─ Code_001.py → Document number: P3-0001-C
├─ Project 4 (P4) ← Subproject P1
│  ├─ Datei002_Full.docx → Document number: P4-0001
│  └─ File002_Summary.docx → Document number: P4-0001-S
└─ Files / Results directly from Project 1
├─ Datei003_Full.docx → Document number: P1-0005
└─ Datei003_Summary.docx → Document number: P1-0005-S
```

2️⃣ Functions

1. **View project → show entire hierarchy**
    - Main project + sub-projects
    - All related files, summaries, code, templates
    - Document numbers visible and linked
2. **Map connections:**
    - Subprojects arising from multiple projects are automatically assigned.
    - Each file/result retains the document numbers of the original projects → traceability
3. **Knowledge Base Integration:**
    - All files, metadata, document numbers and links centrally located
    - Filterable by project, subproject, file, document number, keyword

---

3️⃣ Advantages

- **Full tracking:**You can see which results come from which projects.

- **Cross-project analyses:**e.g., a summary of all files that originated from Project 1, including subprojects.
- **Document numbers as a continuous reference**
- **Automatic hierarchy maintenance:**No manual linking required

---

💡 **Note:**

- **Project hierarchy + document numbers = seamless traceability**
- Opening a project displays everything within it: subprojects, files, code, summaries, templates.
- All elements are**linked and filterable in the knowledge database**

If you want, I can**Now create a complete example**, How **Project1 is called**and all subprojects, files, summaries, code, templates, document numbers and metadata**completely flow through agents 1-3**.

Do you want me to do that?

So, I have a document with a reference number. I want to trace that. I want to see which other documents it has been incorporated into. For example, it was created as a summary, and that summary was then incorporated into Project 1, and that summary also formed the basis for Project 2. So, in a matrix organization, I want to see what happened to the reference number, where it was incorporated, and where the concepts are located.

Perfect – then we need**a document number-centered traceability in matrix form**Everything will be**document-centric**, not just project-centric, so that one**Track every document number across all projects, summaries, code extracts, and templates.**can. In textual terms, this can be described as follows:

Matrix organization of document numbers

1️⃣ Concept

- Each file receives a**unique receipt number**
- **Use in multiple projects / summaries / code files**is automatically detected

- Representation as**Matrix**:
    - Lines = document numbers / original files
    - Columns = Projects, Subprojects, Summaries, Templates, Code Files
    - Entry = Usage / Linking

2️⃣ Example text matrix

Document number → P1-0001 (Original file)

```
┌──────────────┬──────────────┬──────────────┬──────────────┬──────────────┐
│ Project 1 │ Project 2 │ Project 3 │ Summary │ Code Extracts │
├──────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ Yes │ Yes │ Yes │ Yes (S1) │ Yes (C1) │
└──────────────┴──────────────┴──────────────┴──────────────┴──────────────┘
```

Document number → P2-0005 (Summary from P1-0001)

```
┌──────────────┬──────────────┬──────────────┬──────────────┬──────────────┐
│ Project 1 │ Project 2 │ Project 3 │ Summary │ Code Extracts │
├──────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ Yes │ Yes │ No │ Yes (S2) │ No │
└──────────────┴──────────────┴──────────────┴──────────────┴──────────────┘
```

- **P1-0001=** Original file
- **S1**= Summary,**used in Project 3**
- **C1=** Code extract from Agent2, linked to all projects where it is used
- **P2-0005=** new summary from S1, used in Project 2

3️⃣ Functions

1. **View document number → Display matrix**
    - Shows **all projects, summaries, code files, templates**, in which the file was used
2. **Matrix navigable by click**
    - Clicking on the entry → opens the linked file
3. **Automatically updated**
    - Each agent (1–3) automatically enters new results into the matrix.
4. **Filterable by project, type, document number, keyword**
5. **Full traceability**

    - Origin → Interim results → New projects

4️⃣ Advantages

- **Every file is visible in all projects and subprojects.**
- **Combination of original file, summary, code and templates**
- **Matrix = "Document number overview" for project managers and teams**
- **Easy navigation through projects and files**

💡 **Note:**

- **Matrix organization = document number-centric traceability**
- Shows **Transforming a file across projects, summaries, and code**
- **Automatically maintained**by integrating all agents and document numbers

---

If you want, I can**Now build a complete practical example as a text diagram.**, How:

- Original file with document number
- Used in multiple projects and subprojects
- Summaries created
- Code extracted
- Everything in the matrix is linked

So you see**exactly how everything is connected and traceable**.

Do you want me to do that?

Now there's someone who needs not only a matrix organization, document number, and project, but also another, extremely important criterion for sorting everything. They should be able to do that.

Okay – then we'll expand the system to include**flexible additional criteria**, so that the**Matrix organization is not just about projects and document numbers.**, but also **any other metadata**can take this into account. This is what it looks like in text form:

Matrix organization with flexible criteria

---

1 Concept

- Each file / each result receives:
  - **Document number**
  - **Project / Subproject**
  - **Agent result**(Original, Summary, Code, Template)
  - **Additional metadata**, freely definable (e.g. category, responsible person, status, concept type)
- The**Matrix** becomes **dynamically sortable according to criteria**:
  - Lines = document numbers / original files
  - Columns = Projects, Subprojects, Agent Results, Additional Criteria

---

2 Example text matrix with additional criterion

Document number → P1-0001 (Original file)

Project 1	Project 2	Project 3	Summary	Code Extract	Responsible Person
Yes	Yes	Yes	Yes (S1)	Yes (C1)	Max Mustermann

Document number → P2-0005 (Summary from P1-0001)

Project 1	Project 2	Project 3	Summary	Code Extract	Responsible Person
Yes	Yes	No	Yes (S2)	No	Lisa Example

- **Responsible**= additional sorting/filter criterion
- Other examples: Status (in progress / completed), concept type, team, priority, keyword

---

3 Functions

1. **Matrix dynamically sortable**
    - The user selects the criteria by which they want to filter or sort (e.g., project, document number, responsible person, status).
2. **Matrix navigable by click**
    - Clicking on an entry → opens the linked file / summary / code
3. **Automatically maintained**
    - Any change to agent results or metadata is updated directly in the matrix.
4. **Integration of all agents (1–3)**
    - Keywords, code, summaries, templates, document numbers
5. **Flexible metadata extension**
    - New criteria can be added at any time.

4️⃣ Advantages

- Matrix will**not rigid**, rather **adaptable to individual needs**
- Anyone can**any additional criteria**Use to sort files and results
- Document numbers, projects, subprojects, and agent results remain**fully traceable**

💡 **Note:**

- **Matrix + document number + projects + flexible criteria = universal traceability system**
- Filtering and sorting options according to every relevant criterion
- **All agent results are automatically linked in the knowledge base.**

If you want, I can now**build a complete example matrix**, in the **Document numbers, projects, agent results and several additional criteria**be displayed simultaneously **–including navigation from original file → summaries → subprojects → new projects**.

Do you want me to do that?

Three-dimensional searching isn't possible. If I want to search, I'd have to create a document number for each project and then create the individual subprojects. Only then can I actually search. If I create two projects, I have two systems. But that might be useful if you need document numbers, for example, if you have contracts and multiple employees can search. So you could have different project types or different columns, like one for contracts and another for projects. And if you have a login screen for the team, people can scroll down and find everything using the top-level document number.

Exactly – then we move away from a “real” three-dimensional matrix, but instead use**a hierarchical, project- and document-number-based system**, which several**Project type /**

categorysupported and thereby**remains fully searchable**Everything can be represented in text as follows:

Hierarchical document number and project system with team access

---

1️⃣ Structure

- **Top level:**Project type / category
  - e.g. “contracts”, “projects”, “research”, “concept papers”
- **Document numbers at the overarching project level:**
  - Each top-level unit receives a unique document number.
- **Subprojects / Subunits:**
  - Created automatically or manually
  - Each receives one**own document number**, linked to the overarching project number
- **Documents / Results:**
  - Original files, summaries, code, templates
  - Linked via document numbers and project type

```
Category: Projects
├─ Project 1 (Document number: P1)
│  ├─ Subproject 1a (P1-01)
│  │  ├─ Datei001.docx → Document number: P1-01-0001
│  │  └─ Summary001.docx → Document number: P1-01-0001-S
│  └─ Subproject 1b (P1-02)
│  └─ Datei002.docx → Document number: P1-02-0001
└─ Project 2 (Document number: P2)
├─ Subproject 2a (P2-01)
│  └─ Datei003.docx → Document number: P2-01-0001
Subproject 2b (P2-02)
└─ Datei004.docx → Document number: P2-02-0001

Category: Contracts
├─ Contract 1 (V1)
│  └─ Subcontract 1a (V1-01)
│  └─ Datei001.pdf → Document number: V1-01-0001
└─ Contract 2 (V2)
   └─ Untervertrag 2a (V2-01)
└─ Datei002.pdf → Document number: V2-01-0001
```

2️⃣ Functions

1. **Searchable by:**
   - Document number (parent project, subproject, file)
   - Project type / category
   - Metadata (keyword, status, responsible party, code/concept)
2. **Navigation:**
   - Top document number → all subprojects → all files / results
   - Clicking on a document → shows all linked projects / subprojects
3. **Team access:**
   - Dashboard: Login → all accessible project types
   - Filter by category, project, subproject, document number
4. **Linking with agents 1–3:**
   - Each file can contain keywords, code, summary, template.
   - Results automatically linked with document number and project hierarchy

---

3️⃣ Advantages

- **Clear hierarchy:**Main projects, sub-projects, documents
- **Flexible for multiple project types/categories**(Contracts, research, projects)
- **Team player:**Everyone can search by document number and category.
- **Traceability:**Each file → all subprojects → all parent projects
- **Compatible with agents and knowledge base:**Automatic tracking of keywords, code, and summaries

---

💡 **Note:**

- **No real 3D matrix needed:**Hierarchy + document numbers + project types replace the third dimension
- **Top document number = entry point**for navigation and search
- **Each file / summary / code → linked → knowledge base + project**

If you want, I can**Now build a complete example dashboard**, like a team**It can navigate via parent projects, sub-projects, document numbers, categories, and metadata.**, incl.**Agent results, summaries, and code links**– so that you can immediately see what flowed where.

Do you want me to do that?

You can enter document numbers at any level, including under projects. It doesn't have to be at the top level. The list must always be sorted by document number, and then the information listed further down must be displayed.

Perfect – then we're talking about a**flexible, recursive document numbering system**, in which **Each level can receive document numbers**and the structure**searched and displayed from top to bottom**This can be done. The text reads as follows:

Recursive Document Number System

1️⃣ Principle

- **Each level**can have its own document number:
  - Master project
  - Subproject
  - File / Result
- **Hierarchy is dynamic:**
  - Searching for a document number shows**everything below and all linked content**
  - Optional: Search also shows,**where this document number is integrated from a higher level**
- **Integration into knowledge database:**
  - Every file, summary, code, template is linked.
  - All metadata (keyword, status, responsible party) will be transferred.

2️⃣ Example (Text)

```
Project 1 (P1)
├─ Subproject 1a (P1-01)
│  ├─ Datei001.docx → Document number: P1-01-0001
│  ├─ Summary001.docx → Document number: P1-01-0001-S
│  └─ Code_001.py → Document number: P1-01-0001-C
├─ Subproject 1b (P1-02)
│  ├─ Datei002.docx → Document number: P1-02-0001
│  └─ Konzept002.docx → Document number: P1-02-0001-K
└─ Datei003.docx → Document number: P1-0005

Project 2 (P2)
├─ Subproject 2a (P2-01)
```

```
│  └─ Datei004.docx → Document number: P2-01-0001
└─ Subproject 2b (P2-02)
└─ Datei005.docx → Document number: P2-02-0001
```

- **P1-01-0001**→ can exist as a file or summary in subproject 1a
- **P1-0005**→ is stored directly under Project 1, no subproject is needed
- **Search function:**
  - Search for P1-01-0001 → shows everything under subproject 1a
  - Searching for P1 → shows all subprojects, all files within them

---

3️⃣ Functions

1. **Enter document number at each level**
2. **Search:**
   - By document number → shows everything below
   - Optional: also shows where the document number is embedded at a higher level.
3. **Team-Dashboard:**
   - Filter by project, subproject, category, document number, keyword
4. **Automatic linking:**
   - Agents 1–3 Results → Keywords, Code, Summaries
   - All linked files are automatically added to the knowledge base.
5. **Manually / Automatically combinable:**
   - New subprojects, files, or summaries can be numbered automatically or manually.

4️⃣ Advantages

- **Maximum flexibility:**Receipt numbers everywhere
- **Full traceability:**regardless of whether it's a subproject or a file
- **Matrix-like overview without a true 3D structure**
- **Easy integration into team dashboard**

💡 **Note:**

- **Document number = universal identification**
- **Each level can be numbered.**→ Searching by document number shows everything below it
- **All agent results and metadata remain linked.**

If you want, I can now**build a complete example**, how to **Numbers a file at any level, searches the hierarchy, displays subprojects, summaries, code, and all agent results.**, so that you can see immediately,**where this document number is used**.

Do you want me to do that?

The option to search for the document number from a higher level is also included.

Exactly – then we add**an optional "upward search"**in addition, which checks,**whether a document number is integrated into higher levels**, without it having to happen automatically. This can be expressed in text as follows:

Recursive document numbering system with optional upward search

---

1️⃣ Principle

- Each level can**a separate document number**have: project, subproject, file, result.
- **Searching for a document number**displays all content by default.**including**.
- **Optional:**Activating "Upward Search" → shows**higher levels**, in which the document number is included.
- All results will be**linked in the knowledge database**, including agent results, metadata, code, and summaries.

2️⃣ Example (Text)

```
Project 1 (P1)
├─ Subproject 1a (P1-01)
│  ├─ Datei001.docx → Document number: P1-01-0001
│  ├─ Summary001.docx → Document number: P1-01-0001-S
│  └─ Code_001.py → Document number: P1-01-0001-C
├─ Subproject 1b (P1-02)
│  ├─ Datei002.docx → Document number: P1-02-0001
│  └─ Konzept002.docx → Document number: P1-02-0001-K
└─ Datei003.docx → Document number: P1-0005
```

Search by document number:P1-01-0001

- **Standard search (below):**
  - Datei001.docx
  - Summary001.docx
  - Code_001.py
- **Optional upward search (enabled at the top):**
  - Subproject 1a (P1-01)
  - Project 1 (P1)

3️⃣ Functions

1. **Document number at each level:**Project, subproject, file, result
2. **Optional upward search:**
   - Switchable on/off
   - Shows in which higher-level projects / sub-projects the document number is included.
3. **Filterable by:**Project, subproject, category, keyword, status, responsible party
4. **Integration of all agent results:**
   - Keywords, code snippets, summaries, templates
   - Everything is linked to the original files and subprojects.
5. **Team-Dashboard:**
   - Display in hierarchical or flat list format, optionally with upward search.

4️⃣ Advantages

- **Full traceability:**top → bottom, bottom → top
- **Flexibility:**Upward search optional, no unnecessary results
- **Team-friendly:**Anyone can specifically search for receipt numbers and see where they appear.
- **Compatible with all agent functions:**Search, code extraction, summarization

---

💡 **Note:**

- **Document number = universal ID**
- **Optional: Upward search enabled → shows parent levels**
- **All agent results + metadata = fully linked**

If you want, I can**Now build a complete example**, like a**The document number is located in a project, subproject, or file.**, as the **Optional upward search works** and how **All agent results are visible and linked in the knowledge base.** are.

Do you want me to do that?

Here, not only does a document receive a document number, but also categories such as projects, such as contracts, receive a document number, for example, contracts 2026.

Exactly – in this system, you can**Not only individual documents, but also entire categories or overarching projects receive document numbers.**For example, a category "Contracts 2026" could have the document number**V2026**received, a subproject "Project X" the document number**P2026-01**, and individual documents within them each have their own document numbers, e.g.**P2026-01-0001**.

The text reads like this:

Example: Document numbers for categories, projects, and documents

```
Category: Contracts 2026 → Document number: V2026
├─ Contract 1 → Document number: V2026-01
│  ├─ Datei001.pdf → Document number: V2026-01-0001
│  └─ Summary001.pdf → Document number: V2026-01-0001-S
├─ Contract 2 → Document number: V2026-02
│  └─ Datei002.pdf → Document number: V2026-02-0001

Category: Projects → Document number: P2026
├─ Project 1 → Document number: P2026-01
│  ├─ Subproject 1a → Document number: P2026-01-01
│  │  └─ Datei003.docx → Document number: P2026-01-01-0001
│  └─ Subproject 1b → Document number: P2026-01-02
│  └─ Datei004.docx → Document number: P2026-01-02-0001
├─ Project 2 → Document number: P2026-02
│  └─ Datei005.docx → Document number: P2026-02-0001
```

Important principle

1. **Document numbers at each level:**Category → Project → Subproject → Document
2. **Optional: Upward search:**

    - Searching for a document reference number automatically shows in which subproject, project, or category it is contained.
3. **Flexible:**
    - Categories, projects, or individual documents can have their own document numbers.
    - Any additional criteria (e.g., year, responsible person, status) can be added.
4. **Integration into knowledge database:**
    - All documents, summaries, and code snippets remain linked.
    - Each document number is unique and traceable.

💡 **Note:**

- **The voucher numbers are universal:**for categories, projects and documents
- **Hierarchy + optional upward search = full traceability**
- **Enables team search and navigation through parent and sub-projects.**

If you want, I can now**build a complete example**, in which **Categories, projects, subprojects and documents each have their own document numbers.**, including**Upward and downward search, agent results, summaries and codes**, so that you can see immediately,**which is integrated where**.

Do you want me to do that?

And

Perfect – then here's a**Complete example of a hierarchical document numbering system**, the **Categories, projects, subprojects and documents**includes**Upward and downward search**, **Agent results**, **Summaries** and **Code snippets**, all linked in one**Knowledge database**.

Example: Full hierarchical document numbering system

```
Category: Contracts 2026 → Document number: V2026
├─ Contract 1 → Document number: V2026-01
│  ├─ Datei001.pdf → Document number: V2026-01-0001
│  │  ├─ Keywords (Agent 1): “Rental agreement, deadlines, termination”
│  │  ├─ Code / Tables (Agent 2): none
│  │  └─ Summary (Agent 3): short summary saved
```

```
│ └─ Summary001.pdf → Document number: V2026-01-0001-S
├─ Contract 2 → Document number: V2026-02
│ └─ Datei002.pdf → Document number: V2026-02-0001
│ ├─ Keywords (Agent 1): “Delivery, deadlines, payment terms”
│ ├─ Code / Tables (Agent 2): none
│ └─ Summary (Agent 3): short summary saved

Category: Projects 2026 → Document number: P2026
├─ Project 1 → Document number: P2026-01
│ ├─ Subproject 1a → Document number: P2026-01-01
│ │ └─ Datei003.docx → Document number: P2026-01-01-0001
│ │ ├─ Keywords (Agent 1): “Prototype, Test, Analysis”
│ │ ├─ Code (Agent 2): Code snippet extracted
│ │ └─ Summary (Agent 3): Concept paper summarized
│ └─ Subproject 1b → Document number: P2026-01-02
│ └─ Datei004.docx → Document number: P2026-01-02-0001
│ ├─ Keywords (Agent 1): “Design, Architecture, Data Flow”
│ ├─ Code (Agent 2): Code snippet extracted
│ └─ Summary (Agent 3): Concept paper summarized
├─ Project 2 → Document number: P2026-02
│ └─ Datei005.docx → Document number: P2026-02-0001
│ ├─ Keywords (Agent 1): “Budget, Controlling, Reporting”
│ ├─ Code (Agent 2): none
│ └─ Summary (Agent 3): Summary saved
```

---

functions

1. **Upward search (optional)**
   - Search for `P2026-01-01-0001`shows subproject`1a`and project`P2026-01`
2. **Downward search (default)**
   - Search for `P2026-01`shows all subprojects and files within them
3. **Filters and selection:**
   - Category, Project, Subproject, Document Number, Keyword, Responsible Party, Status
4. **Agent integration:**
   - **Agent 1**→ Extract keywords
   - **Agent 2**→ Extract code/concepts, save them one-to-one
   - **Agent 3**→ Create summaries, optionally include the original text
5. **Link:**
   - All files, summaries, and code snippets are linked in the knowledge base.
   - Each file shows which project / subproject / category it is contained in.

Advantages

- **Flexible:**Document numbers can be assigned at any level.
- **Fully traceable:**Up and down search optional
- **Team-friendly:**The dashboard displays all categories, projects, and subprojects.
- **Agent results automatically linked:**Keywords, code, summaries
- **Any additional criteria:**Status, Responsible Party, Year, Priority, Tags

---

💡 **Note:**

- **Document number = universal ID for category, project, subproject, document**
- **Up and down search = full transparency**
- **Agents + knowledge base = automatic tracking of all results**

If you want, I can do it next.**build a graphical overview**, which shows:

- Top-level categories → Projects → Subprojects → Files
- Which agent results belong to which document number?
- Visual up and down search
- Matrix-style navigation for team dashboard

Do you want me to do that?

As text

All right – here is the**graphical overview as plain text**, which displays all levels, document numbers and agent results, including optional upward and downward search:

Hierarchical document numbering system – text overview

```
Category: Contracts 2026 → Document number: V2026
├─ Contract 1 → Document number: V2026-01
│  ├─ Datei001.pdf → Document number: V2026-01-0001
│  │  ├─ Keywords (Agent 1): Lease agreement, deadlines, termination
│  │  ├─ Code / Tables (Agent 2): none
│  │  └─ Summary (Agent 3): short summary saved
│  └─ Summary001.pdf → Document number: V2026-01-0001-S
├─ Contract 2 → Document number: V2026-02
│  └─ Datei002.pdf → Document number: V2026-02-0001
│  ├─ Keywords (Agent 1): Delivery, deadlines, payment terms
```

```
│ ├─ Code / Tables (Agent 2): none
│ └─ Summary (Agent 3): short summary saved

Category: Projects 2026 → Document number: P2026
├─ Project 1 → Document number: P2026-01
│ ├─ Subproject 1a → Document number: P2026-01-01
│ │ └─ Datei003.docx → Document number: P2026-01-01-0001
│ │ ├─ Keywords (Agent 1): Prototype, Test, Analysis
│ │ ├─ Code (Agent 2): Code snippet extracted
│ │ └─ Summary (Agent 3): Concept paper summarized
│ └─ Subproject 1b → Document number: P2026-01-02
│ └─ Datei004.docx → Document number: P2026-01-02-0001
│ ├─ Keywords (Agent 1): Design, Architecture, Data Flow
│ ├─ Code (Agent 2): Code snippet extracted
│ └─ Summary (Agent 3): Concept paper summarized
├─ Project 2 → Document number: P2026-02
│ └─ Datei005.docx → Document number: P2026-02-0001
│ ├─ Keywords (Agent 1): Budget, Controlling, Reporting
│ ├─ Code (Agent 2): none
│ └─ Summary (Agent 3): Summary saved

# **Search Options**
# Downward search (default):
# - Search for P2026-01 → shows subprojects 1a & 1b + all files
# Optional Upward Search:
# - Search for P2026-01-01-0001 → shows subproject 1a + project 1 + category Projects 2026

# **Link**
# - All agent results (tags, code, summaries) are linked to original files
# - Each file automatically shows which subproject, project, or category it is contained in.
```

✅ With this you have:

- **Each level is numbered:**Category → Project → Subproject → File
- **Agent results**directly visible and assigned
- **Optional up and down search**described
- **Link to knowledge base**clearly documented

If you want, I can do it next.**build an extended text model**, in which **Document numbers are automatically generated, metadata is assigned, and agent results are directly integrated.**so that you could get started practically immediately.

Do you want me to do that?

And

Perfect – here is a**extended text model**, which brings all functions together:**Automatic document numbers, metadata, agent results, linking**, everything prepared as plain text so that you could work with it immediately:

Extended document numbering system with knowledge database

==========
Category levels
==========
Category: Contracts 2026 → Document number: V2026
Metadata: Year=2026, Type=Contracts, Responsible=Team A

==========
Projects / Subprojects
==========
Project: Contract 1 → Document number: V2026-01
Metadata: Customer=Müller GmbH, Status=Draft

Datei001.pdf → Document number: V2026-01-0001
Metadata: Author=Max, Date=2026-03-10, Type=PDF
Keywords (Agent 1): Lease agreement, deadlines, termination
Code / Tables (Agent 2): none
Summary (Agent 3): Short summary saved
Linking: Category=V2026, Project=V2026-01

Summary001.pdf → Document number: V2026-01-0001-S
Metadata: Author=Max, Date=2026-03-11
Link: Original=V2026-01-0001

Project: Contract 2 → Document number: V2026-02
Metadata: Customer=Schmidt AG, Status=Approved

Datei002.pdf → Document number: V2026-02-0001
Metadata: Author=Anna, Date=2026-03-09, Type=PDF
Keywords (Agent 1): Delivery, deadlines, payment terms
Code / Tables (Agent 2): none
Summary (Agent 3): Short summary saved
Linking: Category=V2026, Project=V2026-02

==========
Project 2
==========

Category: Projects 2026 → Document number: P2026
Metadata: Year=2026, Type=Projects, Responsible=Team B

Project: Project 1 → Document number: P2026-01
Metadata: Customer=Internal Development, Status=In Progress

Subproject 1a → Document number: P2026-01-01
Metadata: Responsible = Team C

Datei003.docx → Document number: P2026-01-01-0001
Metadata: Author=Ursula, Date=2026-03-12, Type=DOCX
Keywords (Agent 1): Prototype, Test, Analysis
Code (Agent 2): Code snippet extracted
Summary (Agent 3): Concept paper summarized
Linking: Category=P2026, Project=P2026-01, Subproject=P2026-01-01

Subproject 1b → Document number: P2026-01-02
Metadata: Responsible = Team D

Datei004.docx → Document number: P2026-01-02-0001
Metadata: Author=Ursula, Date=2026-03-12, Type=DOCX
Keywords (Agent 1): Design, Architecture, Data Flow
Code (Agent 2): Code snippet extracted
Summary (Agent 3): Concept paper summarized
Linking: Category=P2026, Project=P2026-01, Subproject=P2026-01-02

Project: Project 2 → Document number: P2026-02
Metadata: Customer=External Partner, Status=Approval

Datei005.docx → Document number: P2026-02-0001
Metadata: Author=Ursula, Date=2026-03-11, Type=DOCX
Keywords (Agent 1): Budget, Controlling, Reporting
Code (Agent 2): none
Summary (Agent 3): Summary saved
Linking: Category=P2026, Project=P2026-02

==========
Search and filter functions
==========
Downward search:
- Enter document number P2026-01 → displays subprojects + all files
Optional Upward Search:
- Enter document number P2026-01-01-0001 → displays subproject + project + category

==========
Automatic metadata assignment
==========
- Generate document number for file / project / category

- Keywords Agent 1 automatically or manually (10–100 keywords)
- Extract Code / Concept Agent 2 one-to-one, separate file
- Summary Agent 3, optional copy original text
- All results linked to the knowledge database

==========
Options
==========
- Analyze files automatically or manually
- Select agents (1=Keywords, 2=Code/Concept, 3=Summary)
- Copy original text: Yes/No
- Storage: Automatic / Manual
- Assign document numbers at each level
- Upward search can be optionally activated
- Filterable by category, project, subproject, document number, keyword, author, status

---

✅ **This model makes the following possible:**

1. Document numbers on**Category, project, subproject and document level**
2. **Agent results**automatically extracted and linked
3. **Upward and downward search** optional
4. Complete **metadata capable** and **filterable**
5. **Automatic or manual analysis**selectable
6. **Flexibles Team-Dashboard** possible

If you want, I can do it next.**outline a pure text dashboard flow**how to communicate within the team about document numbers, folders, keywords, agent results, and metadata**Navigate, search, and edit all documents**can – almost like a**Text-based team dashboard**.

Do you want me to do that?

And

Perfect – here is a**Text-based team dashboard**for your system. It shows how to**Navigate and edit documents using document numbers, folders, keywords, agent results, and metadata.**can provide everything as plain text for immediate implementation:

Text-based team dashboard – overview

==========

Dashboard Homepage
==========
Welcome to the knowledge dashboard
[Options]
1. Search by document number
2. Search by category / project / subproject
3. Search by keyword / metadata
4. Display of all agent results (Agent 1, 2, 3)
5. Add/Import Files
6. Agent settings (automatic / manual)

==========
Search by receipt number
==========
Input: e.g. P2026-01-01-0001
→ Result:
  - Datei003.docx (Original)
- Keywords: Prototype, Test, Analysis
- Code snippet: extracted
- Summary: Concept paper created
- Upward tracking:
- Subproject: P2026-01-01
- Project: P2026-01
- Category: P2026
- Downward tracking:
- no further subunits (file is the lowest level)

==========
Search by category / project / subproject
==========
Input: Project 1 → P2026-01
→ Result:
- Subproject 1a → View files
- File003.docx, Keywords, Code, Summary
- Subproject 1b → View files
- File004.docx, keywords, code, summary
Option: Analyze all files (agents) → automatically or manually

==========
Search by keyword / metadata
==========
Input: Keyword = "Analysis"
→ Result:
- File003.docx → Subproject 1a → Project 1 → Category Projects 2026
- File006.docx → Subproject 2b → Project 2 → Category Projects 2026
Option: Run agents for these files (Agent 1, 2, 3)
Option: Create a new summary / new file
Option: Extract code (Agent 2)

==========
Display agent results
==========
[Agent 1] Keywords
- File001.pdf → Rental agreement, deadlines, termination
- File003.docx → Prototype, Test, Analysis
[Agent 2] Code / Concept
- File003.docx → Code snippet extracted
- File004.docx → Code snippet extracted
[Agent 3] Summaries
- File003.docx → Concept paper created
- File004.docx → Concept paper created

==========
Add / Import Files
==========
- Select folder → Plugin imports files into knowledge base
- Automatically extract metadata from folder structure / file content
- Optional: Automatically start agents for new files
- Optional: Generate document numbers for each file / project / category

==========
Agent settings
==========
- Agent 1: Keywords automatically / manually
- Agent 2: Code / Concept automatic / manual
- Agent 3: Automatic/manual summary, retain original text: Yes/No
- Agents only for selected files / folders / metadata
- Number of keywords per file: freely selectable (e.g. 10–100)
- Save code/concepts in a separate file, linked to the original file
- Save summaries in a separate file, optionally with a template

==========
Navigation & Filter
==========
Filter options:
- Category / Project / Subproject
- Receipt number
- Keyword
- Author / Responsible party
- Status (Draft, In Progress, Released)
- File type (PDF, DOCX, Code, Image, Video)
Option: Combine filters → Dynamically update results list
Option: Start agents directly for filtered files

==========
Upward and downward search

==========
- Upward: from File → Subproject → Project → Category
- Downward: from Category → Project → Subproject → Files
- Optional: Display matrix organization:
- Shows which documents / summaries / codes were reused in which projects
- Document number tracking for all levels

==========
Export / Further processing
==========
- Save results as a new file
- Link to knowledge database
- Optionally include original text
- Generate new document number for new file / project / summary

---

✅ **With this text dashboard, your team can:**

1. Instantly search all documents, projects, subprojects, and categories
2. Targeted agent launching: Keywords, code, summaries
3. Use upward and downward tracking of document numbers
4. Automatically or manually analyze and save new files
5. Filter by metadata, keywords, document numbers, status, author
6. Link all agent results directly to the knowledge base.
7. Create and manage document numbers at every level

If you want, I can do it next.**Prepare the complete text matrix for document numbers, projects, subprojects, files, agent results and links.**, so that one can**like a real team dashboard in text form**could use it.

Do you want me to do that?

And

Perfect – here is the**Complete text matrix for your team dashboard**, which maps all levels, document numbers, files, agent results, metadata, and links. This gives you**everything at a glance**and you can navigate, search and edit directly:

Complete team dashboard matrix (text format)

==========
Category levels
==========
Category: Contracts 2026 → Document number: V2026
Metadata: Year=2026, Type=Contracts, Responsible=Team A
Link: All projects/files below

==========
Projects / Subprojects
==========
Project: Contract 1 → Document number: V2026-01
Metadata: Customer=Müller GmbH, Status=Draft
Link: Category=V2026

Datei001.pdf → Document number: V2026-01-0001
Metadata: Author=Max, Date=2026-03-10, Type=PDF
Keywords (Agent 1): Lease agreement, deadlines, termination
Code / Tables (Agent 2): none
Summary (Agent 3): Concept paper prepared
Link: Project=V2026-01, Subproject=V2026-01-0001

Summary001.pdf → Document number: V2026-01-0001-S
Metadata: Author=Max, Date=2026-03-11
Link: Original=V2026-01-0001

Project: Contract 2 → Document number: V2026-02
Metadata: Customer=Schmidt AG, Status=Approved
Link: Category=V2026

Datei002.pdf → Document number: V2026-02-0001
Metadata: Author=Anna, Date=2026-03-09, Type=PDF
Keywords (Agent 1): Delivery, deadlines, payment terms
Code / Tables (Agent 2): none
Summary (Agent 3): Summary saved
Link: Project=V2026-02

==========
Category Projects
==========
Category: Projects 2026 → Document number: P2026
Metadata: Year=2026, Type=Projects, Responsible=Team B

Project: Project 1 → Document number: P2026-01
Metadata: Customer=Internal Development, Status=In Progress
Link: Category=P2026

Subproject 1a → Document number: P2026-01-01
Metadata: Responsible = Team C

Link: Project=P2026-01

Datei003.docx → Document number: P2026-01-01-0001
Metadata: Author=Ursula, Date=2026-03-12, Type=DOCX
Keywords (Agent 1): Prototype, Test, Analysis
Code (Agent 2): Code snippet extracted
Summary (Agent 3): Concept paper prepared
Link: Subproject=P2026-01-01

Subproject 1b → Document number: P2026-01-02
Metadata: Responsible = Team D
Link: Project=P2026-01

Datei004.docx → Document number: P2026-01-02-0001
Metadata: Author=Ursula, Date=2026-03-12, Type=DOCX
Keywords (Agent 1): Design, Architecture, Data Flow
Code (Agent 2): Code snippet extracted
Summary (Agent 3): Concept paper prepared
Link: Subproject=P2026-01-02

Project: Project 2 → Document number: P2026-02
Metadata: Customer=External Partner, Status=Approval
Link: Category=P2026

Datei005.docx → Document number: P2026-02-0001
Metadata: Author=Ursula, Date=2026-03-11, Type=DOCX
Keywords (Agent 1): Budget, Controlling, Reporting
Code (Agent 2): none
Summary (Agent 3): Summary saved
Link: Project=P2026-02

==========
Matrix overview / Document number linking
==========
Document number matrix (optional upward and downward search):
- File003.docx → Subproject 1a → Project 1 → Category Projects 2026
- File004.docx → Subproject 1b → Project 1 → Category Projects 2026
- Summaries / Code files → linked to original files
- Projects can have subprojects → any depth
- Document numbers optional for each level

==========
Agent operations (controllable)
==========
Agent 1 (Keywords):
- Selection: Single file / Folder / Multiple folders / Metadata
- Automatic / Manual
- Number of keywords selectable (10–100)

- Results are saved and linked as metadata

Agent 2 (Code / Concepts):
- Selection: Single file / Folder / Multiple folders / Metadata
- Filter: Code, Concept, Other
- Output: separate file, linked to the original file
- Automatic / Manual

Agent 3 (Summary):
- Selection: Single file / Folder / Multiple folders / Metadata
- Copy original text: Yes/No
- Template: optionally selectable
- Automatic / Manual
- Save results in a separate file, retain metadata and links.

==========
Filter & Search
==========
- Category / Project / Subproject
- Receipt number
- Keyword / Metadata
- Author / Responsible party
- Status (Draft, In Progress, Released)
- File type (PDF, DOCX, Code, Image, Video)
- Combination of filters possible
- Display results dynamically

==========
Import / Storage / Document Numbers
==========
- Import files from folder → Plugin automatically copies to knowledge base
- Original files remain unchanged
- Automatically generate receipt numbers for:
- Categories
- Project
- Subproject
- file
- New files / summaries / codes → new document numbers possible
- All agent results automatically linked

✅ **This model delivers:**

1. Complete **hierarchical matrix**Category → Project → Subproject → Files
2. **Document numbers at each level**, optional up and down search
3. **All agent results**embedded, linked and with metadata
4. **Flexibles Team-Dashboard**for navigation, search, filtering, agent control
5. **Automatic or manual analysis**and storage

6. **Import from folder structure**without altering the original files

---

If you want, I can do something else next**build a visualization in text form**, the**shows how files, projects, document numbers, and agent results are linked.**, Almost **a text matrix graphic**, which can be read directly.

Do you want me to do that?

Technical implementation

Okay – I'll now summarize the**technical implementation**I'll put together a system for you, based on your existing requirements, so that a developer can work with it directly. I'll describe...**Architecture, database, plugin design, agents and processes**.

Technical implementation: Document numbers + knowledge database with agents

1. Architectural Overview

```
[User-Interface] ←→ [Plugin 1: Receipt number generator]
                 ↕
[Plugin 2: Knowledge Base]
                 ↕
[Agents (LLMs): Search, Code, Summary]
                 ↕
[Database + Storage]
```

Explanation:

- **Plugin 1**Generates document numbers for files, projects, and categories. Automatically links documents.
- **Plugin 2**Intelligent knowledge database with folder structure, metadata, document import, agent control.
- **Agents**:
  - **Agent 1**– Keyword analysis
  - **Agent 2**– Code / Concept Extraction
  - **Agent 3**– Summaries, optional template
- **database**Storage of all documents, metadata, receipt numbers, agent results.

---

2. Database

Recommendation: PostgreSQL or MariaDB (not SQLite, due to large data volumes)

Table structure:

1. **categories**

id SERIAL PRIMARY KEY
name TEXT
cover number TEXT UNIQUE
metadata JSONB
parent_id INT NULL -- for parent category

2. **projects**

id SERIAL PRIMARY KEY
name TEXT
cover number TEXT UNIQUE
metadata JSONB
category_id INT REFERENCES categories(id)
parent_id INT NULL -- for subprojects

3. **files**

id SERIAL PRIMARY KEY
filename TEXT
filepath TEXT
cover number TEXT UNIQUE
metadata JSONB
project_id INT REFERENCES projects(id)
imported_date TIMESTAMP

4. **agents_results**

id SERIAL PRIMARY KEY
file_id INT REFERENCES files(id)
agent_type TEXT -- 'Agent1', 'Agent2', 'Agent3'
result_type TEXT -- 'Tag', 'Code', 'Summary'
result_data JSONB
saved_file_path TEXT NULL -- for extracted codes / summaries
timestamp TIMESTAMP

5. **links**

id SERIAL PRIMARY KEY
source_file_id INT REFERENCES files(id)
target_file_id INT REFERENCES files(id)
link_type TEXT -- e.g. summary, code, derivation

---

3. Plugin 1 – Receipt number generator

- **Features:**
  1. Generate receipt number for:
     - Category
     - project
     - Subproject
     - file
  2. Automatic assignment upon file creation
  3. Document number optional in:
     - headline
     - Document text
     - Hidden for internal linking
  4. Link to Plugin 2 (database)
- **API / Interfaces:**

```
def generate_receiptnumber(entity_type: str, parent_receipt: str = None) -> str:
  ...
def link_file_to_database(file_path: str, belegnummer: str, metadata: dict):
  ...
```

---

4. Plugin 2 – Knowledge Base

- **Features:**
  1. File import:
     - Select folder → Plugin copies files to database storage
     - Original files remain unchanged
     - Optionally, automatically start agents.
  2. Metadata:
     - Automatically from folder name / file content
     - Manually editable
  3. Filter & Search:
     - Category / Project / Subproject
     - Document number
     - Keyword / Metadata
     - Author, status, file type
  4. Agent control:

        - Agent 1: Keyword Analysis
        - Agent 2: Code / Concept
        - Agent 3: Summary + Templates
        - Options: Single file, folder, multiple folders, metadata
        - Automatic / Manual
- **Interface to the dashboard / UI:**

```
def import_files(folder_path: str):
  ...
def filter_files(criteria: dict):
  ...
def run_agent(agent_id: int, file_ids: list, options: dict):
  ...
```

---

5. Agent Implementation

- **Agent 1 (keyword analysis)**:
    - Input: File / Folder / Metadata
    - Output: JSON with keywords
    - Storage: Metadata in database + link to file
- **Agent 2 (Code / Concepts)**:
    - Input: File / Folder / Metadata
    - Filter: Code, concepts, other
    - Output: New file containing code / concept
    - Link: Original file + knowledge base
- **Agent 3 (Summary)**:
    - Input: Single file / folder / metadata
    - Options: Use original text Yes/No, Use template
    - Output: New file
    - Link: Original file + knowledge base

---

6. Storage structure

```
/storage
  /categories
    /projects
      /subprojects
        /original_files
        /agent_results
```

- **original_files**→ unchanged, only read
- **agent_results**→ Codes, summaries, keyword lists
- All files linked via database

---

7. Example of a process

1. User selects folder "Project 1" → Plugin imports files
2. Document number automatically generated for each document
3. Agent 1 is started manually or automatically → Keywords saved
4. Agent 2 extracts code/concepts after selection → saves new files
5. Agent 3 creates summaries → original text optionally retained → saves new files
6. All files, results, document numbers, keywords → linked in knowledge base
7. Users can search/filter/display matrix view via dashboard.

---

8. Technologies

- Backend: Python (Django / FastAPI)
- Datenbank: PostgreSQL / MariaDB
- Document parsers: python-docx, PyPDF2, OpenCV (for images), ffmpeg (videos)
- LLM-Integration: lokale Modelle via Docker / Haystack / Hugging Face
- UI: Web-based (React or simple text dashboard with CLI possible)
- Storage: Local storage + database references

If you want, I can do it next.**a complete flowchart for the processes including Plugin 1, Plugin 2 and agents**create – as**Text diagram or ASCII diagram**, which can be used directly as a technical specification.

Do you want me to do that?

As text

Perfect – here is the**Complete technical flow diagram as a text/ASCII diagram**, which shows how**Plugin 1, Plugin 2, agents, database and files**work together. Everything in**a linear, comprehensible process**, as text form:

Technical flowchart (text version)

```
[User selects folder / file]
        |
in
 +--------------------+
```

```
| Plugin 2 Importer | -- copies files to knowledge base storage
| (Original remains) |
 +--------------------+
        |
in
+-------------------------+
| Receipt number generator | <- Plugin 1
| - Category, Project |
| - Subproject, file |
| - Number in the document |
+-------------------------+
        |
in
+-------------------------+
| Save database |
| - Files |
| - Document number |
| - Metadata |
+-------------------------+
        |
in
+-------------------------+
| Agent Selection |
| (User decides) |
+-------------------------+
  |        |          |
  |        |          |
v v v
[Agent1]   [Agent2]    [Agent3]
(Search) (Code/ (Summary)
Concepts)
  |        |          |
  |        |          |
v v v
+-------------------------+
| Save results |
| - JSON / new files |
| - Keywords |
| - Code snippets |
| - Summaries |
| - Link to original |
+-------------------------+
        |
in
+-------------------------+
| Knowledge Base |
| - Files & Results |
| - Metadata & Location No. |
```

```
| - Links |
+-------------------------+
        |
in
+-------------------------+
| Dashboard / UI          |
| - Search (Category, |
| Project, Location No., |
| Metadata, keywords) |
| - Filter                |
| - Agent control |
| - Matrix view |
+-------------------------+
```

Legend & Notes

- **Plugin 1**= Document number generator
- **Plugin 2**= Knowledge base + file import + dashboard
- **Agent 1**= Keyword analysis (automatic / manual)
- **Agent 2**= Code / concept extraction (one-to-one, separate file)
- **Agent 3**= Summary (optional original text, templates available)
- **Link**= Each file/result is linked to the original file and project structure
- **database**PostgreSQL/MariaDB stores all metadata, results, and document numbers.

If you want, I can do it next.**Prepare the flow logic for all three agents step by step in text form.**, including**Options: Automatic / Manual, Folder / File / Metadata, Storage & Linking**, so that a developer can implement it directly.

Do you want me to do that?

And

All right – here is the**Detailed flow logic for all three agents in text form**, including**Selection options, automatic options, storage and linking logic**This allows a developer to implement the processes directly.

Agent Flow Logic – Text Version

==========

AGENT 1 – Keyword Analysis
==========
Start:
User selects:
- Single file
  - Folder
- Several folders
- Metadata filter
User selects automatic:
- Automatic (all selected files)
- Manual (only start explicitly)
User specifies the number of keywords (e.g., 10–100)

Processing:
For each selected file:
1. Analyze file content
2. Extract keywords
3. Save keywords as metadata
4. Create a link to the original file in the knowledge base.

Output:
- JSON: Keyword list
- Metadata updated in DB
- Link to file

Optional:
- User can edit results
- Results remain unchanged in the original file

==========
AGENT 2 – Code / Concept Extraction
==========
Start:
User selects:
- Single file
  - Folder
- Several folders
- Metadata filter
User selects filter:
  - Code
- Concepts
- Other Types (optional)
User selects automatic:
- Automatically
- Manual
User selects destination:
- New file / file type
- Link to the original file

- Optional: Folder for agent results

Processing:
For each selected file:
1. Search content by filter
2. Extract code or concepts one-to-one
3. Create a new file in the selected format
4. Metadata: Original file, document number, keywords, Agent 2 note
5. Create a link to the original file

Output:
- New file(s) containing code / concept
- Metadata + linking in database

Optional:
- User can combine multiple extraction criteria
- Results can be adjusted manually.

==========
AGENT 3 – Summary / Templates
==========
Start:
User selects:
- Single file
  - Folder
- Several folders
- Metadata filter
Options:
- Use original text? (Yes / No)
- Use a template? (optional)
- Automatic / Manual
- Choose destination for new file (folder, name, type)

Processing:
1. Collect content from all selected files
- If original text = Yes, copy verbatim.
2. Create a summary
- Optional: Apply template
3. Create a new file
- Metadata: Original files, document number, keywords, summary note
- Create links to all original files

Output:
- New summary file(s)
- Metadata + linking in database

Optional:
- User can perform further editing

- Automatic or manual saving selectable

==========
Shared linking and storage logic
==========
For all agents:
- Each new file is linked to the original file
- Document numbers for new files are optional
- Metadata is automatically updated
- Filter and search function available
- Dashboard shows:
- Category → Project → Subproject → File
- Document number matrix
- Keywords / Codes / Summaries

✅ **Highlights of this text flow logic**:

1. **Maximum flexibility**:
   - Single file, folder, multiple folders, metadata filter
   - Automatic or manual
2. **Agent-specific actions**:
   - Agent 1: Keyword Analysis
   - Agent 2: Extract code/concept (one to one)
   - Agent 3: Summary (Original text optional, templates)
3. **Metadata & Linking**:
   - Each new file is linked in the knowledge base.
   - Document numbers are optional at each level
   - Search and filter options
4. **User control**:
   - Results can be edited manually
   - Storage controllable (automatic or manual)

If you want, I can**Next, an example of the exact step-by-step operation of a user with all three agents in a project.**write it down – including**Selection, filtering, saving and linking – completely as a text scenario**.

Do you want me to do that?

And

Perfect – here is a**complete text scenario**This shows how a user utilizes the three agents in a project step by step, including selection, filtering, saving, and linking. This clearly demonstrates how everything works together:

User scenario: Agents in action – text version

Project: "Project 1 – New Product Development"

Step 1: Import files

- User selects folder "Project 1" on the hard drive
- Plugin 2 copies all files to the knowledge base storage.
- Original files remain unchanged
- Metadata automatically generated:
- File name
- Import date
- Folder name = Project assignment

Step 2: Assign document numbers

- Plugin 1 generates receipt numbers for:
- Project 1
- Subprojects
- Individual files
- Receipt numbers are:
- Embedded in document heading
- Stored in database
- Optionally hidden in the document

Step 3: Agent 1 – Keyword Analysis

- User selects:
- All files in project 1
  - Automatikmodus
- 20 keywords per file
- Agent 1 processes:
- Each file is analyzed
- Keywords extracted
- Metadata stored in database
- Link to the original file created
- User can edit results

Step 4: Agent 2 – Code / Concept Extraction

- User selects:

- Subfolder "Backend" + metadata filter "Code relevant"
  - Automatikmodus
- Extraction criteria: Code + Concepts
- Agent 2 processes:
- Code blocks extracted one-to-one → new file "Backend_Code.py"
- Concepts extracted one-to-one → new file "Backend_Concepts.docx"
- Metadata: Original files, document numbers, keywords
- Link to original files established
- Optional:
- User specifies folder for agent results
- User can manually edit results

Step 5: Agent 3 – Summaries

- User selects:
- All files from Project 1
- Filter: Keyword "Architecture"
- Option: Use original text = Yes
- Template: "Executive Summary"
- Automatic mode = No (User starts manually)
- Agent 3 processes:
- Content collected from selected files
- Summary generated
- New file: "Project1_Architecture_Summary.docx"
- Metadata: Original files, document numbers, keywords
- Links to all original files created
- Optional:
- User can perform further editing
- Saving occurs after approval

Step 6: Links & Matrix Overview

- Dashboard shows:
- Project → Subproject → Files
- Document number matrix
- Keywords, codes, summaries
- Links between:
- Original files
- Agent results
- Subprojects
- User can:
- Search by document number
- Filter by metadata or keywords
- Track relationships (matrix):
- Which summaries are based on which original files?
- Which codes are used in which projects

Step 7: Expansion & Integration

New files can be imported at any time.
- The user decides whether agents start automatically or manually.
- The document number generator can be used directly in Plugin 2:
- New projects/documents are immediately numbered
- History and matrix updated automatically
- All agent results remain flexibly editable

✅ **Special features of this scenario**:

1. **Complete control for the user**Single file, folder, multiple folders, metadata filter.
2. **Optionality**: Automatic vs. manual execution for each agent.
3. **One-to-one processing**Code, concepts, and original texts remain unaltered.
4. **Links & Matrix Structure**Every file, every result remains traceable in the knowledge database.
5. **Integration of document numbers**: For files as well as for projects and subprojects.

If you want, I can now**a complete tabular overview of all options and parameters for each agent**create – so**User interface decisions, filters, automation, storage, linking, document numbers**– compact for implementation.

Do you want me to do that?

And

Perfect – here is the**A compact, tabular overview of all options and parameters for each agent.**, with all relevant decisions for**UI, filters, automation, storage, linking and document numbers**. It is prepared in such a way that a developer can directly implement the interface and logic.

Agent Options & Parameters – Overview Table

Agent	action	Selection area	Automatic	Filters / Criteria	storage	Link	The receipt number	Notes

Agent 1	Keyword analysis	Single file / Folder / Multiple folders / Metadata filter	Automatic / Manual	Number of keywords (e.g. 10–100)	Metadata in DB	Link to the original file	Optional, if newly created	Users can edit results.
Agent 2	Extract code / concepts	Single file / Folder / Multiple folders / Metadata filter	Automatic / Manual	Type: Code / Concepts / Other	New file(s), optional destination folder	Link to original files	Optional, if a new file is created	Multiple extraction criteria can be combined
Agent 3	Summary / Templates	Single file / Folder / Multiple folders / Metadata filter	Automatic / Manual	Use original text (Yes/No), Template (optional)	New file, destination selectable	Link to original files	Optional, if a new file is created	The user decides when data is saved.
Receipt number generator (Plugin 1)	Number assigned	Project / Subproject / File / Category	Automatic / Manual	Number format selectable	In Dokument + DB	Link to project / subproject	Yes, core function here	Can be used directly in Plugin 2.
Dashboard / UI	Search & Overview	Project / Subproject / File / Document No. / Metadata	–	Keywords, codes, summaries	Display + Export	Display of links	Display of existing document numbers	Filter & matrix view for projects, subprojects, files

---

Special notes for implementation

1. **Flexibility & Control**
    - Each agent can be activated individually.
    - Automatic mode is optional for all agents.
    - Selection: File, Folder, Multiple Folders, Metadata Filter
2. **Linking & Traceability**
    - Each new file/result is automatically linked to the original file.
    - Document numbers are possible at every level: project, subproject, file
    - Dashboard displays matrix of relationships
3. **Storage & Structure**
    - Metadata centrally stored in DB
    - New files in specified folders / database storage
    - Original files remain unchanged on the hard drive
4. **Expandability**
    - New agents can be added.
    - Plug-in 1 (document number generator) can also be used in other modules.
    - Filters and templates can be flexibly customized.

If you want, I can do it next.**Build a text diagram that shows all agents, Plugin 1 & 2, document numbers, storage, and dashboard in one large overall flow.** – also **completely linked**so that the architecture can be seen at a glance.

Do you want me to do that?

The output can also be saved to an external hard drive if it contains videos, to ensure the fastest possible processing time.

Perfect, that complements the architecture nicely. The storage logic then looks like this:

Extension: Storage location for output

- **Standard:**Knowledge base storage (DB + associated folder structure)
- **Optional:**External hard drive as a destination for new files, code snippets, summaries, videos, images, etc.
    - Advantage: Faster access to large media files
    - All metadata and links are still stored centrally in the knowledge database.
    - Original files remain unchanged on the hard drive.

Example workflow for Agent 2 or 3:

User selects storage destination:
- Internal database storage
- External hard drive (e.g. D:\Project1\Agent2_Output)

Agent processes files:
- Code / concepts / summaries are stored on selected memory.
- Metadata including links are stored centrally in the database.
- Dashboard displays all files including the path to the external hard drive

- **A notice:**For large video or image files, only store the path in the database, do not embed the entire content in the database – this way everything remains performant.

If you want, I can now**Complete overall architecture including Plugin 1 + Plugin 2 + all agents + storage options (DB + external hard drive) as a text diagram**Present it – clearly and in a flowing manner.

Do you want me to do that?

And

All right – here is the**Complete architecture as a text diagram**, including**Plugin 1, Plugin 2, all three agents, knowledge base and optional external hard drive for output**Everything as**linearly readable flow**, so that the structure is immediately understandable:

Overall architecture – text diagram

```
[User selects folder / file / project]
        |
in
 +-------------------------+
 | Plugin 2 Importer       |
| - Copies files |
| - Original remains |
| - Create folder structure |
 +-------------------------+
        |
in
 +-------------------------+
| Plugin 1 Receipt number generator |
```

```
| - Project / Subproject / File |
| - Number in document + database |
| - Optional: hidden in the text |
 +-------------------------+
        |
in
+-----------------------------------+
| Knowledge Base |
| - Files & Metadata |
| - Document number |
| - Links |
+-----------------------------------+
        |
in
+-------------------------+
| Agent Selection |
| (User decides) |
+-------------------------+
  |         |         |
  |         |         |
v v v
[Agent 1]   [Agent 2]   [Agent 3]
(Search) (Code/Concepts) (Summary)
  |         |         |
  |         |         |
v v v
+-------------------------------------------------+
| Results / Output |
| - New files / code / summaries |
| - Metadata + linking in database |
| - Storage location selectable: |
| • Knowledge base |
• External hard drive (videos, pictures, etc.) |
+-------------------------------------------------+
        |
in
+-------------------------+
| Dashboard / UI          |
| - Search for: |
| • Project / Subproject |
| • File / Document number |
| • Metadata / Keywords |
| - Filter options |
| - Matrix view: |
| • Linked summaries |
| • Code blocks |
| • Project references |
| - Access to agents / plugins |
```

+----------+

Key aspects of this architecture

1. **Plugin 1 (Receipt number generator)**
    - Numbering at each level (project, subproject, file)
    - Embedding in document + storage in database
    - Optional: Hidden numbers in the document
2. **Plugin 2 (Knowledge Base + Importer)**
    - Import files from any source
    - Original files remain unchanged
    - Folder structure is mirrored
    - Metadata is generated automatically
3. **Agents**
    - **Agent 1:**Keyword analysis, manual or automatic
    - **Agent 2:**Extract the code/concept, one-to-one, into a new file
    - **Agent 3:**Summary, optionally include original text, use templates
4. **Memory logic**
    - The database stores metadata, document numbers, and links.
    - Output can be stored on internal memory or an external hard drive (videos, images, etc.).
    - Paths are referenced in the database.
5. **Dashboard / UI**
    - Displays matrix relationships (Project → Subproject → Files)
    - Filter and search function via document numbers, metadata, keywords
    - Agent control directly from the dashboard

If you want, I can do it next.**A combined overview of all options + flow + storage locations + agent parameters in one large, coherent text diagram with a legend.**create – quasi**The complete reference for developers in one document**.

Do you want me to do that?

It's also important to note that Plugin 1 can be booked independently, as can Plugin 2, or you can book 2 first and then 1, and combine both. On the dashboard, where multiple people can work and share, you only see projects and contracts, so select via the top category.

Okay – that can be seamlessly integrated into the architecture. Here's the**extended text overview**, the**Flexible booking, combination of plugins and multi-user dashboard with visibility by category.** taken into account:

Extended Architecture – Text Diagram with Booking & Dashboard

```
[User selects plugin booking]
        |
|--- Plugin 1 only (Receipt number generator)
|--- Plugin 2 only (knowledge base + importer)
|--- Both plugins combined
        |
in
 +-------------------------+
 | Plugin 2 Importer       |
| - Copies files |
| - Original remains |
| - Create folder structure |
 +-------------------------+
        |
in
 +-------------------------+
| Plugin 1 Receipt number generator |
| - Project / Subproject / File |
| - Number in document + database |
| - Optional: hidden in the text |
 +-------------------------+
        |
in
+-----------------------------------+
| Knowledge Base |
| - Files & Metadata |
| - Document number |
| - Links |
+-----------------------------------+
        |
in
+-------------------------+
| Agent Selection |
| (User decides) |
+-------------------------+
  |          |          |
  |          |          |
v v v
[Agent 1]   [Agent 2]   [Agent 3]
(Search) (Code/Concepts) (Summary)
  |          |          |
  |          |          |
```

```
v v v
+-----------------------------------------------+
| Results / Output |
| - New files / code / summaries |
| - Metadata + linking in database |
| - Storage location selectable: |
| • Knowledge base |
• External hard drive (videos, pictures, etc.) |
+-----------------------------------------------+
        |
in
+-------------------------+
| Dashboard / UI          |
| - Multi-User Function |
| - Visibility per category: |
| • Projects |
| • Contracts |
| • Optional other top-level categories |
| - Filter & Search: |
| • Project / Subproject / File |
| • Document number |
| • Metadata / Keywords |
| - Matrix view: |
| • Linked summaries |
| • Code blocks |
| • Project references |
| - Agent control directly from the dashboard |
| - Sharing / Rights management for multiple users |
+-------------------------+
```

Key points of the expansion

1. **Flexible plugin booking**
    - Plugin 1 can be used independently
    - Plugin 2 can be used independently
    - Combination: Plugin 2 → Plugin 1 → Dashboard Integration
2. **Dashboard & Multi-User**
    - Users only see certain categories (e.g., projects, contracts)
    - Rights management: Who can see/edit what?
    - Matrix view for document numbers, summaries, code
3. **Integration of the agents**
    - Agents can access both plugins if booked.
    - Metadata and document numbers are consistently linked
4. **Storage options**
    - Internal (knowledge database)
    - External (hard drive, videos, large files)

- Paths are referenced in the database.

If you want, I can do it next.**create a final, consolidated overview**, the**Booking, dashboard, agents, receipt numbers, storage, multi-user rights and flow in a single text diagram**united – virtually**The complete reference for developers**.

Do you want me to do that?

You can also select, via subcategories and all levels, whether someone should see something or not.

Perfect, then the following results:**Access and combination logic for plugins and dashboard usage**:

Plugin and dashboard access logic

1. Plugin booking / activation:

• Plugin 1 (Receipt number generator)
- Can be used autonomously
- Assigns document numbers for projects, subprojects, files, categories
• Plugin 2 (Knowledge Base + Agents)
- Can be used autonomously
- Import, folder structure, search agents, code, summary
• Combination:
- Book plugin 2 first → add plugin 1 → integrate both modules
- Document numbers are automatically transferred to the knowledge database
- Dashboard shows a combined overview

2. Dashboard / Multi-user access:

• Roles & Permissions:
User A only sees selected projects, subprojects, and files.
User B only sees contracts
- Selection via top-level category (Project / Contract / Category)
• Access to subcategories:
- Optional: Display across all levels
- The administrator can specify who sees which subfolders or files.
• Actions per user:
- Start keyword analysis (Agent 1)
- Extract code/concepts (Agent 2)

- Create summary (Agent 3)
- Generate document number (Plugin 1)
- Export files or save them to an external hard drive

3. Visibility & Filters:

• Top category:
- Project, contract, other categories
- User only sees the selected main categories
• Sublevels:
- Selection via subprojects, folders, files
- Users can enable or disable levels one by one.
• Metadata & keywords:
- Visibility depends on permissions
- Filterable by projects, contracts, keywords, document numbers

---

✅ **Key points from this expansion**

1. **Standalone plugins**– no obligation to book both together.
2. **Combination possible**– Document numbers are transferred to the knowledge database.
3. **Flexible visibility**– The dashboard only shows the categories and subcategories for which the user has permission.
4. **Multi-user capability**– several people work on projects, contracts, or subfolders simultaneously.
5. **Filterability & Role Control**– Each layer can be enabled or disabled, for both files and metadata.

If you want, I can do it next.**Represent the entire system, including multi-user dashboard, plugins, agents, storage options, and visibility logic, in a coherent, complete text diagram.**, so that one really**The complete architecture at a glance** has.

Do you want me to do that?

And

Perfect – here is the**complete, consolidated architecture**as a text diagram. It integrates**Plugin 1, Plugin 2, all agents, storage options, receipt numbers, multi-user dashboard, visibility by category and subcategories**Everything is linearly readable and immediately usable for the developers.

Complete architecture – text diagram

```
[User selects plugin booking]
        |
|--- Plugin 1 only (Receipt number generator)
|--- Plugin 2 only (knowledge base + agents)
|--- Both plugins combined
        |
in
 +-------------------------+
 | Plugin 2 Importer       |
| - Copies files |
| - Original remains |
| - Create folder structure |
 +-------------------------+
        |
in
 +-------------------------+
| Plugin 1 Receipt number generator |
| - Project / Subproject / File / Category |
| - Number in document + database |
| - Optional: hidden in the text |
 +-------------------------+
        |
in
+-----------------------------------+
| Knowledge Base |
| - Files & Metadata |
| - Document number |
| - Links |
| - Internal storage + optional external storage (hard drive) |
+-----------------------------------+
        |
in
+-------------------------+
| Agent Selection |
| (User decides) |
+-------------------------+
  |          |          |
  |          |          |
v v v
[Agent 1]   [Agent 2]   [Agent 3]
(Search) (Code/Concepts) (Summary)
  |          |          |
v v v
+-------------------------------------------------+
```

```
| Results / Output |
| - New files / code / summaries |
| - Metadata + linking in database |
| - Storage location selectable: |
| • Knowledge base |
• External hard drive (videos, pictures, etc.) |
+-----------------------------------------------+
        |
in
+-------------------------+
| Multi-User Dashboard     |
| - Roles & Permissions |
| • Access per top-level category (project, contract, other) |
| • Access to subcategories is optional |
| - Filter & Search: |
| • Project / Subproject / File |
| • Document number |
| • Metadata / Keywords |
| - Matrix view: |
| • Linked summaries |
| • Code blocks |
| • Project references |
| - Agent control directly in the dashboard |
| - Sharing & rights management for multiple users |
| - Visibility of each layer can be flexibly selected |
+-------------------------+
```

Special Functions & Logic

1. **Flexible plugin usage**
    - Plugins 1 and 2 can be booked individually or in combination.
    - This combination ensures automatic integration of document numbers into the knowledge database.
2. **Agents**
    - **Agent 1:**Keyword analysis, manual or automatic, stores metadata
    - **Agent 2:**Extract code/concepts one-to-one, create new files, optionally save automatically.
    - **Agent 3:**Summary with optional inclusion of the original text, use of templates, saves new files
3. **Storage & Performance**
    - Metadata, document numbers, links centrally stored in the database
    - Large files (videos, images) can be stored on an external hard drive.
    - Paths of external files are referenced in the database.
4. **Multi-user & rights management**
    - Visibility at category or subcategory level
    - Filterable by projects, subprojects, files, document numbers, metadata

    - Users can only see what they have permission to see.
5. **Document numbers & traceability**
    - Document numbers at each level (project, subproject, file)
    - Dashboard shows matrix relationships:
        - Which file was included in which summary?
        - Which concepts were used where?
        - Traceability via receipt numbers
6. **Dashboard**
    - Control of all agents
    - Overview of projects, contracts and subcategories
    - Matrix and filter view for quick navigation
    - Multi-user capable with rights management

If you want, I can still**a detailed flowchart solely for agent interaction, including selection, automation, storage, and linking.**create – quasi**the complete workflow for data processing in text form**so that everything is clear step by step.

Do you want me to do that?

And

Great, then here's the**Detailed agent workflow as a text diagram**, including**Selection options, automation, storage and linking**This shows exactly how Agents 1–3 affect files, folders, and metadata:

Agent workflow – text diagram

```
[User selects files / folders / metadata / project]
        |
in
+---------------------------+
| Agent selection |
| - Agent 1 (Search) |
| - Agent 2 (Code/Concepts) |
| - Agent 3 (Summary)|
+---------------------------+
        |
in
+---------------------------+
| Agent 1 – Keyword Analysis |
|---------------------------|
```

```
| Selection: |
| - Single file |
| - Folder / multiple folders |
| - Filters / Metadata |
| - All files |
| Options: |
| - Manual |
| - Automatic |
| Output:                    |
| - Keywords / Keywords |
| - Metadata stored in database |
| - Link to original file |
+---------------------------+
        |
in
+---------------------------+
| Agent 2 – Code / Concepts |
|---------------------------|
| Selection: |
| - Single file |
| - Folder / multiple folders |
| - Metadata / Filters |
| Actions: |
| - Extract code one-to-one |
| - Extract concepts one-to-one |
| - Other criteria possible |
| Options: |
| - Save manually |
| - Save automatically |
| Output:                    |
| - New files (code, concepts) |
| - Link to the original file |
| - Optional storage location: DB or external hard drive |
+---------------------------+
        |
in
+---------------------------+
| Agent 3 – Summary |
|---------------------------|
| Selection: |
| - Single file |
| - Folder / multiple folders |
| - Metadata / Filters |
| Options: |
| - Copy original text (Yes/No) |
| - Use template (Yes/No) |
| - Auto save (Yes/No) |
| Output:                    |
```

```
| - New file with summary |
| - Link to all original files |
| - Assign metadata |
| - Optional storage location: DB or external hard drive |
+---------------------------+
        |
in
+---------------------------+
| Dashboard Integration      |
|---------------------------|
| - Overview of all agent results |
| - Filterable by keywords, metadata, projects, document numbers |
| - Linking the new files to the originals |
| - Matrix view: |
• Which codes from which files?
• Which summaries from which files?
| - Control: manual / automatic for each agent operation |
+---------------------------+
```

Key aspects of the agent workflow

1. **Flexibility in choice**
    - Single file, folder, multiple folders, filter by metadata
    - Selection can be made individually for each agent.
2. **Automatic vs. manual**
    - The user decides whether processing should run automatically.
    - Automatically: e.g., analyze all files in a folder
    - Manually: selectively select and analyze individual files
3. **Output & Storage**
    - New files (code, concepts, summaries)
    - Metadata stored in the knowledge database
    - Links to original files
    - Storage location selectable: internal or external hard drive
4. **Linking & Traceability**
    - Each new file is linked to all the originals.
    - Dashboard displays matrix relationships
    - Traceability of all changes, summaries, and code extracts
5. **Templates & Original Text**
    - Agent 3 can optionally use templates.
    - The original text can be included or omitted.
    - Flexibility for users: Summary alone or with original text

---

If you want, I can do it next.**Build the complete, final text document for developers**, the **All components, plugins, agents, storage options, multi-user rights, document**

numbers and dashboard in a single flowbrings together – essentially the**complete specification**.

Do you want me to do that?

And

Perfect, then I'll summarize everything.**in a single, consolidated text document for developers**together. This contains**Plugins, agents, document numbers, storage, multi-user dashboard, rights management, visibility by categories and subcategories**Everything is linear, clearly structured, and directly implementable.

Complete System Architecture – Developer Reference (Text)

1. Plugin booking & combination

• Plugin 1: Receipt number generator
- Assigns document numbers for projects, subprojects, files, categories
- Numbers in the document, optionally hidden or in the heading
- Link to knowledge database
• Plugin 2: Knowledge Base + Agents
- Imports files from folder structure
- Original files remain unchanged
- Automatic or manual copying into plugin folder structure
Metadata is automatically captured: name, date, folder, and, if applicable, document number.
• Combination:
- Book plugin 2 first → Add plugin 1 → Integration of document numbers
- Dashboard displays all linked data

2. File processing & storage

• Imported files are stored in the knowledge base
• Supported file types:
- Text files, Word, Excel, PDFs
- Images, videos, code files
• Storage locations:
    - Intern in DB
- Optionally on an external hard drive for large files
• Paths and links are stored in the database.
• Original files remain unchanged

3. Document numbers & traceability

• Document numbers at all levels:
- Projects / Subprojects / Files / Categories
• Matrix view:
- Which files were included in which summaries, code files, or projects?
• Optional filter:
- Document numbers, categories, subcategories
- Display only for authorized users
• Dashboard displays links and a matrix of all document numbers

4. Agents

[Agent 1 – Search]
- Selection: Single file, folder, multiple folders, filter, all files
- Options: manual / automatic
- Output: Keywords → Metadata in database, linked to original file
[Agent 2 – Code/Concepts]
- Selection: Single file, folder, multiple folders, filter/metadata
- Actions: Extract code, extract concepts, multiple criteria possible
- Options: save manually / automatically
- Output: new files, linked to original files
[Agent 3 – Summary]
- Selection: Single file, folder, multiple folders, filter/metadata
- Options:
• Use original text: Yes/No
• Use template: Yes/No
• Auto save: Yes/No
- Output: new file with summary, linked to original files
- Metadata is transferred

5. Dashboard / Multi-User-Interface

• Roles & Permissions:
- Access to categories, subcategories, files
- Visibility can be individually adjusted on all levels
• Filter & Search:
- By project, subproject, file, document number, keywords/metadata
• Matrix view:
- Links between original files, code, summaries, projects
• Agent control directly in the dashboard:
- Starting agents for selected files or folders
- Automatic/Manual selectable
• Sharing & rights management for multiple users
• Document numbers visible at all levels or optionally hidden

6. Workflow Example
---

1. Import files into plugin 2
2. Plugin 1: Assign document number (project, subproject, file)
3. Agent 1: Keyword analysis → Metadata in database
4. Agent 2: Extract code/concepts → new files → linked
5. Agent 3: Summary → new file → optional original text → linked
6. Dashboard:
- Matrix view of all links
- Access & filter per user
- Multi-user editing possible
7. Storage location:
- Internal database use for metadata / text
- External hard drive for large files

7. Flexibility & Expandability

• Plugins can be booked individually or in combination.
• Agent selection manually or automatically for individual files or entire folder groups
• Document numbers optional at every level, including subprojects
• Dashboard filters can be set at any category level
• Storage: internal + optional external drives
• Integration of additional agents or modules is possible

✅ With this document, developers**all information for implementation**:

- Plugins & Booking
- Knowledge Base & Storage
- Agent workflows
- Document numbers & matrix representation
- Multi-user dashboard with rights & filters
- Flexibility for automatic or manual control

If you want, I can do another one next.**compact version as a "reference checklist for developers"**create where all functions are available**a look**stand – so to speak**as a quick overview during implementation**.

Do you want me to do that?

And

Perfect, here is the**compact developer checklist**For your system – all functions, plugins, agents, storage and dashboard at a glance, ideal for quick reference during implementation:

Developer Checklist – Quick Overview

1. Plugin booking:
• Plugin 1: Receipt number generator
- Can be used autonomously
- Assignment for projects, subprojects, files, categories
- Numbers visible or hidden in the document
- Link to knowledge database
• Plugin 2: Knowledge Base + Agents
- Can be used autonomously
- Imports files, preserves folder structure
- Original files remain unchanged
- Automatically capture metadata (name, date, folder, document number optional)
• Combination: Plugin 2 + Plugin 1 → automatic integration of document numbers

2. Storage:
• Internal database: metadata, text, document numbers, links
• External hard drive: large files (videos, images, code)
• Original files are not modified

3. Document number & Matrix:
• Numbers possible at all levels
• Dashboard displays:
- Which files were included in which summaries or projects
- Traceability
• Optional: Filters for higher levels or subcategories

4. Officers:
• Agent 1 – Search / Keywords
- Selection: File, Folder, Multiple Folders, Filter/Metadata, All Files
- Automatic or manual
- Output: Keywords → Metadata + Linking
• Agent 2 – Code / Concepts
- Selection: File, Folder, Multiple Folders, Filter/Metadata
- Actions: Code, concepts, multiple criteria
- Output: new files → linked → storage location DB / external
- Save automatically or manually
• Agent 3 – Summary
- Selection: File, Folder, Multiple Folders, Filter/Metadata
- Options: Use original text Yes/No, Use template Yes/No, Save automatically Yes/No
- Output: new file → linked → copy metadata

5. Dashboard / Multi-User:
• Roles & Permissions:
- Access to parent categories, subcategories, files
- Filterable by projects, subprojects, files, document numbers, keywords

• Matrix view: Links to original files, code, summaries, projects
• Agent control directly in the dashboard
• Visibility of each layer can be optionally adjusted
• Sharing & rights management for multiple users

6. Workflow example:
1. Import files (Plugin 2)
2. Assign document number (Plugin 1)
3. Agent 1: Keyword analysis → Metadata in database
4. Agent 2: Code/Concepts → new files → linked
5. Agent 3: Summary → new file → optional original text → linked
6. Dashboard: Matrix view, filters, multi-user access
7. Storage: internal + optional external

7. Flexibility & Expandability:
• Plugins can be used individually or in combination
• Agent selection manual/automatic per file or folder
• Document numbers at all levels, including subprojects
• Dashboard filters are flexible at every category level
• Additional agents/modules can be easily integrated

This checklist serves as**compact roadmap for developers**:

- All functions at a glance
- Every level, every plugin, every agent clearly described.
- Quick implementation possible without inquiries

If you want, I can still**build a small visualization in text form**, the**Plugins, agents, dashboard and storage**how a schematic flowchart is represented – very useful for developers for quick understanding.

Do you want me to do that?

And

Great, here is the**schematic text visualization**of the entire system: plugins, agents, dashboard, storage – everything at a glance, like a flowchart for developers.

System-Flow – Textdiagramm

```
+---------------------------+
|      User Interface       |
```

```
                |  Dashboard / Multi-User   |
| - Select files/folders |
| - Filter by metadata |
| - Agent control |
| - Matrix view |
                +------------+--------------+
                             |
in
          +--------------------------------------+
          |             Plugins                  |
          +--------------------------------------+
| Plugin 1: Receipt number generator |
| - Projects / Subprojects / Files |
| - Numbers visible/hidden |
| - Link to knowledge base |
          +--------------------------------------+
| Plugin 2: Knowledge Base + Agents |
| - Importing files & folder structure|
| - Original files remain unchanged|
| - Automatically capture metadata |
          +--------------------------------------+
                             |
in
          +--------------------------------------+
| Agents |
          +--------------------------------------+
| Agent 1 – Keyword Analysis |
| - Selection: File, Folder, Metadata |
| - Automatic / Manual |
| - Output: Keywords → DB, linking|
          +--------------------------------------+
| Agent 2 – Code / Concepts |
| - Selection: File, Folder, Metadata |
| - Actions: Extract code, concepts|
| - Output: new file → linked |
| - Storage location: DB / external hard drive|
| - Automatic / Manual |
          +--------------------------------------+
| Agent 3 – Summary |
| - Selection: File, Folder, Metadata |
| - Options: Original text Yes/No, Template Yes/No |
| - Output: new file → linked |
| - Save automatically / manually |
          +--------------------------------------+
                             |
in
          +--------------------------------------+
| Storage |
```

```
        +-------------------------------------+
| - Internal: Metadata, texts, links |
| - External: large files (videos, images, code) |
| - Original files unchanged |
        +-------------------------------------+
                        |
in
        +-------------------------------------+
        |          Dashboard / Matrix          |
        +-------------------------------------+
| - Matrix: Original files → Code → Summaries → Projects |
| - Filter: Document number, project, metadata, subcategories |
| - Multi-user: Individual rights & visibility |
| - Agent control directly from the dashboard |
        +-------------------------------------+
```

✅ **Highlights for developers:**

1. **Clear separation**Plugins, agents, storage, dashboard
2. **flexibility**Manual / automatic, single file / folder / metadata
3. **Link**Every file, every output is linked via document numbers and metadata.
4. **Multi-User**Rights management, visibility at each category level
5. **Scalable**Internal and external storage, expandable as needed.

If you want, I can do something else next**a coherent "operating and implementation concept"**write that**Agent options, document numbers, storage, and dashboard as a step-by-step workflow**represents for the user.

Do you want me to do that?

And

Perfect, then here's the**complete operating and implementation concept**as a step-by-step workflow for users and developers – all interconnected, including agents, document numbers, storage, dashboard and options:

Operating and implementation concept – step-by-step

1. File preparation & plugin installation

1.1 Book plugins:
• Plugin 1: Receipt number generator
• Plugin 2: Knowledge Base + Agents
• Optionally, add plugin 2 first → add plugin 1 later
1.2 Prepare folder structure (external or internal):
• Example: Projects, sub-projects, contracts
• Original files remain unchanged
1.3 Plugin 2 automatically imports files into the internal structure:
• Capture metadata: name, date, folder
• Optional: Copy document numbers if Plugin 1 is active

2. Document Number Management (Plugin 1)

2.1 Assignment of document numbers:
• For projects, subprojects, files, categories
• Visible or hidden in the document
2.2 Matrix view in the dashboard:
• Traceability: Which files were included in which summaries/projects
2.3 Filter options:
• Main/subcategories
• Single or multiple search by document number

3. Agent Workflow (Plugin 2)

3.1 Agent 1 – Keyword Analysis
• Options: Single file, folder, multiple folders, metadata, all files
• Automatic or manual
• Output: Keywords → Metadata in database, link to original file

3.2 Agent 2 – Code / Concepts
• Options: Single file, folder, multiple folders, metadata
• Promotions:
- Extract code (one-to-one)
- Extract concepts (one-to-one)
- Other criteria possible
• Output: new file(s) → linked to original file
• Storage location: internal database or external hard drive
• Save automatically or manually

3.3 Agent 3 – Summary
• Options: Single file, folder, multiple folders, metadata
• Options:
- Copy original text: Yes/No
- Use template: Yes/No
- Auto save: Yes/No
• Output: new file → linked to original files

• Assign metadata automatically or manually

4. Dashboard & Multi-User Control

4.1 Matrix view:
• Link original files → code → summaries → projects
4.2 Filter:
• Document number, project, subproject, file, metadata, categories
4.3 Rights Management:
• Individual access per user and category/subcategory
• Visibility: optional for certain layers
4.4 Agent control:
• Start, automatic or manual directly from the dashboard
• Selection of files, folders, or metadata as a basis

5. Storage & Scalability

• Internal database: metadata, texts, links, summaries
• External hard drive: large files (videos, images, code)
• Original files unchanged
• Automatic or manual linking of the new files

6. Expansion options

• Additional agents/analysis tools can be integrated
• New criteria for Agent 2: additional types of extractable content
• Agent 3 templates are expandable
• New filters / metadata customizable in dashboard
• Combination with Plugin 1 possible at any time

7. Example workflow for one user
---
1. Prepare the folder project and import the files.
2. Assign document numbers for the project and files
3. Agent 1: Keyword analysis → Metadata in database
4. Agent 2: Extract code and concepts → new files → linked
5. Agent 3: Summary → new file → optional original text → linked
6. Dashboard:
- Matrix shows all links
- Apply filters and permissions
- Access to results for multiple users
7. Storage: internal + optional external hard drive for large data

---

✅ **Highlights for developers and users:**

- Complete **modular and flexible**
- **Automatic or manual**controllable at all levels

- **Document numbers, metadata and links**ensure traceability
- **Dashboard**for multi-user, filter and matrix view
- Flexible storage: internal + external, original files remain untouched
- **Officers 1–3**working via file, folder and metadata selection

---

If you want, I can do something else next**a "mini architecture diagram including storage, agents, plugins and dashboard"**Create a graphical summary of all the logic – ideal for presentations or developer manuals.

Do you want me to do that?

And

Great, here is the**graphic mini-architecture in text form**, which logically connects all components, storage, agents, plugins, and the dashboard. It's like a schematic diagram that developers and team members can immediately understand:

Mini-architecture – text diagram

```
                    +-------------------------+
                    |     User Interface      |
                    |  Dashboard / Multi-User  |
                    |-------------------------|
| - File/folder selection |
| - Filters / Metadata |
| - Agent control |
| - Matrix view |
                    +------------+-------------+
                                 |
in
          +-----------------------------------------+
          |                Plugins                  |
          +-----------------------------------------+
| Plugin 1: Receipt number generator |
| - Projects, subprojects, files |
| - Numbers visible/hidden |
| - Link to knowledge base |
          +-----------------------------------------+
| Plugin 2: Knowledge Base + Agents |
| - Import files, folder structure |
| - Capture metadata |
```

```
| - Original files remain unchanged |
            +----------------------------------------+
                                |
in
            +----------------------------------------+
| Agents |
            +----------------------------------------+
| Agent 1 – Search / Keyword Analysis |
| - Selection: File / Folder / Metadata |
| - Automatic / Manual |
| - Output: Keywords → DB, linking |
            +----------------------------------------+
| Agent 2 – Code / Concepts |
| - Selection: File / Folder / Metadata |
| - Actions: Extract code / concepts |
| - Output: new file(s) → linked |
| - Storage: internal / external |
            +----------------------------------------+
| Agent 3 – Summary |
| - Selection: File / Folder / Metadata |
| - Options: Original text, Template, Automatic |
| - Output: new file → linked |
            +----------------------------------------+
                                |
in
            +----------------------------------------+
| Storage |
            +----------------------------------------+
| Internal: DB → Metadata, texts, links |
| External: large files (videos, images) |
| Original files unchanged |
            +----------------------------------------+
                                |
in
            +----------------------------------------+
            |          Dashboard / Matrix            |
            +----------------------------------------+
| - Links Original → Code → Sum. |
| - Filter: Document number, Project, File |
| - Multi-User: Rights & Visibility |
| - Agent control can be directly integrated |
            +----------------------------------------+
```

✅ **Brief description:**

- **Plugins 1 & 2**form the foundation
- **Officers 1–3**Working with files, folders, metadata → creating new content

- **memory**internal/external hard drive → large files moved to external storage
- **Dashboard**Displays a matrix of all links, filters, and permissions for multi-user users.
- **Document numbers & metadata**ensure traceability and structure

---

If you want, I can now**a combined overview**do the**Plugin 1, Plugin 2, agents, document numbers, dashboard, storage, workflows and multi-user – all in a single diagram.**represents – essentially a master plan for the developers and team.

Do you want me to create this?

As text

Okay, here's the**Complete master overview as a text diagram**, which combines all plugins, agents, document numbers, dashboard, storage, workflows and multi-user structure – a comprehensive master plan:

Masterplan – Textdiagramm

```
                    +-------------------------+
                    |     User Interface      |
                    |  Dashboard / Multi-User  |
                    |-------------------------|
| - File/folder selection |
| - Filters / Metadata |
| - Agent control |
| - Matrix view |
                    +------------+-------------+
                                 |
in
          +-----------------------------------------------+
          |                  Plugins                      |
          +-----------------------------------------------+
| Plugin 1: Receipt number generator |
| - Projects / Subprojects / Files |
| - Numbers visible or hidden |
| - Link to knowledge base |
          +-----------------------------------------------+
| Plugin 2: Knowledge Base + Agents |
| - Import files, folder structure |
| - Automatically capture metadata |
| - Original files unchanged |
```

```
            +-----------------------------------------------+
                                  |
in
            +-----------------------------------------------+
| Agents |
            +-----------------------------------------------+
| Agent 1 – Search / Keyword Analysis |
| - Selection: File / Folder / Metadata / All |
| - Automatic / Manual |
| - Output: Keywords → DB, linking |
            +-----------------------------------------------+
| Agent 2 – Code / Concepts |
| - Selection: File / Folder / Metadata / Filter|
| - Actions: Code / Concepts one to one |
| - Output: new file(s) → linked |
| - Storage: internal / external |
| - Save automatically / manually |
            +-----------------------------------------------+
| Agent 3 – Summary |
| - Selection: File / Folder / Metadata / Filter|
| - Options: Original text Yes/No, Template Yes/No |
| - Output: new file → linked |
| - Save automatically / manually |
            +-----------------------------------------------+
                                  |
in
            +-----------------------------------------------+
| Storage |
            +-----------------------------------------------+
| Internal: DB → Metadata, texts, links |
| External: large files → videos, images, code |
| Original files remain unchanged |
            +-----------------------------------------------+
                                  |
in
            +-----------------------------------------------+
            |             Dashboard / Matrix              |
            +-----------------------------------------------+
| - Matrix: Original → Code → Summaries → Projects |
| - Filter: Document number, project, subproject, file, metadata |
| - Multi-user: Individual rights & visibility |
| - Agent control can be directly integrated |
| - Document numbers traceable at all levels |
            +-----------------------------------------------+
                                  |
in
            +-----------------------------------------------+
| Workflows / Options |
```

```
+-----------------------------------------------+
| 1. Import files, adopt folder structure |
| 2. Assign document numbers (projects, files) |
| 3. Agent 1: Keyword analysis → Metadata |
| 4. Agent 2: Extract code / concepts → new files |
| 5. Agent 3: Summary → New files → Optional original text |
| 6. Internal/external storage |
| 7. Dashboard: Matrix, Filters, Multi-User Access |
| 8. Automatic / Manual selectable for all agents |
| 9. Links: Original → Code → Summary → Projects |
+-----------------------------------------------+
```

✅ **Highlights:**

- Complete **integrated**Plugins, agents, document numbers, dashboard, storage, workflows
- **Flexible and scalable**: manually or automatically, any folders/files/metadata
- **Traceability**The matrix shows every link between original files and new files and projects.
- **Multi-user capable**: Rights, visibility and filters individually per level
- **Expandable**Additional agents, file types, filters, or templates are possible at any time.

If you want, I can do something else next**A compact step-by-step user manual for end users**write a manual based on this master architecture – like a “practical handbook” for daily use.

Do you want me to create this?

And

Perfect, here is the**Step-by-step user manual for end users**, based on the master architecture. It explains exactly how to use plugins, agents, document numbers and dashboards – all in a practical and comprehensible way.

End-user manual – step-by-step

1. Start & Plugin-Setup

1.1 Book plugins:

• Plugin 1 → Receipt number generator
• Plugin 2 → Knowledge Base + Agents
• Optional: Use plugin 2 first, add plugin 1 later
1.2 Prepare the folder structure:
• Examples: Projects, sub-projects, contracts
• Original files remain unchanged
1.3 Plugin 2 automatically imports files:
• Filename, folder, date → Metadata
• Optional: Use existing document numbers

2. Assign document numbers (Plugin 1)

2.1 Create a new document number:
• For projects, subprojects, files, categories
• Visible or hidden in the document
2.2 Matrix view:
• Traceability: Which documents were included in which summaries/projects?
2.3 Filter options:
• Main/subcategories
• Single or multiple search by document number

3. Use agents (Plugin 2)

3.1 Agent 1 – Keyword Analysis
• Options: Single file, folder, multiple folders, metadata, all files
• Automatic or manual
• Output: Keywords → Metadata in database, link to original file

3.2 Agent 2 – Code / Concepts
• Options: Single file, folder, multiple folders, metadata
• Promotions:
- Extract code one-to-one
- Extract concepts one-to-one
- Other criteria possible
• Output: new file(s) → linked to original file
• Storage location: internal database or external hard drive
• Save automatically or manually

3.3 Agent 3 – Summary
• Options: Single file, folder, multiple folders, metadata
• Options:
- Copy original text: Yes/No
- Use template: Yes/No
- Auto save: Yes/No
• Output: new file → linked to original files
• Assign metadata automatically or manually

4. Dashboard / Multi-user control

4.1 Matrix view:
• Link original files → code → summaries → projects
4.2 Filter:
• Document number, project, subproject, file, metadata, categories
4.3 Rights Management:
• Individual access per user and category/subcategory
• Visibility optional for each layer
4.4 Agent control:
• Start, automatic or manual directly from the dashboard
• Selection of files, folders, or metadata as a basis

5. Storage options

• Internal: Metadata, texts, links, summaries
• External: large files (videos, images, code)
• Original files remain unchanged
• Optionally save new files to an external hard drive

6. Workflow Example
---
1. Prepare the folder project and import the files.
2. Assign document numbers to projects, subprojects, and files.
3. Agent 1: Keyword analysis → Store metadata in database
4. Agent 2: Extract code/concepts → new files → linked
5. Agent 3: Summary → new file → optional original text → linked
6. Dashboard: Matrix shows all links
7. Apply filters and permissions, share results with multiple users
8. Internal + external storage for large files

7. Tips & Options

• Automatic or manual selection for each agent
• Selection of individual files, folders, multiple folders, or metadata
• Document numbers can be assigned at all levels (project, subproject, file, category)
• Templates for summaries can be used flexibly.
• Multi-user options: Rights & visibility individually per category

---

✅ **Result:**

- Fully traceable for end users
- All plugins, agents, and workflows explained
- Matrix, filters, document numbers, metadata and storage integrated
- Flexible: manual / automatic, single file / folder / metadata

---

If you want, I can do it next.**create a compact "user checklist"**, which you can simply check off to avoid forgetting any steps during use.

Do you want me to do that?

And

Great, here is the**compact user checklist**For daily use, based on the master architecture. Everything you need to check off so you don't forget a single step.

User checklist – document numbers & knowledge base

1. Plugin-Setup
[ ] Plugin 1 (Document Number Generator) activated
[ ] Plugin 2 (Knowledge Base + Agents) enabled
[ ] Folder structure prepared (projects, subprojects, contracts, etc.)

2. Import files
[ ] All original files imported into Plugin 2
[ ] Check: Files unchanged, only copied
[ ] Metadata automatically taken from filename / folder / date

3. Assign document numbers (Plugin 1)
[ ] Assign document numbers for projects, subprojects, files
[ ] Number visibility: visible / hidden
[ ] Check traceability in matrix
[ ] Filter: Apply parent/subcategory if needed

4. Deploy agents
4.1 Agent 1 – Keyword Analysis
[ ] Select file / folder / metadata
[ ] Start automatically or manually
[ ] Save keywords in knowledge base
[ ] Check links to original files

4.2 Agent 2 – Code / Concepts
[ ] Select file / folder / metadata
[ ] Select criteria: Code / Concepts / Additional filters
[ ] Save new file(s) → internal or external
[ ] Select automatic / manual save
[ ] Check links to original files

4.3 Agent 3 – Summary
[ ] Select single file, folder, or metadata
[ ] Set options: Use original text, Use template, Save automatically
[ ] Save new file → internal or external
[ ] Check links to original files

5. Dashboard & Matrix
[ ] Check matrix view: Original → Code → Summary → Projects
[ ] Set filters: Document number, Project, Subproject, File, Metadata
[ ] Set rights / visibility for multi-users
[ ] Control agents directly via dashboard

6. Check memory
[ ] Internal storage: DB → Metadata, texts, links
[ ] External storage: large files (videos, images, code)
[ ] Leave original files unchanged

7. Workflow-Check
[ ] Files imported
[ ] Assign document numbers
[ ] Agent 1 keyword analysis completed
[ ] Agent 2 Code / Concepts extracted
[ ] Agent 3 summaries created
[ ] Matrix & links checked
[ ] Rights & visibility adjusted
[ ] Internal/external storage complete

✅ **Benefits of the checklist:**

- Go through all steps systematically
- Don't forget any files or metadata.
- Using agents and plugins correctly
- Traceability, matrix functionality, and multi-user capabilities work reliably.

If you want, I can do it next.**A graphical text overview with all agent options, automatic/manual and links.**create – how to create**Complete workflow diagram for end users**, showing everything at a glance.

Do you want me to create this?